



TRABAJO FIN DE GRADO

MovieMate: app de gestión y valoración de películas y series

Murcia, a 28 de mayo de 2026

Christian Matas Conesa
24427423C
christian.m.c@um.es

Francisco García Sánchez

ÍNDICE DE CONTENIDOS

I.	Resumen.....	vii
II.	Extended abstract	viii
1	Introducción	1
2	Estado del arte	2
2.1	Análisis de la situación de partida	2
2.2	Tecnologías empleadas.....	5
2.2.1	<i>Back-end</i>	5
2.2.2	<i>Front-end</i>	6
2.2.3	Integración de datos.....	7
2.2.4	Miscelánea.....	7
3	Análisis de objetivos y metodología.....	8
3.1	Objetivos.....	8
3.2	Metodología.....	8
4	Diseño y resolución del trabajo realizado	11
4.1	Análisis	11
4.1.1	Usuarios no registrados.....	11
4.1.2	Usuarios registrados.....	12
4.2	Diseño	16
4.2.1	Modelo de datos	16
4.2.2	Diseño del <i>back-end</i>	20
4.2.3	Diseño del <i>front-end</i>	22
4.3	Implementación.....	23
4.3.1	Implementación del <i>back-end</i>	25
4.3.2	Implementación del <i>front-end</i>	33
4.4	Pruebas	42
4.5	Despliegue	44
4.5.1	Contenerización con Docker.....	45
4.5.2	Orquestación con Docker Compose	45
4.5.3	Integración continua y automatización (CI/CD)	45
4.5.4	Estrategia de ramificación	47
5	Conclusiones y vías futuras	48
5.1	Conclusiones	48
5.2	Trabajo futuro	48

6	Bibliografía	50
	Anexo A – Especificación de casos de uso	53
	A.1 Usuarios no registrados	53
	A.2 Interacción social	54
	A.3 Listas personalizadas.....	56
	A.4 Valoraciones y reseñas.....	58
	A.5 Administración y moderación	59
	Anexo B – Tabla con los <i>endpoints</i> de la aplicación	61
	Anexo C – Manual de usuario	66
	C.1 Página de descubrimiento.....	66
	C.2 Ficha de detalle de contenido	66
	C.3 Perfil de usuario	67
	C.4 Listas personalizadas.....	68
	C.5 Actividad, notificaciones y comunicación en tiempo real.....	70
	C.6 Panel de administración y moderación de contenido	71
	Anexo D - Despliegue en Kubernetes con Minikube	73

ÍNDICE DE FIGURAS

Figura 1. Aplicación web Letterboxd	2
Figura 2. Aplicación web IMDb.....	3
Figura 3. Aplicación web Serializd	4
Figura 4. Diagrama de casos de uso de un usuario no registrado	11
Figura 5. Diagrama de casos de uso de la interacción social de un usuario registrado	12
Figura 6. Diagrama de casos de uso sobre listas personalizadas de un usuario registrado	13
Figura 7. Diagrama de casos de uso de valoraciones y reseñas de un usuario registrado	14
Figura 8. Diagrama de casos de uso del administrador	15
Figura 9. Diagrama entidad-relación de MovieMate	17
Figura 10. <i>Mockup</i> de MovieMate	22
Figura 11. <i>Mockup</i> de MovieMate versión móvil.....	23
Figura 12. Arquitectura de MovieMate.....	24
Figura 13. Estructura de carpetas del <i>back-end</i> de MovieMate	26
Figura 14. Ejemplo de controlador REST (<i>FollowerController</i>) en MovieMate	27
Figura 15. Ejemplo DTO.....	28
Figura 16. <i>Endpoints</i> para gestión de seguidores en MovieMate.....	28
Figura 17. Ejemplo de servicio (<i>FollowerService</i>) en MovieMate	29
Figura 18. Ejemplo de repositorio JPA (<i>FollowerRepository</i>) en MovieMate	31
Figura 19. Configuración de seguridad.....	32
Figura 20. Árbol de directorios del proyecto <i>front-end</i> de MovieMate.....	34
Figura 21. Página de inicio de MovieMate para un usuario autenticado	35
Figura 22. Fragmento de definición de tipos <i>ContentResponse</i> y <i>UserResponse</i>	35
Figura 23. Configuración del cliente HTTP <i>apiClient</i>	36
Figura 24. Módulo <i>tmdbApi</i> con funciones tipadas para obtener contenido popular y tendencias	36
Figura 25. Definición de claves de caché de TanStack Query en <i>queryKeys</i> (extracto).....	37
Figura 26. <i>Store</i> de autenticación <i>authStore</i>	37
Figura 27. Fragmento de <i>HomePage</i>	38
Figura 28. Componente <i>ContentCarousel</i>	39
Figura 29. Hooks <i>useSyncContent</i> y <i>useCreateRating</i> para la ficha de detalle	40
Figura 30. Interfaz de MovieMate en un iPad Air y un iPhone 12 Pro	42
Figura 31. Estructura de la suite de pruebas del <i>back-end</i> de MovieMate	42
Figura 32. Fragmento de <i>ListServiceTest</i> mostrando un <i>test</i> de creación de lista	43
Figura 33. Informe de cobertura de <i>test</i> de JaCoCo	44
Figura 34. <i>Pipeline</i> CI de GitHub Actions.....	46
Figura 35. <i>Pipeline</i> CD de GitHub Actions	47
Figura 36. Página de descubrimiento de MovieMate	66
Figura 37. Ficha de detalle de contenido	67
Figura 38. Página de perfil de usuario.....	68
Figura 39. Página de listas personalizadas del usuario	69
Figura 40. Página de detalle de una lista personalizada	69
Figura 41. Página de actividad.....	70
Figura 42. Bandeja de notificaciones de MovieMate.....	71

Figura 43. Panel de administración de usuarios	72
Figura 44. Panel de administración de reportes	72
Figura 45. Formulario para reportar contenido	72

ÍNDICE DE TABLAS

Tabla 1. Resumen comparativo de las soluciones analizadas	4
Tabla 2. Plan de trabajo: planificación mensual de las tareas	9
Tabla 3. Plan de trabajo: estimación de horas por tarea	10
Tabla 4. <i>Endpoints</i> de la aplicación (verde: públicos; rojo: autenticados; azul: administrador)	61

I. Resumen

MovieMate es una plataforma web de carácter social orientada a la gestión, valoración y descubrimiento de contenido audiovisual. El sistema aborda la fragmentación funcional presente en las herramientas actuales mediante una solución unificada que permite a los usuarios registrar su historial de visionado, emitir valoraciones y establecer interacción social dentro de un único entorno.

La aplicación se implementa como un sistema *full-stack* basado en una arquitectura cliente-servidor. El *back-end* se desarrolla con **Spring Boot** siguiendo un modelo en capas que favorece la mantenibilidad y la escalabilidad, con persistencia en **PostgreSQL** y autenticación sin estado mediante **JSON Web Tokens**. El *front-end* se construye con **React** y **TypeScript**, empleando una arquitectura modular basada en componentes y mecanismos de gestión eficiente del estado mediante **TanStack Query** y **Zustand**.

La plataforma se integra con la API de *The Movie Database* (TMDB), incorporando una estrategia de caché bajo demanda que reduce la latencia y la dependencia de servicios externos. Asimismo, se implementa comunicación en tiempo real mediante **WebSockets** para la gestión de notificaciones, mejorando la capacidad de interacción del sistema.

El despliegue se realiza mediante contenedores **Docker** orquestados con Docker Compose, junto con un *pipeline* de integración continua basado en **GitHub Actions**. Como contribución, el trabajo demuestra la viabilidad de diseñar una arquitectura modular y desacoplada capaz de integrar funcionalidades sociales, consumo eficiente de APIs externas y comunicación en tiempo real en una única plataforma coherente.

II. Extended abstract

The consumption of audiovisual content has grown significantly in recent years, driven by the proliferation of streaming platforms and the widespread availability of high-speed internet access. In this context, tools dedicated to cataloguing and rating films and television series — such as Letterboxd, IMDb, or Serializd — have become widely used applications for users who wish to track their viewing history, share opinions, and discover new content within a community-driven environment.

However, these platforms present important limitations that fragment the user experience. Most of them specialize in a single type of content, either films or television series, lack advanced social features, or do not provide a unified environment in which users can manage ratings, reviews, and personal lists in an integrated manner. This fragmentation results in a suboptimal experience for users who consume both types of content and require a single platform to centralize their audiovisual activity.

MovieMate is a full-stack social web application designed to address these limitations by integrating content cataloguing, social interaction, and tracking functionalities into a unified system. The platform combines key aspects of existing solutions such as IMDb, Letterboxd, and Serializd into a single coherent environment.

It allows users to rate and review both films and television series, create and manage custom lists, follow other users, track their activity, receive real-time notifications, and access personalized content recommendations based on their viewing history.

The main objective of this bachelor's Thesis is the design and development of a responsive full-stack web application for the consultation, management, and evaluation of films and television series.

This objective is supported by several sub-objectives, including the analysis of existing platforms and their limitations, the selection of appropriate technologies, the design and implementation of a scalable system architecture, the development of a functional prototype, and its deployment in a production-like environment. External data is obtained from The Movie Database (TMDB) API, which provides real films and series metadata.

The project aims to deliver a functional prototype and does not address content distribution or streaming services.

The project follows an iterative and incremental development methodology structured into five phases: analysis, design, implementation, testing, and documentation.

The initial phase focuses on the analysis of existing platforms and the evaluation of suitable technologies for both backend and frontend development. The second phase addresses system design, including requirements definition, data modelling, API specification, and interface design. The third phase corresponds to full-stack implementation. The fourth phase focuses on testing and validation using automated unit tests and integration-level verification scripts. The final phase covers documentation and project reporting.

Version control is managed using Git, following a structured branching strategy. Development is carried out in a dedicated development branch, while the main branch is reserved for stable releases. Merges are only performed after successful validation through an automated CI/CD pipeline, ensuring that the main branch always reflects a stable and deployable version of the application.

The server-side component of MovieMate is implemented using Java and Spring Boot, following a layered architecture that separates presentation, business logic, persistence, and security concerns. This design improves maintainability, scalability, and testability of the system.

Data persistence is handled using a relational database, structured to support complex relationships between users, content, ratings, and social interactions.

Security is implemented using a stateless authentication model based on JSON Web Tokens (JWT). Upon successful authentication, the system issues a signed token that is included in subsequent requests for authorization purposes. Passwords are securely stored using hashing techniques, and access control is enforced based on user roles.

Content metadata is obtained from The Movie Database (TMDB) API using an on-demand synchronization strategy. Content is retrieved when first requested and stored locally to reduce external dependencies and improve performance. A caching mechanism ensures that frequently accessed content remains up to date while minimizing unnecessary external requests.

The system implements real-time communication using WebSockets, enabling server-to-client notifications without polling. Events such as follows, likes, and comments trigger instant updates that are delivered to the corresponding users in real time.

Additional back-end features include a gamification subsystem with badge types awarded automatically based on user activity milestones, a personalized recommendation engine that drives the user's top genre from their rating history and queries TMDB's discovery endpoint for highly rated titles in that genre, an episode-level tracking system for television series, a content reporting and moderation system managed through the admin panel, and a DataSeeder component that populates a fresh deployment with a realistic set of test users, content, ratings, and social connections.

The client-side application is developed as a Single Page Application using React and TypeScript. It follows a modular architecture based on functional domains, promoting separation of concerns and maintainability.

State management is divided between local and server-driven state, ensuring efficient handling of asynchronous data and reducing redundant network requests.

Global synchronous state — primarily the authenticated user's session data and JWT token — is managed with Zustand. Server-derived asynchronous state is managed with TanStack Query v5, which handles the full lifecycle of HTTP requests: caching results under a structured key hierarchy, automatically revalidating stale data, and performing selective cache invalidation

after mutations. All HTTP communication is channeled through a global Axios instance configured with a request interceptor that attaches the JWT Authorization header to every authenticated request.

Navigation is handled by React Router v6 with a centralized route declaration in App.tsx. Routes are divided into public routes, accessible without authentication and protected routes guarded by a PrivateRoute component that redirects unauthenticated visitors to the login screen.

The application comprises the following main pages. The home page presents trending content sourced from TMDB, a personalized recommendations carousel labelled "For You", and a section of suggested users to follow. The discovery page provides a debounced search bar that queries both films and series simultaneously, interleaving results by relevance. The content detail page — the most complex view — displays the backdrop image, poster, metadata, a five-star rating widget that supports both creation and in-place editing of existing ratings, a dialogue for adding the title to any personal list, the cast gallery, and for television series an episode tracking accordion. The profile page is organized into four tabs: activity timeline, lists, advanced statistics including rating distribution by score value and top genres and earned badges. The lists pages allow users to create, edit, and delete custom lists, browse their contents, and leave public comments. The activity page displays a chronological feed of actions performed by followed users. Real-time notifications are reflected immediately in the navigation badge through the WebSocket connection established by the root layout component.

The visual identity follows a dark theme defined through Tailwind CSS 4 custom tokens: a deep navy-to-charcoal background scale, a golden accent colour used for all interactive elements, and a dual typography system pairing Playfair Display for titles with DM Sans for body text. Accessible components from Radix UI and Shadcn/ui ensure compatibility with assistive technologies. The interface is fully responsive: a fixed bottom navigation bar replaces the sidebar on mobile viewports, and all content grids adapt fluidly to any screen size.

The quality of the server-side code is validated by a suite of 340 automated unit tests written with JUnit 5 and Mockito. Tests follow the Arrange-Act-Assert pattern and make extensive use of mock objects to isolate the unit under test from its dependencies.

The system is validated through automated unit testing and integration-level verification scripts that ensure the correct behaviour of the backend API.

The application is containerized to ensure portability and reproducibility, and continuous integration mechanisms automate testing and build verification during development.

The system is designed with deployment in mind, supporting scalable and reproducible environments.

MovieMate delivers a fully functional social platform for audiovisual content management that meets all the objectives established at the outset of the project. The application successfully integrates content discovery, personal cataloguing, social interaction, real-time communication, and personalized recommendations in a single, coherent system accessible

from any device. From a technical standpoint, the project demonstrates the practical application of a broad set of modern web development technologies and patterns: RESTful API design, stateless JWT-based security, on-demand external API synchronization, reactive client-side state management, WebSocket-based real-time communication, containerized deployment, and automated CI/CD pipelines. The layered back-end architecture and the feature-based front-end organization proved effective choices at this scale, promoting maintainability and separation of concerns throughout the development lifecycle.

Future work includes improving the recommendation system using collaborative filtering or machine learning techniques, developing native mobile applications, integrating streaming availability information, and evolving the system towards a microservices-based architecture for improved scalability.

1 Introducción

El consumo de contenido audiovisual ha experimentado un crecimiento significativo en los últimos años, impulsado por la proliferación de plataformas de *streaming* y por la generalización del acceso a internet de alta velocidad. Este contexto ha favorecido la aparición de herramientas digitales orientadas a la gestión, valoración y descubrimiento de películas y series, que permiten a los usuarios registrar su actividad de visionado, compartir opiniones y acceder a recomendaciones basadas en la interacción con otros usuarios.

Entre las plataformas más relevantes en este ámbito se encuentran Letterboxd [1], IMDb [2] y Serializd [3], cada una de ellas con un enfoque particular. Mientras que IMDb se centra en la provisión de información y metadatos, Letterboxd prioriza la dimensión social en torno al cine, y Serializd está orientada al seguimiento de series de televisión. No obstante, estas soluciones presentan limitaciones significativas, ya que no ofrecen un entorno unificado que permita gestionar de forma conjunta distintos tipos de contenido audiovisual ni integrar de manera completa funcionalidades sociales, de valoración y de organización personal.

Como consecuencia, los usuarios que consumen tanto películas como series se ven obligados a utilizar múltiples plataformas, lo que introduce una fragmentación tanto a nivel funcional como en la experiencia de usuario. Esta fragmentación dificulta la gestión coherente del historial de visionado, la centralización de valoraciones y la interacción social en un único entorno.

En respuesta a esta problemática, el presente Trabajo de Fin de Grado (TFG) aborda el diseño y desarrollo de **MovieMate**, una plataforma web de carácter social que integra en un único sistema la gestión, valoración y descubrimiento de contenido audiovisual. La aplicación permite a los usuarios registrar su historial de visionado, emitir valoraciones y reseñas, organizar contenido en listas personalizadas e interactuar con otros usuarios mediante mecanismos sociales, configurando una red temática centrada en el contenido audiovisual.

Desde una perspectiva técnica, el proyecto se orienta al diseño e implementación de una aplicación web *full-stack* basada en una arquitectura moderna, que integra persistencia de datos, comunicación con servicios externos, mecanismos de autenticación segura y capacidades de interacción en tiempo real. De este modo, se abordan de forma conjunta distintos retos propios del desarrollo de sistemas web actuales, tales como la escalabilidad, la mantenibilidad y la integración de múltiples componentes tecnológicos.

El resto del documento se organiza de la siguiente manera. El **Capítulo 2 (Estado del arte)** presenta el estado del arte, analizando las principales plataformas existentes y las tecnologías relevantes en este dominio. El **Capítulo 3 (Análisis de objetivos y metodología)** describe los objetivos del proyecto y la metodología de trabajo seguida. El **Capítulo 4 (Diseño y resolución del trabajo realizado)** detalla el diseño y la implementación del sistema, incluyendo el modelo de datos, la arquitectura del *back-end* y del *front-end*, así como las pruebas realizadas y el proceso de despliegue. Finalmente, el **Capítulo 5 (Conclusiones y vías futuras)** recoge las conclusiones del trabajo y las líneas de trabajo futuro identificadas.

2 Estado del arte

En este capítulo se analiza el contexto previo al desarrollo de **MovieMate**, incluyendo tanto las soluciones existentes para la gestión y valoración de contenido audiovisual como las tecnologías consideradas para su implementación.

2.1 Análisis de la situación de partida

Actualmente, existen numerosas aplicaciones y plataformas web destinadas a la consulta, organización y valoración de películas y series. La digitalización del consumo audiovisual y el auge de los servicios de *streaming* han impulsado la aparición y consolidación de herramientas digitales orientadas a la gestión y descubrimiento de contenido audiovisual [4]. Para el análisis de la situación de partida, se han seleccionado tres aplicaciones representativas: **Letterboxd**, **IMDb** y **Serialized**. La elección de estas plataformas se ha basado en su relevancia y popularidad en el sector, confirmada por su presencia destacada en rankings de aplicaciones y artículos especializados [5]. Además, se ha buscado una diversidad de enfoques: Letterboxd se centra en el aspecto social y la crítica comunitaria, IMDb actúa como una base de datos global con un componente social secundario, y Serialized se especializa específicamente en el seguimiento de series de televisión, lo que permite obtener una visión amplia de las opciones disponibles para los usuarios.

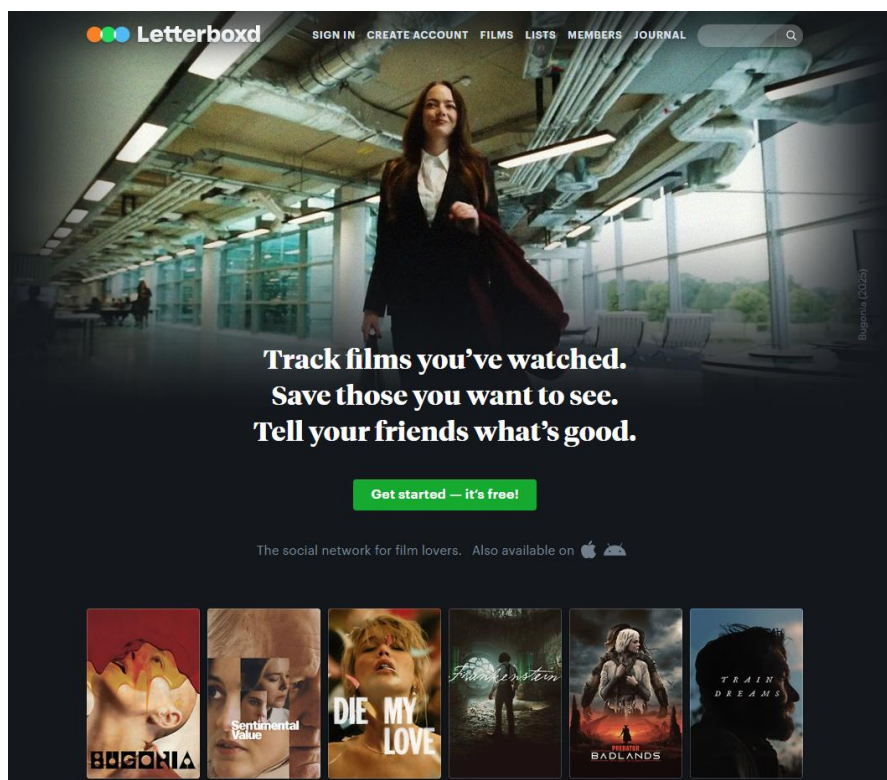


Figura 1. Aplicación web Letterboxd

En la evaluación, se han considerado las siguientes características clave, justificadas por ser los pilares fundamentales del objetivo de MovieMate:

- **Gestión de listas personales:** Capacidad para crear, editar y organizar listas personalizadas de películas y series (p. ej., "Favoritas", "Por ver").

- **Sistema de valoración y diario:** Funcionalidades para puntuar contenido y registrar la fecha de visualización (diario personal).
- **Enfoque de red social:** Capacidad para seguir a otros usuarios, comentar sus actividades y descubrir contenido a través de su actividad.
- **Experiencia de usuario y diseño:** Facilidad de uso, diseño visual, claridad en la presentación de la información y adaptabilidad a diferentes dispositivos (diseño *responsive*).

Letterboxd [6] (ver Figura 1) es una red social centrada en el cine que permite a los usuarios registrar las películas que ven, crear listas personalizadas, escribir críticas y seguir la actividad de otros cinéfilos. Su interfaz presenta un diseño minimalista orientado al contenido generado por la comunidad, lo que favorece la interacción social. Sin embargo, la ausencia de soporte equivalente para series de televisión limita su aplicabilidad como solución integral.

IMDb (Internet Movie Database) es una de las bases de datos en línea más grandes y populares sobre cine y televisión. Además de proporcionar información técnica y artística exhaustiva, permite a los usuarios puntuar títulos, escribir críticas y mantener una lista personal de "Watchlist". Si bien su componente social existe, está menos desarrollado que en Letterboxd, y su interfaz puede resultar sobrecargada de información (ver Figura 2) para el usuario que busca una experiencia simple de gestión y seguimiento.

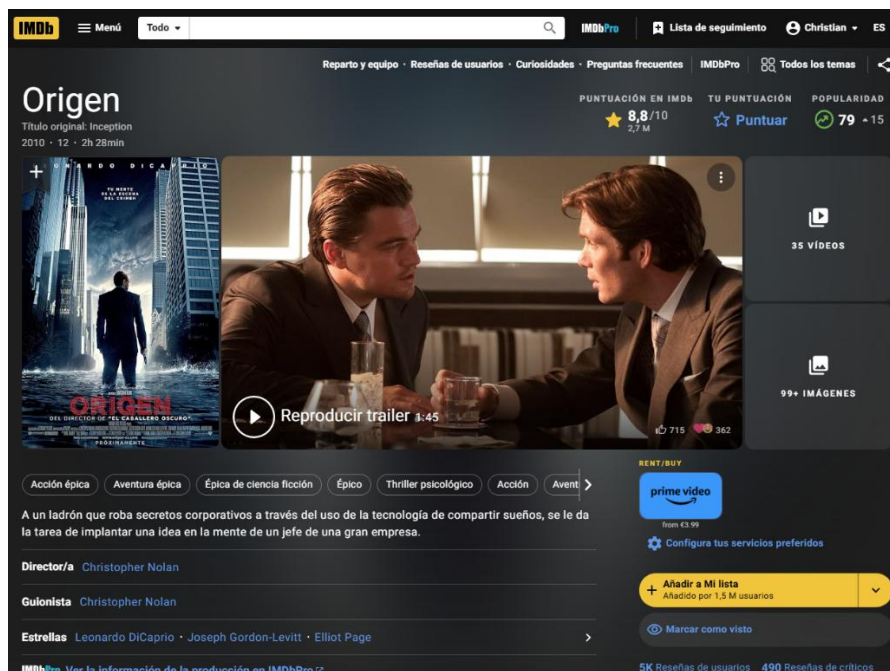


Figura 2. Aplicación web IMDb

Serializd es una aplicación relativamente nueva (2023) que adopta el modelo y la filosofía de Letterboxd pero aplicada exclusivamente al mundo de las series de televisión. Los usuarios pueden marcar episodios vistos, puntuar temporadas completas o episodios individuales, y seguir a otros usuarios para descubrir nuevas series (ver Figura 3). Su principal limitación es,

precisamente, su especialización, lo que obliga a los usuarios interesados tanto en cine como en televisión a utilizar dos aplicaciones diferentes.

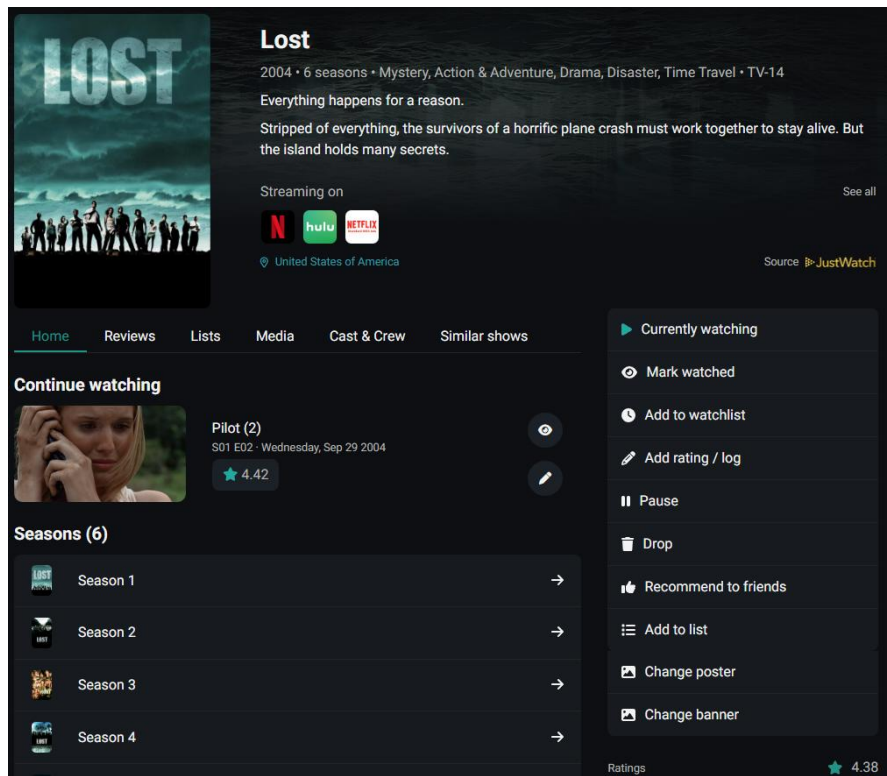


Figura 3. Aplicación web Serializd

Tabla 1. Resumen comparativo de las soluciones analizadas

Aplicación	Gestión de Listas	Valoración y Diario	Red Social	Experiencia de Usuario	Cine/ Series
Letterboxd	Listas personalizadas, watchlist, vistas	Diario personal, puntuación por película, reseñas	Seguir usuarios, comentarios, likes	Interfaz minimalista	Solo Cine
IMDb	Watchlist básica, listas personalizadas limitadas	Puntuación global, reseñas de usuarios	Sistema de reseñas, perfil básico	Interfaz funcional pero sobrecargada de información	Cine y series
Serializd	Listas personalizadas, seguimiento por episodios	Puntuación por episodio/temporada, diario de visionado	Seguir usuarios, comentarios en episodios	Interfaz inspirada en Letterboxd	Solo Series
MovieMate	Listas personalizadas completas, watchlist, categorías múltiples	Diario personalizado, puntuación flexible, reseñas	Seguir usuarios, comentarios, reseñas	Interfaz unificada e intuitiva, diseño adaptable	Cine y series

La aplicación desarrollada en este TFG, **MovieMate**, pretende integrar las mejores características de las soluciones analizadas, ofreciendo una experiencia unificada para la

gestión tanto de películas como de series. Propone una visión centrada en la usabilidad y la interacción social, con un diseño intuitivo y adaptable. En la Tabla 1 se muestra un resumen comparativo de las soluciones analizadas. Como se observa, ninguna de las soluciones analizadas proporciona una integración completa de todas las funcionalidades consideradas, lo que refuerza la motivación del desarrollo de MovieMate.

2.2 Tecnologías empleadas

En este apartado se describen las tecnologías seleccionadas para el desarrollo de MovieMate, justificando su elección en función de los requisitos del sistema y de las características del dominio del problema.

2.2.1 Back-end

Para el desarrollo del servidor, se ha elegido el **framework Spring Boot** [7] junto con el lenguaje de programación **Java** [8], debido a su madurez, amplio ecosistema y capacidad para facilitar la construcción de aplicaciones escalables basadas en arquitecturas en capas. Spring Boot es un *framework* que simplifica enormemente el desarrollo de aplicaciones basadas en Spring, proporcionando configuración automática y permitiendo crear aplicaciones listas para producción con mínimo esfuerzo. Java, como lenguaje de programación orientado a objetos, robusto y con un amplio ecosistema, garantiza escalabilidad, mantenibilidad y alto rendimiento, siendo una elección muy extendida en el desarrollo de *enterprise applications*.

Como sistema de gestión de bases de datos se utilizará **PostgreSQL** [9]. Se ha optado por esta base de datos relacional de código abierto debido a su robustez, cumplimiento de estándares SQL y su excelente rendimiento para cargas de trabajo complejas. PostgreSQL ofrece características avanzadas como tipos de datos complejos, consultas JSON nativas y una fuerte consistencia transaccional, lo que la hace ideal para gestionar las relaciones estructuradas entre usuarios, películas, series, listas y valoraciones en MovieMate. Frente a alternativas no relacionales, se ha optado por un modelo relacional debido a la naturaleza estructurada de los datos y a la necesidad de garantizar consistencia en las relaciones entre entidades.

La comunicación entre el *front-end* y el *back-end* se realizará a través de una **API (Application Programming Interface) RESTful (representational state transfer)** [11], siguiendo los principios arquitectónicos REST para garantizar una interfaz coherente y sin estado. Este enfoque permite una separación clara entre cliente y servidor, facilitando la escalabilidad y la interoperabilidad del sistema.

En el desarrollo concreto del *back-end* se han utilizado distintos módulos del ecosistema de Spring Boot que facilitan la implementación de las distintas capas de la aplicación. Para la construcción de la API REST se ha empleado el módulo `spring-boot-starter-web`, que proporciona las funcionalidades necesarias para definir controladores mediante anotaciones y gestionar peticiones HTTP siguiendo el modelo MVC (Modelo-Vista-Controlador). De este modo, los distintos endpoints del sistema pueden exponerse de forma estructurada y coherente.

Para la persistencia de datos se ha utilizado `spring-boot-starter-data-jpa`, que implementa la especificación JPA (*Java Persistence API*) y permite realizar el mapeo objeto-relacional entre las entidades del dominio y las tablas de la base de datos. Esto facilita la abstracción del acceso a datos mediante repositorios, reduciendo la complejidad del código y mejorando su mantenibilidad. La conexión con PostgreSQL se realiza a través del *driver* oficial incluido como dependencia del proyecto.

En el ámbito de la seguridad se ha integrado `spring-boot-starter-security`, que proporciona los mecanismos de autenticación y autorización de la aplicación. Sobre esta base se ha implementado un sistema de autenticación basado en **JSON Web Tokens (JWT)** [10], utilizando la librería `JWT` para la generación y validación de los tokens. Este enfoque permite mantener una arquitectura sin estado (*stateless*), adecuada para una API REST moderna.

Asimismo, se ha incorporado `spring-boot-starter-validation` para la validación de datos de entrada, garantizando la integridad de la información recibida por los *endpoints*. Para la documentación automática de la API se ha empleado `SpringDoc OpenAPI`, que genera una interfaz interactiva compatible con el estándar **OpenAPI** [12]/**Swagger** [13], facilitando la exploración y prueba de los servicios desarrollados.

Finalmente, se ha incluido el módulo `Actuator` para exponer información de monitorización y estado de la aplicación, así como dependencias específicas para pruebas automatizadas, incluyendo una base de datos en memoria **H2** [14] para el entorno de *testing* y herramientas de análisis de cobertura como **JaCoCo** [15].

2.2.2 Front-end

El *front-end* de la aplicación se implementa utilizando **React** [16] junto con **TypeScript** [17], seleccionados por su capacidad para construir interfaces complejas basadas en componentes reutilizables y tipado estático. React es una biblioteca de JavaScript ampliamente adoptada que permite crear interfaces de usuario interactivas y basadas en componentes reutilizables. TypeScript, al añadir tipado estático, contribuye a la detección temprana de errores, mejora la mantenibilidad del código y proporciona una mejor experiencia de desarrollo, lo que es especialmente valioso en proyectos de mediana y gran escala.

Para el diseño de la interfaz de usuario y garantizar que sea completamente *responsive*, se empleará **Tailwind CSS** [18]. Tailwind es un *framework* de CSS de utilidad que permite construir diseños personalizados directamente en el marcado, agilizando el desarrollo de interfaces modernas y adaptables a cualquier tamaño de pantalla.

Para la gestión del estado de la aplicación se han combinado dos herramientas complementarias. **Zustand** [19] se emplea como store global ligero para el estado sincrónico, como los datos del usuario autenticado y el token de sesión. **TanStack Query** [20] gestiona el estado asíncrono derivado de peticiones al servidor, automatizando el ciclo de vida de las solicitudes, el almacenamiento en caché de resultados y la revalidación de datos cuando estos quedan desactualizados, lo que reduce considerablemente la complejidad del código de acceso a datos en los componentes.

La comunicación HTTP con el *back-end* se centraliza a través de **Axios** [21], configurado con una instancia global que incorpora automáticamente las cabeceras de autorización en las peticiones autenticadas. La navegación entre páginas se gestiona mediante **React Router v6** [22], que soporta rutas anidadas y permite proteger las rutas que requieren sesión activa mediante componentes de guardia.

Los componentes de interfaz se construyen sobre **Radix UI** [23] y su capa de estilos **Shadcn/ui** [24], que proporcionan elementos accesibles y personalizables compatibles con Tailwind CSS. Las notificaciones emergentes se implementan con la librería `Sonner`. Por último, para la comunicación en tiempo real se integra **SockJS** [25] junto con el protocolo **STOMP (Simple/Streaming Text Oriented Messaging Protocol)** [26], permitiendo al cliente recibir

eventos del servidor de forma asíncrona a través de una conexión **WebSocket** [27] persistente durante toda la sesión.

2.2.3 Integración de datos

Para poblar la base de datos de la aplicación con información actualizada de películas y series, se integrará **The Movie Database (TMDB)** [28]. TMDB es una API gratuita y muy completa que proporciona un extenso repositorio de metadatos, imágenes y puntuaciones sobre contenido audiovisual, siendo la opción estándar *de facto* para este tipo de proyectos.

2.2.4 Miscelánea

Además de las tecnologías mencionadas, se han empleado diversas herramientas que han facilitado el desarrollo y la gestión del proyecto:

- Visual Studio Code [29]: Editor de código fuente para el desarrollo.
- Git [30] & GitHub [31]: Para el control de versiones y alojamiento del repositorio.
- Postman [32]: Para la prueba y documentación de los *endpoints* de la API REST.
- pgAdmin 4 [33]: Para la gestión y visualización de las tablas de la base de datos.
- Dbeaver [34]: Para la elaboración del diagrama entidad-relación.
- ChatGPT [35]: Apoyo para la programación, revisión de la arquitectura y documentación técnica.
- JUnit [36] & Mockito [37]: *Frameworks* utilizados para la implementación de pruebas unitarias del *back-end*, permitiendo validar la lógica de negocio de forma aislada mediante simulación de dependencias.
- Docker [38]: Plataforma de contenerización utilizada para empaquetar la aplicación *back-end* junto con sus dependencias en imágenes portables y reproducibles.
- Docker Compose [39]: Herramienta empleada para la orquestación de múltiples contenedores, permitiendo ejecutar conjuntamente el *back-end* y la base de datos PostgreSQL mediante una única configuración declarativa.
- GitHub Actions [40]: Servicio de integración continua utilizado para automatizar la ejecución de pruebas, construcción del proyecto y generación de la imagen Docker en cada actualización del repositorio.
- Claude [41]: IA usada para apoyo en la escritura de código, elaboración de *mockups*, depuración de errores y redacción de fragmentos de documentación
- Kubernetes [42]: Es una plataforma de orquestación de contenedores de código abierto que automatiza el despliegue, el escalado y la gestión de aplicaciones contenerizadas.
- Minikube [43]: Esta herramienta levanta un clúster Kubernetes de un único nodo directamente en el equipo del desarrollador, simulando un entorno de producción real con un coste operativo mínimo.

3 Análisis de objetivos y metodología

En esta sección se describen los objetivos que se persiguen con el desarrollo de este proyecto, así como la metodología empleada (tareas y temporalización) para alcanzar estos objetivos.

3.1 Objetivos

El objetivo general que se plantea en este Trabajo Fin de Grado (TFG) es el desarrollo de una aplicación Web adaptable (responsiva) para la consulta, gestión y valoración de películas y series. Este objetivo se puede dividir en los siguientes subobjetivos específicos:

- (OB1) Identificar las tecnologías y herramientas más apropiadas para acometer el trabajo.
- (OB2) Determinar las principales propiedades y carencias de las soluciones actuales en el dominio de la aplicación.
- (OB3) Desarrollar una aplicación que subsane las carencias identificadas satisfaciendo los principales estándares de calidad en el desarrollo de software.
- (OB4) Desplegar la aplicación en un escenario de producción.
- (OB5) Documentar el trabajo realizado.

Es importante señalar que en este trabajo se persigue elaborar un primer prototipo de esta aplicación y no pretende abordar aspectos como distribución/*streaming* de contenido. Se integrarán APIs externas (TMDB), recopilando datos para poblar el prototipo. Por otro lado, también se establece como objetivo de este TFG la realización de una memoria en que se describan las principales características del trabajo realizado.

3.2 Metodología

A continuación, se enumeran las etapas en las que se ha dividido la realización del TFG. Para cada tarea se establecen las subtareas específicas y los objetivos abordados.

Tarea 1. Análisis preliminar

Objetivo(s): OB1, OB2

- 1.1. Consultar aplicaciones similares en internet.
- 1.2. Estudiar el estado del arte sobre herramientas para la gestión de películas y series para identificar funcionalidades comunes.
- 1.3. Estudiar las tecnologías para el desarrollo de un *front-end* adaptable (responsivo): React, TypeScript, Tailwind CSS.
- 1.4. Estudiar las tecnologías para el desarrollo del back-end: Java, Spring, servicios REST.
- 1.5. Estudiar herramientas y librerías que puedan ser útiles para el trabajo.

Tarea 2. Análisis y diseño del sistema

Objetivo(s): OB2, OB3

- 2.1. Definir el alcance del sistema.
- 2.2. Establecer los requisitos funcionales y no funcionales del sistema.
- 2.3. Diseñar la base de datos del sistema.
- 2.4. Diseñar el componente *back-end* del sistema a partir de servicios REST.
- 2.5. Diseñar el componente *front-end* del sistema siguiendo un modelo responsivo.

Tarea 3. Desarrollo del sistema

Objetivo(s): OB3

- 3.1. Crear el esquema de la base de datos.

- 3.2. Desarrollar los servicios que constituyen la lógica de negocio (*back-end*).
- 3.3. Desarrollar los componentes que constituyen la interfaz de usuario adaptable (*responsive*) (*front-end*).
- 3.4. Conectar el *front-end* y el *back-end*.

Tarea 4. Verificación y validación

Objetivo(s): OB3, OB4

- 4.1. Verificar el sistema frente a los requisitos especificados.
- 4.2. Definir los casos de prueba.
- 4.3. Poblar la base de datos con conjunto de datos necesarios para las pruebas.
- 4.4. Validar el sistema a partir de los casos de prueba definidos.
- 4.5. Desplegar el sistema en un entorno de producción.

Tarea 5. Documentación del sistema

Objetivo(s): OB5

- 5.1. Reuniones periódicas con el tutor para revisar el estado del Trabajo Fin de Grado.
- 5.2. Documentar internamente cada uno de los componentes software desarrollados.
- 5.3. Elaborar la memoria del Trabajo Fin de Grado.
- 5.4. Elaborar la presentación del Trabajo Fin de Grado.

En la Tabla 2 se muestra la planificación mensual para las distintas tareas y subtareas a realizar, mientras que en la Tabla 3 se puede ver una estimación de las horas que se van a invertir en cada tarea y subtarea.

Tabla 2. Plan de trabajo: planificación mensual de las tareas

Tarea	10/25	11/25	12/25	01/26	02/26	03/26	04/26	05/26	06/26
T1		X	X						
1.1		X							
1.2		X							
1.3		X	X						
1.4			X						
1.5			X						
T2		X	X	X					
2.1		X							
2.2		X							
2.3		X	X						
2.4		X	X	X					
2.5		X	X	X					
T3			X	X	X	X	X		
3.1			X						
3.2			X	X	X				
3.3				X	X	X			
3.4					X	X	X		
T4				X	X	X	X		
4.1				X					
4.2				X	X				
4.3					X	X			
4.4						X			
4.5							X		
T5		X	X	X	X	X	X	X	

5.1		X	X	X	X	X	X	X	
5.2			X	X	X				
5.3		X	X	X	X	X	X	X	
5.4							X	X	

Tabla 3. Plan de trabajo: estimación de horas por tarea

Tarea	Descripción	Horas estimadas	Horas reales
T1	Análisis preliminar	33	36
1.1	Consultar apps similares	2	3
1.2	Estudiar estado del arte	4	5
1.3	Estudio tecnologías front-end	10	9
1.4	Estudio tecnologías back-end	12	12
1.5	Estudio herramientas y librerías	5	5
T2	Análisis/diseño sistema	40	42
2.1	Definir alcance del sistema	2	2
2.2	Establecer requisitos	5	6
2.3	Diseñar la BBDD	3	4
2.4	Diseñar back-end	10	9
2.5	Diseñar front-end	20	21
T3	Desarrollo de sistema	155	178
3.1	Crear esquema de la BBDD	5	6
3.2	Desarrollar el back-end	55	62
3.3	Desarrollar el front-end	80	88
3.4	Conectar back-end y front-end	15	22
T4	Verificación/Validación	33	37
4.1	Verificar los requisitos	2	3
4.2	Definir los casos de prueba	5	5
4.3	Poblar la base de datos	10	11
4.4	Validar el sistema	13	14
4.5	Desplegar el sistema	4	4
T5	Documentación	44	55
5.1	Reuniones con tutor	6	5
5.2	Documentar el desarrollo	8	9
5.3	Elaborar la memoria	24	35
5.4	Elaborar la presentación	6	6
Total		305	348

4 Diseño y resolución del trabajo realizado

En este capítulo se detalla el proceso de análisis, diseño e implementación de la aplicación MovieMate, poniendo especial énfasis en las decisiones técnicas adoptadas y en la arquitectura del sistema. Se describen los principales elementos que componen la solución, así como los mecanismos empleados para dar soporte a las funcionalidades definidas en capítulos anteriores.

4.1 Análisis

En este apartado se identifican y estructuran los principales casos de uso del sistema, organizados en función del tipo de usuario y de las funcionalidades clave de la plataforma. Este análisis permite definir de manera precisa los requisitos funcionales del sistema y sirve como base para las decisiones de diseño adoptadas en las fases posteriores.

El sistema distingue entre usuarios no registrados, usuarios registrados y administrador, cada uno con distintos niveles de acceso y capacidades. Esta diferenciación permite establecer restricciones de acceso claras y definir distintos flujos de interacción, alineados con el objetivo de proporcionar una plataforma social estructurada y segura.

Los casos de uso descritos en este apartado representan el alcance funcional general del sistema y se muestran de forma resumida mediante los diagramas correspondientes. Para proporcionar una especificación más detallada, cada caso de uso ha sido documentado mediante una plantilla formal que incluye actores, objetivos y resultados esperados. Esta documentación se recoge en el Anexo A – Especificación de casos de uso.

4.1.1 Usuarios no registrados

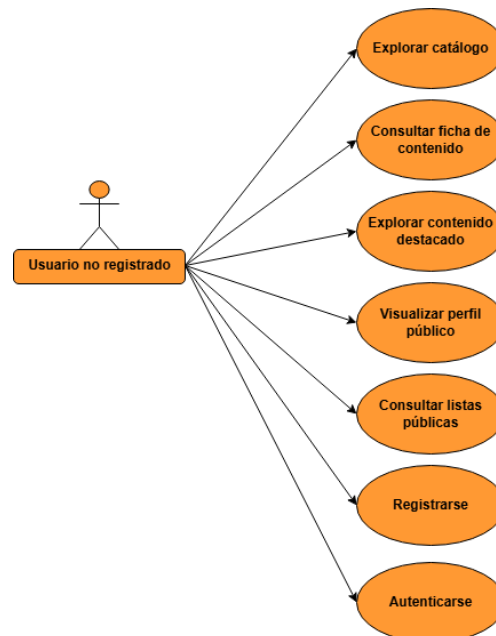


Figura 4. Diagrama de casos de uso de un usuario no registrado

Los usuarios no registrados tienen acceso a un subconjunto limitado de funcionalidades orientadas principalmente a la exploración de contenido y a la consulta de información pública como se muestra en la Figura 4. Sus principales casos de uso son los siguientes:

- Explorar el catálogo de películas y series.
- Consultar ficha de contenido de películas y series, incluyendo información descriptiva y valoraciones agregadas.
- Explorar contenido popular o destacado, accediendo a listados generales de películas y series sin necesidad de autenticarse.
- Visualizar perfiles públicos de otros usuarios, incluyendo su nombre, imagen de perfil, biografía y listas públicas, sin posibilidad de interacción directa más allá de la consulta.
- Consultar listas públicas creadas por otros usuarios, incluyendo su contenido y las valoraciones asociadas.
- Iniciar el proceso de registro para crear una cuenta nueva.
- Acceder al formulario de inicio de sesión si ya disponen de credenciales

Estas funcionalidades permiten a los usuarios explorar el sistema sin autenticación, actuando como mecanismo de captación y facilitando la transición hacia usuarios registrados para acceder a funcionalidades avanzadas.

4.1.2 Usuarios registrados

4.1.2.1 Interacción social

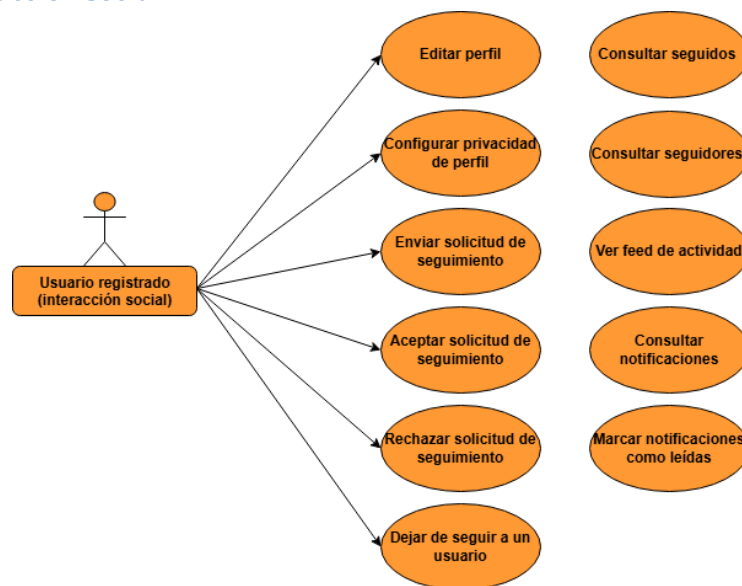


Figura 5. Diagrama de casos de uso de la interacción social de un usuario registrado

Los usuarios registrados disponen de un conjunto ampliado de funcionalidades que abarcan la gestión del perfil, la interacción social y la organización del contenido, constituyendo el núcleo funcional de la plataforma como se puede observar en la Figura 5.

Gestión de perfil:

- Gestionar su cuenta de usuario, incluyendo la actualización de datos básicos de perfil como nombre, nombre de usuario, dirección de correo electrónico, imagen de perfil y biografía.
- Configurar privacidad del perfil.

Ámbito social:

- Solicitud de seguimiento a otros usuarios de la plataforma para mantenerse al día de su actividad pública, incluyendo nuevas listas, valoraciones y otras acciones relevantes.
- Aceptar solicitud de seguimiento.
- Rechazar solicitud de seguimiento.
- Dejar de seguir a un usuario cuando se desea dejar de recibir información sobre su actividad.
- Consultar la lista de seguidos, tanto en el propio perfil como en el de otros usuarios cuando la visibilidad lo permita.
- Consultar la lista de seguidores, tanto en el propio perfil como en el de otros usuarios cuando la visibilidad lo permita.

Actividad y notificaciones:

- Acceder a un *feed* de actividad personalizado, donde se muestran acciones recientes de los usuarios seguidos, como nuevas valoraciones, listas creadas o contenido añadido a listas.
- Consultar notificaciones.
- Marcar notificaciones como leídas.

4.1.2.2 Listas personalizadas

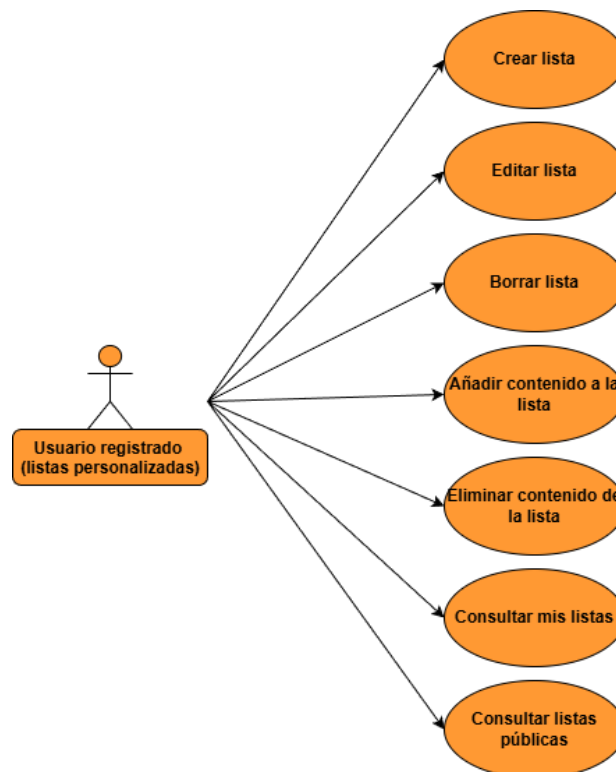


Figura 6. Diagrama de casos de uso sobre listas personalizadas de un usuario registrado

La gestión de listas personalizadas constituye una de las funcionalidades centrales de la plataforma (ver Figura 6), permitiendo a los usuarios organizar contenido de forma flexible según sus preferencias. Los casos de uso son:

- Crear listas personalizadas, indicando un título, descripción, tipo de lista y nivel de visibilidad (por ejemplo, listas públicas o privadas).
- Editar una lista existente, modificando su nombre, descripción, tipo o parámetros de visibilidad.
- Borrar una lista existente propia.
- Añadir contenido a una lista concreta, seleccionando películas o series previamente consultadas en la aplicación.
- Eliminar contenido de una lista, manteniendo el resto de la lista intacta.
- Consultar mis propias listas.
- Consultar listas públicas de otros usuarios.

Estas funcionalidades proporcionan un mecanismo estructurado y flexible para la organización de contenido, facilitando tanto el uso personal como la compartición dentro del entorno social de la plataforma.

4.1.2.3 Valoraciones y reseñas

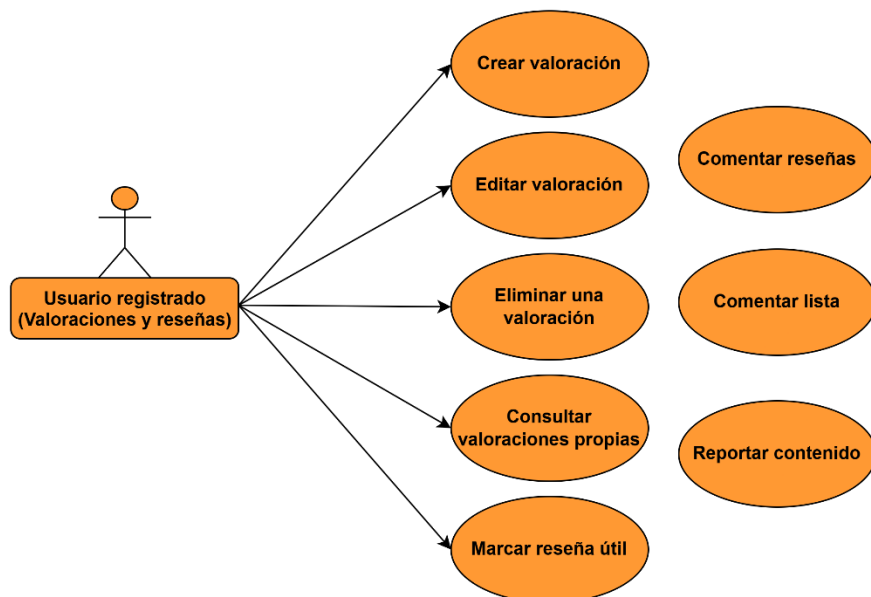


Figura 7. Diagrama de casos de uso de valoraciones y reseñas de un usuario registrado

El sistema de valoraciones y reseñas constituye otro de los elementos centrales de MovieMate, ya que permite a los usuarios registrar su opinión sobre el contenido consumido y generar información relevante para el resto de la comunidad como se puede ver en la Figura 7. Las funcionalidades asociadas a este sistema se pueden agrupar en tres bloques principales: gestión de valoraciones, creación de reseñas y mecanismos de interacción social sobre el contenido generado por los usuarios.

Gestión de valoraciones:

- Crear una valoración para un contenido determinado, indicando una puntuación y, opcionalmente, un comentario o reseña asociada.
- Editar una valoración, actualizando su puntuación y, opcionalmente, un comentario o reseña asociada.

- Eliminar una valoración previamente registrada.
- Consultar el historial de sus propias valoraciones, accediendo a un listado de películas y series valoradas por el usuario autenticado.
- Consultar las valoraciones asociadas a un contenido concreto, permitiendo ver cómo ha sido recibido por el resto de los usuarios de MovieMate.

Reseñas e interacción social:

- Indicar que una reseña de otro usuario resulta de interés mediante mecanismos de interacción como los “me gusta”.
- Comentar reseñas de otros usuarios.
- Comentar en listas de otros usuarios.
- Enviar reportes de contenido.

Estas funcionalidades permiten generar información agregada sobre el contenido, facilitando tanto la toma de decisiones en la exploración de nuevas películas o series como el desarrollo de dinámicas sociales dentro de la plataforma. De este modo, el sistema no solo permite la gestión individual del contenido consumido, sino que también fomenta la interacción entre usuarios y la generación de contenido compartido, reforzando el carácter social de la aplicación.

4.1.2.4 Administrador

El administrador es un usuario con privilegios elevados, responsable de la moderación del contenido y de la gestión de usuarios. Puede acceder al panel de administración, revisar reportes de contenido, eliminar valoraciones o comentarios inapropiados y gestionar tanto el estado como el rol de las cuentas de usuario.

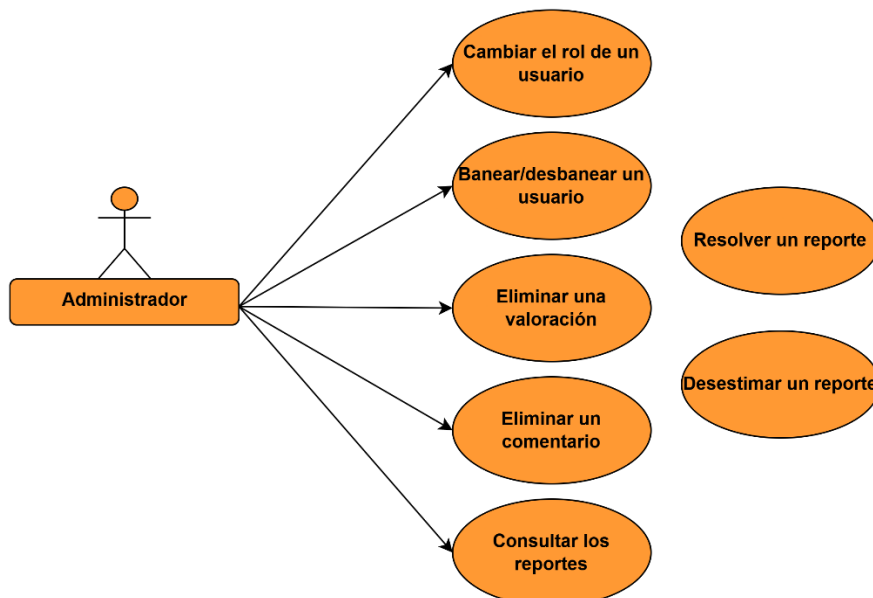


Figura 8. Diagrama de casos de uso del administrador

En la Figura 8 se representan los casos de uso asociados al usuario administrador, que se detallan a continuación.

Administrador

- Cambiar el rol de un usuario, promoviendo o degradando cuentas entre los perfiles de usuario estándar y administrador.
- Banear o desbanear a un usuario, controlando si una cuenta puede acceder o no a la plataforma en función de su comportamiento.
- Consultar los reportes de contenido enviados por los usuarios, accediendo a un listado de valoraciones y comentarios denunciados junto con su información asociada.
- Eliminar una valoración previamente registrada cuando se considere inapropiada o contraria a las normas de uso de la plataforma.
- Eliminar un comentario publicado por cualquier usuario cuando infrinja las políticas de la comunidad.
- Resolver un reporte, marcándolo como atendido tras revisar el contenido relacionado y, en su caso, aplicar las acciones de moderación correspondientes.
- Desestimar un reporte, registrando que no procede realizar ninguna acción sobre el contenido denunciado.

4.2 Diseño

En este apartado se describe el diseño del sistema desarrollado, comenzando por el modelo de datos sobre el que se apoya la aplicación MovieMate y continuando posteriormente con el diseño de los componentes *back-end* y *front-end*.

El diseño se ha planteado siguiendo una arquitectura en capas, separando claramente la persistencia de datos, la lógica de negocio y la exposición de servicios mediante una API REST, lo que facilita el mantenimiento y la evolución futura del sistema.

4.2.1 Modelo de datos

El modelo de datos constituye la base sobre la que se construyen las funcionalidades de gestión, consulta y valoración de películas y series, así como las características sociales de la plataforma. Como se puede ver en la Figura 9, el modelo de datos de MovieMate se ha diseñado siguiendo un enfoque relacional. El objetivo principal ha sido representar de forma coherente las entidades clave del dominio (películas, series, usuarios, listas, valoraciones y relaciones sociales), así como las relaciones entre ellas, manteniendo un equilibrio entre normalización, eficiencia en las consultas y claridad conceptual.

En el núcleo del modelo se encuentra la entidad `User`, que representa a los usuarios registrados en la plataforma. Cada usuario dispone de credenciales de acceso, información de perfil (nombre de usuario, nombre completo, correo electrónico, biografía e imagen de perfil) y metadatos de auditoría, como las fechas de creación y actualización. Asimismo, la entidad incorpora un atributo que determina el nivel de visibilidad del perfil (público o privado), permitiendo introducir restricciones condicionales en el acceso a la información social del usuario. Esta entidad actúa como punto de anclaje para gran parte de las relaciones del sistema, dado que los usuarios son los responsables de crear listas, emitir valoraciones y establecer vínculos sociales con otros miembros de la plataforma.

La entidad `Content` modela el contenido audiovisual gestionado por MovieMate, abarcando tanto películas como series mediante un atributo que indica el tipo de contenido. Para cada

elemento se almacenan identificadores internos y externos, incluyendo el identificador de TMDB, así como atributos descriptivos como el título, la sinopsis, la fecha de estreno y los recursos gráficos asociados.

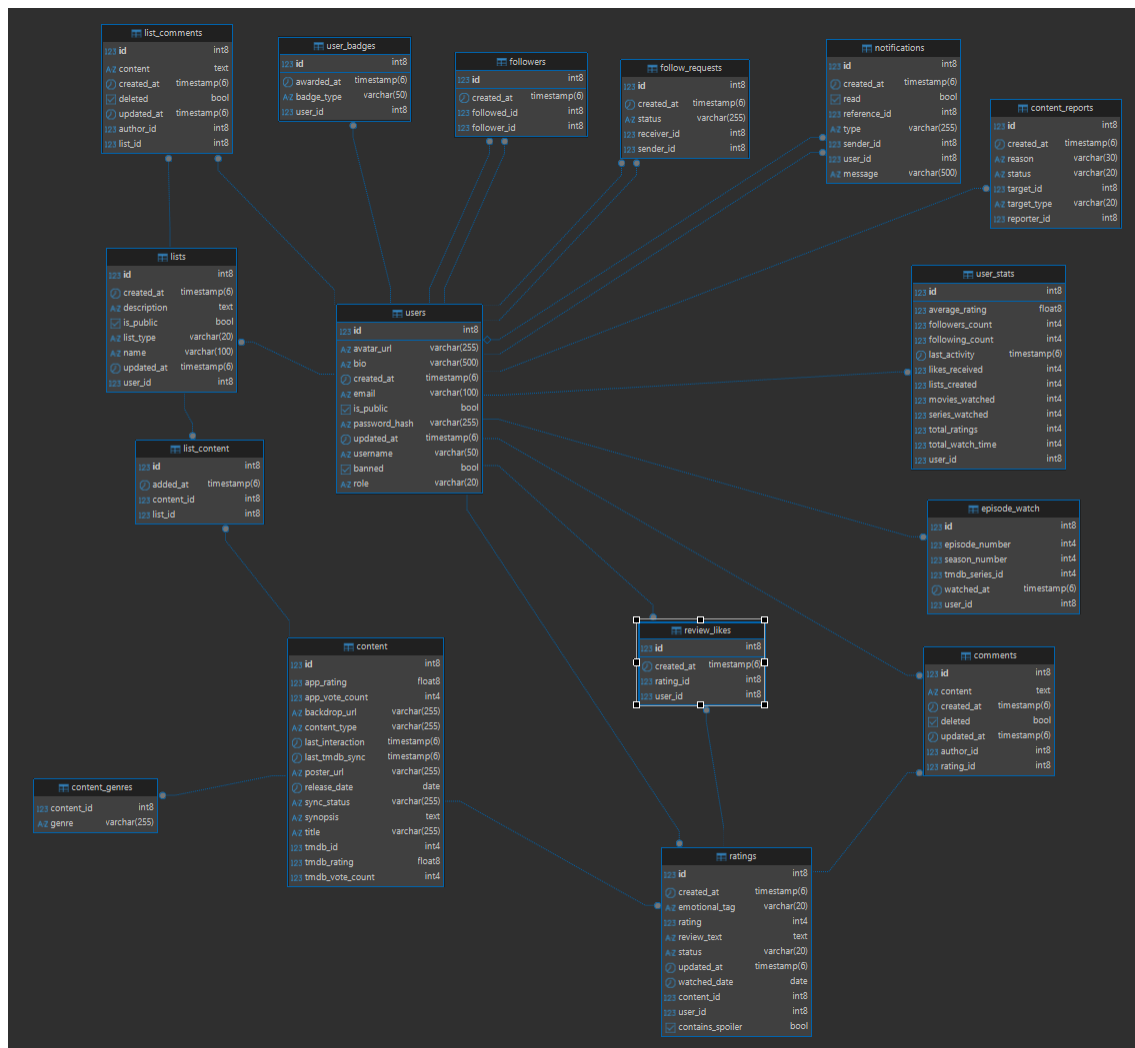


Figura 9. Diagrama entidad-relación de MovieMate

Además de los datos descriptivos, la entidad distingue entre métricas externas, procedentes de la API de TMDB (puntuación global y número de votos), y métricas internas generadas por la propia aplicación (puntuación media y número de valoraciones de los usuarios de MovieMate). Esta separación permite mantener independencia respecto a la fuente externa y facilita el cálculo eficiente de estadísticas propias.

Esta separación entre métricas externas e internas permite desacoplar la lógica de la aplicación de la fuente de datos externa, evitando inconsistencias y facilitando la evolución del sistema sin dependencia directa de la API de terceros.

El modelo incorpora también información relacionada con la sincronización del contenido con la API externa. Para ello se almacenan marcas temporales de 'última actualización' y 'última interacción', así como un estado que indica si el contenido se encuentra actualizado, potencialmente desactualizado o en proceso de sincronización. Este mecanismo permite

adoptar una estrategia de persistencia bajo demanda: el contenido se almacena únicamente cuando existe interacción real por parte de los usuarios, reduciendo así la dependencia directa de la API externa y optimizando el rendimiento del sistema.

En relación con la clasificación temática, los géneros asociados a cada contenido se gestionan mediante una colección almacenada en una tabla independiente, lo que permite modelar de forma flexible la relación entre contenido y categorías sin necesidad de crear una entidad específica para cada género.

La relación entre usuarios y contenido se articula principalmente a través de la entidad `Rating`, que representa las valoraciones y reseñas emitidas por los usuarios sobre películas y series. Cada registro de esta tabla asocia un usuario con un contenido concreto, almacenando la puntuación otorgada, el texto opcional de la reseña y las marcas temporales de creación y actualización. Este diseño permite calcular estadísticas agregadas sobre el contenido, como puntuaciones medias o número de valoraciones, al tiempo que ofrece un historial de la actividad de cada usuario. Este diseño permite desacoplar la interacción del usuario del contenido, facilitando tanto el cálculo de métricas agregadas como el análisis individual del comportamiento del usuario.

La organización personalizada del contenido se materializa mediante la entidad `List`, que recoge las listas creadas por los usuarios para agrupar películas y series según distintos criterios (por ejemplo, “Por ver”, “Favoritas” o colecciones temáticas). Cada lista está asociada a un único usuario propietario y cuenta con atributos como el nombre, la descripción, el tipo de lista y el nivel de visibilidad (pública o privada). La relación muchos-a-muchos entre listas y contenido se resuelve mediante una tabla intermedia que almacena los elementos incluidos en cada lista, permitiendo además incorporar información adicional como el orden de aparición o la fecha de incorporación.

La utilización de una tabla intermedia en la relación muchos-a-muchos permite enriquecer la relación con atributos adicionales, lo que resulta clave para funcionalidades como la ordenación personalizada o el seguimiento temporal.

El componente social del sistema se modela a través de la entidad `Follower`, que representa el vínculo entre usuarios seguidores y usuarios seguidos. Cada registro enlaza dos usuarios distintos, evitando duplicidades mediante restricciones de unicidad y previniendo la autorreferencia. Esta estructura permite construir las listas de seguidores y seguidos de cada usuario y soporta funcionalidades como el *feed* de actividad personalizado, basado en las acciones de los usuarios seguidos. La imposición de restricciones de unicidad y la prevención de autorreferencias garantizan la integridad del grafo social del sistema.

Por otro lado, la entidad `FollowRequest` modela el proceso previo a la confirmación de una relación social cuando el perfil del usuario es privado. Cada solicitud enlaza un usuario emisor y un usuario receptor, incorporando un estado que permite distinguir entre solicitudes pendientes, aceptadas, rechazadas o canceladas. La existencia de esta entidad permite representar explícitamente el ciclo de vida completo de una solicitud de seguimiento y aplicar reglas de negocio diferenciadas en función de su estado. Este modelo permite implementar

flujos de interacción más complejos en escenarios de privacidad, alineándose con el comportamiento esperado en redes sociales modernas.

Con el fin de reforzar la interacción social, el modelo incluye además la entidad `Notification`, destinada a registrar eventos relevantes para el usuario receptor. Cada notificación se asocia a un usuario receptor y, opcionalmente, a un usuario emisor, almacenando el tipo de evento que la ha generado y una referencia al recurso relacionado. La entidad contempla siete tipos de evento: solicitud de seguimiento recibida, aceptación de una solicitud de seguimiento, incorporación de un nuevo seguidor, interacción positiva en una valoración, comentario sobre una reseña propia, comentario en una lista propia y eliminación de contenido por moderación. Incorpora además un campo de texto libre que almacena información contextual para determinados tipos: el título del contenido valorado en notificaciones de comentario sobre reseña, y el nombre de la lista en notificaciones de comentario sobre lista, permitiendo al receptor identificar el recurso afectado sin consultas adicionales. Un indicador de lectura permite distinguir entre notificaciones pendientes y ya procesadas, facilitando la gestión eficiente de la actividad reciente. Este diseño permite desacoplar la generación de eventos de su consumo, facilitando su integración con mecanismos de comunicación en tiempo real.

Por último, el modelo de datos incluye una entidad `UserStats` destinada a almacenar estadísticas agregadas sobre el comportamiento de los usuarios en la plataforma, tales como el número de películas y series vistas, el total de valoraciones emitidas o el tiempo de visionado acumulado. Esta tabla se encuentra vinculada uno-a-uno con la entidad `User` y se actualiza de forma periódica o bajo demanda a partir de la información registrada en `Ratings` y otros componentes del sistema. Esta separación de responsabilidades permite optimizar las consultas frecuentes sobre estadísticas sin sobrecargar las tablas transaccionales.

Para registrar la interacción de los usuarios sobre las valoraciones, el modelo incorpora dos entidades adicionales. `Comment` almacena los comentarios escritos por otros usuarios sobre una reseña concreta, asociando el texto del comentario tanto al usuario autor como a la valoración referenciada, con soporte de borrado lógico. `ReviewLike` modela los «me gusta» que los usuarios pueden dar a las reseñas de otros, garantizando mediante una restricción de unicidad que un mismo usuario no puede dar más de un *like* a la misma valoración. Estas entidades permiten modelar la interacción de los usuarios sobre las valoraciones, reforzando el componente social.

El seguimiento del visionado de contenido televisivo se gestiona a través de la entidad `EpisodeWatch`, que almacena los episodios individuales marcados como vistos por cada usuario. Permite modelar el progreso de visionado a nivel de episodio, una funcionalidad clave en contenido serializado. Cada registro identifica la serie mediante su identificador de TMDB, el número de temporada y el número de episodio, con una restricción de unicidad sobre la combinación de estos cuatro campos que impide registros duplicados. Los comentarios sobre las listas personalizadas se modelan mediante `ListComment`, que asocia un texto con un usuario autor y la lista comentada, e incorpora igualmente borrado lógico para preservar la integridad referencial. Permite extender la interacción social al ámbito de las listas personalizadas.

La plataforma incluye además un sistema de moderación de contenido basado en reportes generados por los usuarios y en las acciones del rol administrador, que puede revisar y eliminar valoraciones o comentarios inapropiados cuando sea necesario. Este está articulado mediante la entidad `ContentReport`, que permite a los usuarios reportar valoraciones o comentarios inapropiados. Cada reporte registra el tipo de objeto denunciado (`RATING` o `COMMENT`), el motivo de la denuncia (`SPAM`, `INAPPROPRIATE`, `SPOILER` u `OTHER`) y el estado de gestión (`PENDING`, `RESOLVED` o `DISMISSED`). Introduce mecanismos de control necesarios para mantener la calidad del contenido generado por los usuarios.

Finalmente, la entidad `UserBadge` materializa el sistema de gamificación de la plataforma, almacenando las insignias concedidas a cada usuario con la fecha de concesión. La combinación de usuario e insignia está sujeta a una restricción de unicidad para garantizar que cada usuario sólo puede obtener cada insignia una vez.

En conjunto, este modelo de datos proporciona una base robusta y extensible que soporta tanto las funcionalidades principales de gestión de contenido como los mecanismos de interacción social, permitiendo la evolución futura del sistema sin comprometer su coherencia estructural.

4.2.2 Diseño del *back-end*

El diseño del *back-end* de MovieMate sigue una arquitectura en tres capas basada en el patrón MVC, donde se separan claramente los controladores web, la lógica de negocio y el acceso a datos. El servidor expone una API RESTful consumida por el cliente web y potencialmente por otros clientes de terceros. La elección de esta arquitectura basada en capas responde a la necesidad de separar responsabilidades y facilitar la mantenibilidad del sistema, permitiendo evolucionar de forma independiente cada componente sin afectar al resto de la aplicación

En la capa de **presentación** se sitúan los controladores REST, encargados de recibir las peticiones HTTP, validar los datos de entrada y devolver respuestas JSON adecuadas. Estos controladores están organizados por dominios funcionales, agrupando *endpoints* relacionados con autenticación, usuarios, contenido, listas, valoraciones, seguidores o actividad. Cada controlador delega la lógica de negocio en servicios especializados, evitando mezclar reglas de negocio con detalles de transporte.

La capa de **aplicación** está formada por servicios que encapsulan la lógica de negocio de la plataforma, coordinando las operaciones entre las distintas entidades y repositorios. Estos servicios implementan casos de uso como la creación de listas, la emisión de valoraciones, la actualización de estadísticas de usuario o la generación del *feed* de actividad, garantizando la consistencia de los datos y aplicando las validaciones necesarias. Los servicios constituyen el núcleo de la lógica de negocio, asegurando la consistencia de las operaciones mediante el uso de transacciones cuando es necesario y coordinando múltiples repositorios dentro de una misma operación lógica.

Adicionalmente, se incluye un servicio de integración con TMDB bajo demanda, responsable de sincronizar información externa de películas y series con el modelo interno de la aplicación. Los endpoints de sincronización con TMDB son invocados desde el cliente cuando un usuario interactúa con un contenido. La aplicación consulta primero la base de datos local y, solo si no

existe el registro o ha expirado su tiempo de vida, realiza una llamada a la API externa para refrescar los datos y almacenarlos de nuevo. Esta política de caché reduce la latencia percibida por el usuario y minimiza la dependencia de TMDb. Este mecanismo está restringido a usuarios autenticados para evitar un uso abusivo de la API externa.

En relación con la moderación de contenido, MovieMate incorpora un mecanismo de **reporte** que permite a cualquier usuario autenticado señalar valoraciones o comentarios potencialmente inapropiados. Estas incidencias quedan registradas como entidades de reporte asociadas al contenido afectado y a su autor. El rol administrador dispone de un panel específico desde el que puede consultar los reportes pendientes, revisar el contenido asociado y decidir si procede eliminar la valoración o comentario denunciado o desestimar el reporte. De este modo, se establece un flujo de moderación apoyado en la participación de la comunidad y gestionado de forma centralizada por el administrador.

La capa de **persistencia** define repositorios por cada entidad principal del dominio. Estos repositorios proporcionan operaciones CRUD (*Create, Read, Update, Delete*) básicas y consultas derivadas a partir de nombres de método o mediante consultas personalizadas cuando es necesario.

La API REST de MovieMate se ha diseñado siguiendo principios RESTful [44], usando sustantivos en plural para los recursos y métodos HTTP estándar para expresar las operaciones. A modo completo, en la Tabla 4 del Anexo B – Tabla con los *endpoints* de la aplicación se describen todos los *endpoints* disponibles y las operaciones asociadas a cada uno de ellos.

Se distinguen tres categorías de *endpoints*, agrupados por niveles de acceso en función del tipo de usuario:

- *Endpoints* públicos (color verde): accesibles sin autenticación. Permiten operaciones de registro e inicio de sesión, consulta de usuarios y sus estadísticas, acceso a listas públicas, consulta de contenido y metadatos externos desde TMDb, visualización de seguidores, obtención de valoraciones y comentarios públicos, sincronización de películas y series, así como acceso a información de actores, tendencias y catálogos de películas y series.
- *Endpoints* autenticados (color rojo): requieren un usuario autenticado mediante JWT. Permiten la gestión del perfil del usuario, cambio de contraseña y avatar, consulta de información privada propia, gestión de listas personales, creación y eliminación de valoraciones, interacción social mediante seguimientos, comentarios, notificaciones y *feed* personalizado, seguimiento del progreso de episodios vistos, obtención de recomendaciones e insignias, y creación de reportes de contenido inapropiado.
- *Endpoints* de administrador (color azul): restringidos a usuarios con rol administrativo. Permiten la administración global de usuarios, incluyendo listado, cambio de rol y baneo, así como tareas de moderación sobre valoraciones, comentarios y reportes, pudiendo revisar, eliminar, resolver o desestimar incidencias reportadas.

Esta clasificación permite aplicar políticas de seguridad coherentes y garantizar el control de acceso a los distintos recursos del sistema. En algunos casos, como en las operaciones de *like* o en el marcado de episodios como vistos, se han adoptado decisiones pragmáticas que se apartan parcialmente de la ortodoxia REST, priorizando la simplicidad de uso desde el cliente y la eficiencia en la interacción.

4.2.3 Diseño del *front-end*

El diseño de la interfaz de usuario de MovieMate se ha elaborado a partir de *mockups* de alta fidelidad que definen tanto la estructura visual como la organización funcional de la aplicación. Estos diseños no solo establecen la identidad visual, sino que permiten validar la usabilidad y la accesibilidad de las funcionalidades antes de su implementación.

Los *mockups* recogen las principales pantallas de la aplicación: la página de inicio con carruseles de contenido destacado y secciones de actividad social, el catálogo de descubrimiento con filtros por género, año y puntuación, la ficha de detalle de una película o serie con información completa e integración de valoraciones, el perfil de usuario con estadísticas y listas, la sección de listas personalizadas y el panel de configuración. Cada pantalla se ha diseñado considerando los casos de uso definidos en el análisis, garantizando que todas las funcionalidades sean accesibles de forma intuitiva.

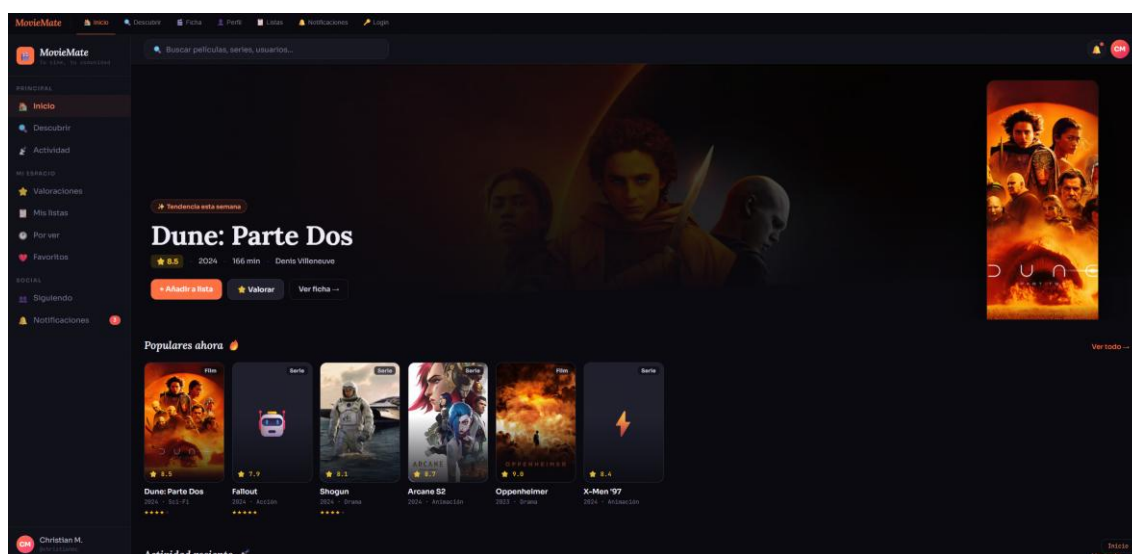


Figura 10. *Mockup* de MovieMate

Estos *mockups*, como el que se puede observar en la Figura 10, se han utilizado como herramienta de validación de los casos de uso definidos en la fase de análisis, asegurando que cada funcionalidad dispone de un flujo de interacción claro y accesible para el usuario.

La estructura de navegación se ha diseñado teniendo en cuenta distintos contextos de uso. En dispositivos de escritorio se emplea una barra lateral fija que permite acceso rápido a las principales secciones sin pérdida de contexto, mientras que en dispositivos móviles se adopta una barra inferior para optimizar el acceso con una sola mano y mejorar la ergonomía.

La aplicación distingue entre rutas públicas, accesibles sin autenticación (página de inicio, catálogo, fichas de contenido y perfiles públicos de usuario), y rutas protegidas, que requieren

sesión activa (listas personalizadas, historial de valoraciones, notificaciones y configuración). Esta distinción se aplica tanto en el nivel de navegación como en la presentación de las opciones de interacción disponibles para el usuario. Esta separación entre rutas públicas y protegidas no solo responde a requisitos de seguridad, sino que también permite adaptar dinámicamente la interfaz en función del estado de autenticación del usuario.

Además, los *mockups* se han diseñado con un enfoque plenamente responsivo, contemplando variaciones específicas para escritorio, tablet y dispositivos móviles como se puede observar en la Figura 11. La disposición de elementos se adapta a cada anchura de pantalla: en entornos de escritorio se prioriza el uso de una barra lateral fija que maximiza el espacio disponible para los listados y las fichas de contenido, mientras que en dispositivos móviles inicialmente se planteó una barra de navegación superior, mientras que en la versión final se ha optado por una barra inferior fija, más ergonómica para el uso con una sola mano. Este planteamiento garantiza que las funcionalidades definidas en los casos de uso sigan siendo accesibles y usables con independencia del dispositivo desde el que se acceda a MovieMate.

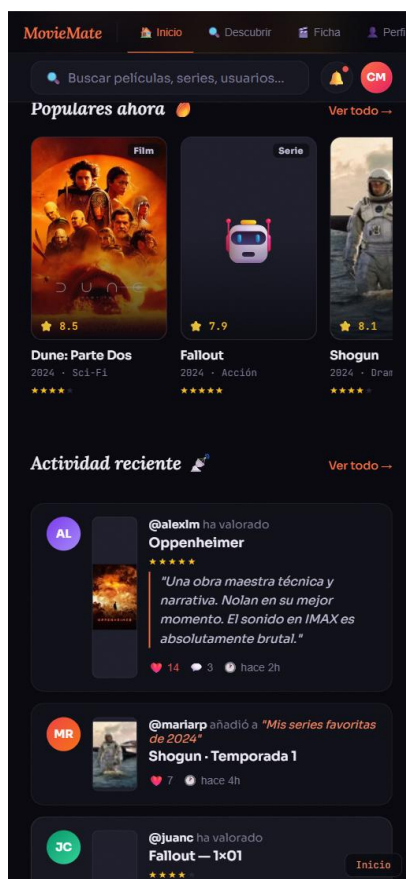


Figura 11. Mockup de MovieMate versión móvil

4.3 Implementación

El código fuente completo está disponible en el repositorio público de GitHub: https://github.com/GivenCloud/TFG_MovieMate

La arquitectura del sistema constituye uno de los pilares fundamentales en el desarrollo de la aplicación MovieMate, ya que define la forma en la que los distintos componentes del sistema se organizan, interactúan entre sí y evolucionan a lo largo del tiempo. En este proyecto, la

arquitectura se ha diseñado con el objetivo de garantizar un sistema robusto, modular, escalable y fácilmente mantenible, alineado con las necesidades funcionales de una aplicación web moderna orientada al consumo intensivo de datos.

MovieMate se concibe como una aplicación web basada en un **modelo cliente-servidor**. En este enfoque, el servidor actúa como proveedor de servicios y concentra la lógica de negocio, la gestión de datos y los mecanismos de seguridad, mientras el cliente se encarga de la interacción con el usuario. Esta separación permite desacoplar responsabilidades, facilitando tanto la evolución independiente de cada componente como la integración con distintos tipos de clientes. Este modelo permite escalar de forma independiente el cliente y el servidor, así como facilitar la integración con futuros clientes sin modificar la lógica de negocio. La Figura 12 muestra la arquitectura general del sistema, incluyendo la interacción entre cliente, servidor, base de datos y servicio externo.

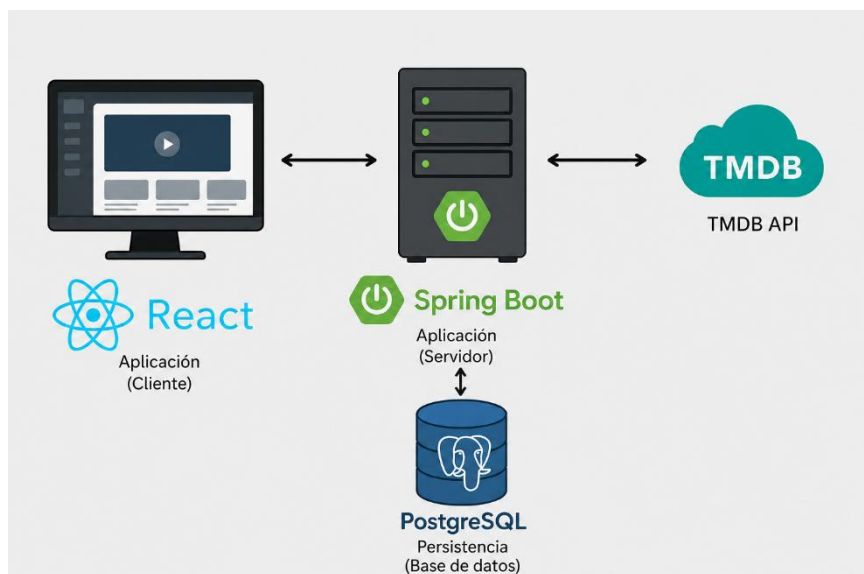


Figura 12. Arquitectura de MovieMate

El sistema adopta un enfoque monolítico modular. Esto permite mantener la simplicidad operativa de un despliegue unificado, evitando la complejidad inherente a arquitecturas distribuidas, al tiempo que conserva una separación lógica interna que facilita la evolución hacia arquitecturas más desacopladas en caso necesario. En primer lugar, se ha optado por una arquitectura basada en **API REST**, ya que este estilo arquitectónico permite desacoplar completamente el cliente del servidor. De este modo, el *back-end* puede ser consumido por distintos tipos de clientes (aplicaciones web, móviles o incluso servicios externos) sin necesidad de modificar su implementación interna. Esta decisión resulta especialmente relevante en un proyecto como MovieMate, que aspira a integrar múltiples funcionalidades y a evolucionar con el tiempo. En segundo lugar, la adopción de una **arquitectura en capas** facilita la separación de responsabilidades dentro del sistema. Cada capa se encarga de un conjunto específico de tareas, como la recepción de peticiones, la aplicación de reglas de negocio o el acceso a datos. Esta organización reduce el acoplamiento entre componentes, mejora la legibilidad del código y favorece la realización de pruebas y tareas de mantenimiento.

La arquitectura incorpora además una integración con la API externa de TMDb para la obtención de información cinematográfica. Esta integración se realiza íntegramente desde el servidor, que actúa como intermediario entre el cliente y la fuente de datos externa. De este modo, las credenciales de acceso a la API permanecen protegidas en el servidor y no se exponen al cliente. En particular, la integración con TMDb se gestiona mediante un servicio dedicado (`TmdbService`) que encapsula toda la lógica de comunicación con la API externa. Este servicio expone métodos para buscar películas y series, obtener detalles completos de un título, recuperar el reparto, los géneros disponibles o los proveedores de *streaming* para una región geográfica determinada.

Con el objetivo de reducir la dependencia del servicio externo y optimizar el rendimiento, se ha implementado una estrategia de persistencia bajo demanda. El contenido no se precarga masivamente en la base de datos, sino que se almacena únicamente cuando un usuario interactúa con él (por ejemplo, al valorar una película o añadirla a una lista). Este proceso de sincronización se gestiona a través del método `getOrFetch` del `ContentService`: si el contenido ya existe en la base de datos local, se devuelve directamente; si no existe, se consulta la API de TMDb, se persiste en la base de datos y se devuelve al solicitante. Adicionalmente, cuando el contenido almacenado supera un tiempo de vida (TTL) configurable, se programa una actualización asíncrona en segundo plano para mantener los datos vigentes sin bloquear la respuesta al cliente. La estrategia de persistencia bajo demanda, junto con el mecanismo de actualización basado en TTL, permite reducir significativamente la dependencia de la API externa, mejorar los tiempos de respuesta y garantizar la coherencia de los datos sin penalizar la experiencia de usuario.

La adopción de una arquitectura de microservicios se ha descartado debido a que introduciría una complejidad innecesaria en aspectos como la comunicación entre servicios, la gestión de transacciones distribuidas y el despliegue [45]. Dado el alcance del proyecto, un monolito modular proporciona un equilibrio más adecuado entre simplicidad y capacidad de evolución. En conjunto, la arquitectura definida permite construir un sistema cohesivo, mantenible y preparado para su evolución futura, alineado con las buenas prácticas del desarrollo de aplicaciones web modernas. En los siguientes apartados se describe en detalle los principales aspectos relacionados con la implementación del *back-end* y con la implementación del *front-end*. Todo el código de esta aplicación se encuentra en el siguiente repositorio de GitHub: https://github.com/GivenCloud/TFG_MovieMate

4.3.1 Implementación del *back-end*

El *back-end* se ha implementado como una aplicación **monolítica modular**, en la que todos los componentes residen en una única aplicación **Spring Boot**, pero organizados internamente en módulos bien definidos. Este enfoque permite beneficiarse de la simplicidad de despliegue propia de los sistemas monolíticos, manteniendo al mismo tiempo una estructura interna clara y mantenible. La modularización se logra mediante la organización del código en paquetes cohesionados, cada uno responsable de un dominio funcional concreto, como la gestión de usuarios, contenido, valoraciones o seguridad.

En MovieMate, el sistema se organiza en las siguientes capas (ver Figura 13):

- Capa de presentación

- Capa de negocio
- Capa de persistencia
- Capa de seguridad

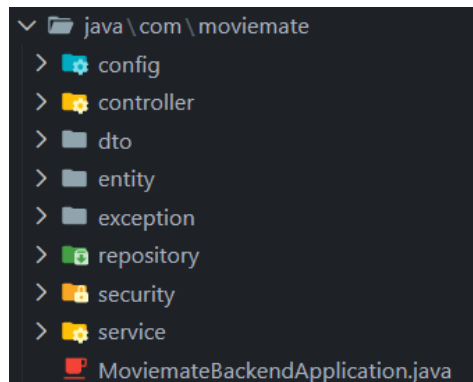


Figura 13. Estructura de carpetas del *back-end* de MovieMate

Esta estructura, ampliamente utilizada en aplicaciones basadas en Spring Boot, parte de la necesidad de aislar los distintos aspectos del sistema, evitando acoplamientos innecesarios y facilitando la evolución independiente de cada componente. En los siguientes apartados se describen los aspectos más importantes de cada capa.

4.3.1.1 Capa de presentación (Controladores)

La capa de presentación está formada por los controladores REST (carpeta `controller` en la Figura 13), cada uno de ellos asociado a un dominio funcional concreto. Entre los controladores más relevantes se encuentran:

- `AuthController`, encargado de las operaciones de registro, inicio de sesión y renovación de tokens.
- `UserController`, que gestiona la consulta y actualización de perfiles de usuario, así como las opciones de privacidad.
- `FollowerController`, responsable de las relaciones de seguimiento entre usuarios (enviar solicitudes, aceptar o rechazar, seguir y dejar de seguir).
- `ListController`, que permite crear, editar y eliminar listas personalizadas, así como gestionar su contenido.
- `RatingController`, que expone los *endpoints* para crear, actualizar y consultar valoraciones y comentarios asociados.
- `ContentController`, orientado a la consulta de películas y series almacenadas en la base de datos de MovieMate.
- `NotificationController`, que proporciona acceso a la bandeja de notificaciones y a las operaciones de marcado como leídas.
- `AdminController`, que agrupa las operaciones de administración y moderación (gestión de usuarios y reportes de contenido).

Estos controladores reciben las solicitudes, validan los datos de entrada y delegan la lógica de negocio en la capa de servicios correspondiente.

Una de las decisiones clave en esta capa ha sido mantener los controladores lo más ligeros posible, evitando la inclusión de lógica de negocio compleja. Este enfoque permite reducir la complejidad del código, facilitar la lectura y el mantenimiento, y mejorar la capacidad de prueba aislada de los controladores.

```
1  @RestController
2  @RequestMapping("/api/users")
3  @RequiredArgsConstructor
4  public class FollowerController {
5
6      private final FollowerService followerService;
7      private final UserService userService;
8      private final FollowRequestService followRequestService;
9
10     @Operation(
11         summary = "Seguir a un usuario",
12         description = "Permite que el usuario autenticado siga a otro usuario."
13     )
14     @PostMapping("/{userId}/follow-requests")
15     public ResponseEntity<Void> sendFollowRequest(
16         @AuthenticationPrincipal CustomUserDetails userDetails,
17         @Parameter(description = "ID del usuario al que se quiere seguir, example = 1")
18         @PathVariable Long userId) {
19
20         User currentUser = userDetails.getUser();
21         User userToFollow = userService.findById(userId);
22         followRequestService.sendFollowRequest(currentUser, userToFollow);
23
24         return ResponseEntity.ok().build();
25     }
26
27     @Operation(
28         summary = "Aceptar solicitud de seguimiento",
29         description = "Permite que el usuario autenticado acepte una solicitud de seguimiento."
30     )
31     @PostMapping("/follow-requests/{requestId}")
32     public ResponseEntity<FollowRequestActionResponse> acceptFollowRequest(@AuthenticationPrincipal CustomUserDetails userDetails, @PathVariable Long requestId) {
33         FollowRequestActionResponse response = followRequestService.acceptRequest(requestId, userDetails.getUser());
34         return ResponseEntity.ok(response);
35     }
36
37     @Operation(
38         summary = "Rechazar solicitud de seguimiento",
39         description = "Permite que el usuario autenticado rechace una solicitud de seguimiento."
40     )
41     @DeleteMapping("/follow-requests/{requestId}")
42     public ResponseEntity<FollowRequestActionResponse> rejectFollowRequest(@AuthenticationPrincipal CustomUserDetails userDetails, @PathVariable Long requestId) {
43         FollowRequestActionResponse response = followRequestService.rejectRequest(requestId, userDetails.getUser());
44         return ResponseEntity.ok(response);
45     }
46 }
```

Figura 14. Ejemplo de controlador REST (*FollowerController*) en MovieMate

En la Figura 14 se muestra un ejemplo de controlador REST (*FollowerController*), que define distintos *endpoints* relacionados con el seguimiento de usuarios. Cada método está anotado con *@GetMapping*, *@PostMapping*, etc., lo que permite mapear las operaciones HTTP a métodos concretos. Se observa el uso de anotaciones como *@PathVariable* (en otros también se usa *@RequestBody*) para extraer parámetros de la petición. Además, la anotación *@Operation* se emplea para documentar automáticamente los *endpoints* mediante OpenAPI/Swagger, facilitando la generación de documentación interactiva de la API.

La capa de presentación se apoya en el uso intensivo de *Data Transfer Objects* (DTOs) como mecanismo de intercambio de información con el cliente. Estos DTOs actúan como una capa de abstracción entre las entidades JPA del dominio y la representación externa expuesta por los controladores, lo que permite controlar de forma explícita los datos enviados al cliente, evitar la exposición directa de entidades persistentes y reducir el acoplamiento entre la API y la capa de persistencia. En un modelo de datos complejo como el de MovieMate, con múltiples relaciones entre usuarios, valoraciones, listas y seguidores, esta estrategia resulta especialmente relevante para proteger atributos sensibles (como credenciales o

configuraciones de privacidad) y adaptar las respuestas a las necesidades específicas de cada *endpoint*. La Figura 15 muestra un ejemplo de DTO utilizado para exponer información de contenido simplificada para su visualización en el cliente.



```
1  @Data
2  @Builder
3  public class ContentResponse {
4      private Long id;
5      private Integer tmdbId;
6      private String title;
7      private Content.ContentType contentType;
8      private String releaseDate;
9      private String posterUrl;
10     private String backdropUrl;
11     private String synopsis;
12     private String[] genres;
13     private Double averageRating;
14     private Integer voteCount;
15     private String lastSync;
16 }
```

Figura 15. Ejemplo DTO

Cabe destacar que el proyecto integra documentación automática de la API mediante **SpringDoc/OpenAPI**, exponiendo Swagger UI en `/swagger-ui/index.html` para facilitar la exploración de los *endpoints* durante el desarrollo como se puede ver en la Figura 16.



follower-controller		
GET	/api/users/{userId}/followers	Obtener seguidores de un usuario
POST	/api/users/{userId}/followers	Seguir a un usuario
DELETE	/api/users/{userId}/followers	Dejar de seguir a un usuario
GET	/api/users/{userId}/following	Obtener usuarios seguidos por un usuario
GET	/api/users/{userId}/following-status	Comprobar si el usuario autenticado sigue a otro usuario

Figura 16. *Endpoints* para gestión de seguidores en MovieMate

4.3.1.2 Capa de negocio (Servicios)

La capa de negocio está formada por los servicios (carpeta `service` en la Figura 13), esta constituye el núcleo funcional del sistema, ya que encapsula toda la lógica de negocio de la aplicación.

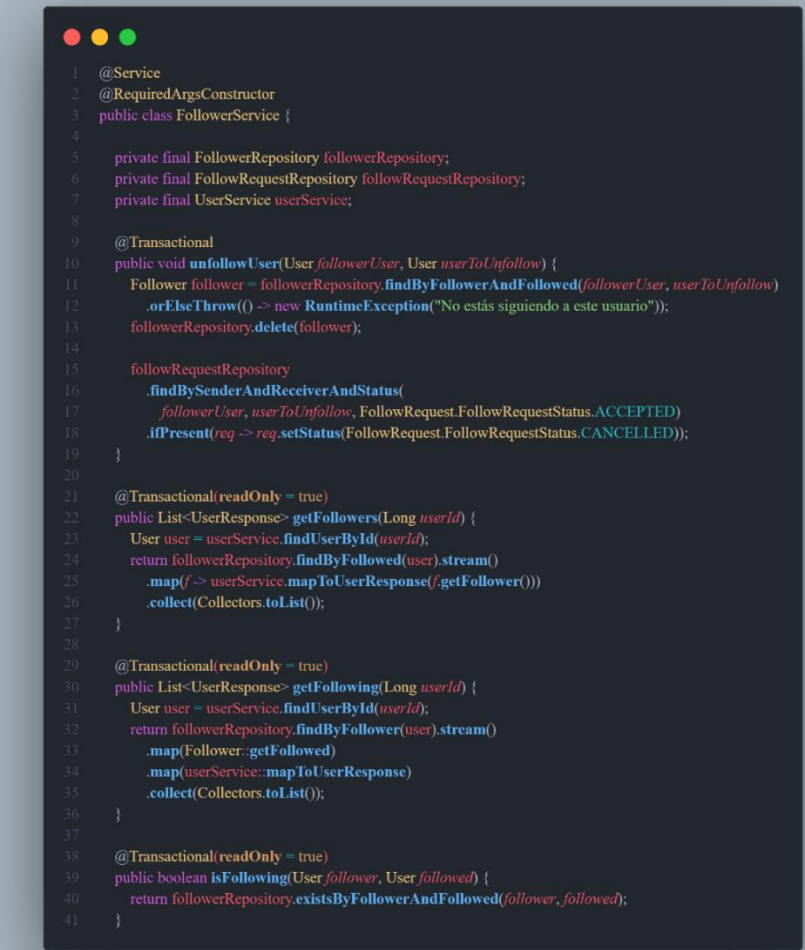
En esta capa se implementan, entre otras, las siguientes responsabilidades:

- Gestión de usuarios y autenticación.
- Creación y actualización de valoraciones.
- Organización de contenidos en listas.

- Gestión de relaciones sociales (seguimientos y solicitudes).
- Generación de notificaciones.
- Integración y sincronización con la API externa.

Separar la lógica de negocio en esta capa permite desacoplarla tanto de la presentación como del acceso a datos, favoreciendo la reutilización del código y la posibilidad de modificar la lógica interna sin afectar a los consumidores de la API.

Además, esta capa actúa como intermediaria entre los controladores y los repositorios, coordinando las operaciones necesarias y garantizando la coherencia del sistema. A continuación, en la Figura 17 se puede ver un ejemplo.



```
1  @Service
2  @RequiredArgsConstructor
3  public class FollowerService {
4
5      private final FollowerRepository followerRepository;
6      private final FollowRequestRepository followRequestRepository;
7      private final UserService userService;
8
9      @Transactional
10     public void unfollowUser(User followerUser, User userToUnfollow) {
11         Follower follower = followerRepository.findByFollowerAndFollowed(followerUser, userToUnfollow)
12             .orElseThrow(() -> new RuntimeException("No estás siguiendo a este usuario"));
13         followerRepository.delete(follower);
14
15         followRequestRepository
16             .findBySenderAndReceiverAndStatus(
17                 followerUser, userToUnfollow, FollowRequest.FollowRequestStatus.ACCEPTED)
18             .ifPresent(req -> req.setStatus(FollowRequest.FollowRequestStatus.CANCELLED));
19     }
20
21     @Transactional(readOnly = true)
22     public List<UserResponse> getFollowers(Long userId) {
23         User user = userService.findUserById(userId);
24         return followerRepository.findByFollowed(user).stream()
25             .map(f -> userService.mapToUserResponse(f.getFollower()))
26             .collect(Collectors.toList());
27     }
28
29     @Transactional(readOnly = true)
30     public List<UserResponse> getFollowing(Long userId) {
31         User user = userService.findUserById(userId);
32         return followerRepository.findByFollower(user).stream()
33             .map(follower -> follower.getFollowed())
34             .map(userService::mapToUserResponse)
35             .collect(Collectors.toList());
36     }
37
38     @Transactional(readOnly = true)
39     public boolean isFollowing(User follower, User followed) {
40         return followerRepository.existsByFollowerAndFollowed(follower, followed);
41     }
42 }
```

Figura 17. Ejemplo de servicio (**FollowerService**) en MovieMate

La clase `FollowerService` está anotada con `@Service`, indicando a Spring que debe gestionarla como un componente de lógica de negocio dentro del contenedor de inversión de control.

En este servicio se implementa la lógica asociada a las relaciones de seguimiento entre usuarios. Por ejemplo, los métodos las funcionalidades de dejar de seguir a un usuario, obtener los usuarios que sigo, los seguidores y consultar si un usuario sigue a otro. Además, en

estas capas se hacen validaciones como, por ejemplo, comprobar que sigue al usuario antes de dejar de seguirlo.

4.3.1.3 Capa de persistencia (Repositorios)

La capa de persistencia está formada por los repositorios (carpeta `repository` en la Figura 13), esta se encarga del acceso a la base de datos PostgreSQL y se implementa mediante **Spring Data JPA** como se puede observar en la Figura 18. El uso de JPA (*Java Persistence API*) permite mapear las entidades Java al modelo relacional de PostgreSQL, aprovechando características como las relaciones entre tablas, la gestión de transacciones y la paginación nativa en las consultas.

Los repositorios actúan como interfaces de acceso a datos, proporcionando operaciones CRUD (*Create, Read, Update, Delete*) y consultas personalizadas cuando es necesario. Esta abstracción evita la inclusión de consultas SQL explícitas en la lógica de negocio y mejora la portabilidad del sistema frente a posibles cambios del gestor de base de datos.

Durante el diseño se ha priorizado el uso de carga diferida (*lazy loading*) en las relaciones entre entidades, con el objetivo de evitar recuperaciones innecesarias de información y optimizar el rendimiento.

Este enfoque ofrece varias ventajas:

- Reducción del código repetitivo.
- Mayor legibilidad.
- Portabilidad frente a cambios en el sistema gestor de bases de datos.

```
1 public interface FollowerRepository extends JpaRepository<Follower, Long> {
2     Optional<Follower> findByFollowerAndFollowed(User follower, User followed);
3     List<Follower> findByFollower(User follower); // Usuarios que yo sigo
4     List<Follower> findByFollowed(User followed); // Mis seguidores
5     boolean existsByFollowerAndFollowed(User follower, User followed);
6
7     @Query("SELECT COUNT(f) FROM Follower f WHERE f.followed.id = :userId")
8     Integer countFollowersByUserId(@Param("userId") Long userId);
9
10    @Query("SELECT COUNT(f) FROM Follower f WHERE f.follower.id = :userId")
11    Integer countFollowingByUserId(@Param("userId") Long userId);
12
13    Page<Follower> findByFollowerInAndCreatedAtAfterOrderByCreatedAtDesc(
14        List<User> followers,
15        LocalDateTime createdAt,
16        Pageable pageable
17    );
18
19    @Query("SELECT COUNT(f) FROM Follower f WHERE f.followed = :user")
20    Integer countByFollowed(@Param("user") User user);
21
22    @Query("SELECT COUNT(f) FROM Follower f WHERE f.follower = :user")
23    Integer countByFollower(@Param("user") User user);
24 }
```

Figura 18. Ejemplo de repositorio JPA (*FollowerRepository*) en MovieMate

En esta imagen se observa la definición de la interfaz extendiendo `JpaRepository`, junto con métodos de consulta derivados del nombre y consultas personalizadas mediante `@Query` para el conteo de seguidores y seguidos. También se incluye un método con paginación y filtro temporal para la generación eficiente de *feeds* de actividad.

4.3.1.4 Capa de seguridad

La capa de seguridad está formada por los archivos de seguridad (carpeta `security` en la Figura 13) y los ficheros de configuración de seguridad (carpeta `config` en la Figura 13). La seguridad se implementa como una capa transversal apoyada en Spring Security. Esta capa intercepta las peticiones entrantes antes de que alcancen los controladores, aplicando mecanismos de autenticación y autorización, como se puede observar en la Figura 19.

El sistema adopta un enfoque sin estado (*stateless*) basado en JSON Web Tokens (JWT). Tras validar las credenciales del usuario, el servidor genera un *token* firmado digitalmente que el cliente debe incluir en las peticiones posteriores. En cada solicitud, un filtro de seguridad verifica la validez del *token* y construye un contexto de seguridad asociado al usuario autenticado. A partir de dicho contexto, se aplican reglas de autorización que restringen el acceso a determinados *endpoints* según el estado de autenticación o los roles asignados. Esta separación permite mantener la lógica de seguridad desacoplada de la lógica de negocio, reforzando el principio de responsabilidad única.

La Figura 19 corresponde a un fichero de configuración de seguridad y realiza diferentes acciones como definir un filtro de autenticación basado en JWT

(JwtAuthenticationFilter), una política de sesiones sin estado (SessionCreationPolicy.STATELESS) y la desactivación de CSRF para una API REST. Se especifican las rutas públicas y se restringe el acceso a /api/admin/** únicamente al rol ADMIN.

```

1  @Configuration
2  @EnableWebSecurity
3  @EnableMethodSecurity(prePostEnabled = true)
4  @RequiredArgsConstructor
5  public class SecurityConfig {
6
7      private final UserRepository userRepository;
8
9      @Bean
10     public UserDetailsServiceImpl userDetailsService() {
11         return username -> userRepository.findByUsernameOrEmail(username, username)
12             .map(CustomUserDetails::new)
13             .orElseThrow(() -> new UsernameNotFoundException("Usuario no encontrado"));
14     }
15
16     @Bean
17     public AuthenticationProvider authenticationProvider() {
18         DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
19         provider.setUserDetailsService(userDetailsService());
20         provider.setPasswordEncoder(passwordEncoder());
21         return provider;
22     }
23
24     @Bean
25     public JwtAuthenticationFilter jwtAuthenticationFilter(JwtService jwtService, UserDetailsServiceImpl userDetailsService) {
26         return new JwtAuthenticationFilter(jwtService, userDetailsService);
27     }
28
29     @Bean
30     public SecurityFilterChain securityFilterChain(HttpSecurity http, JwtAuthenticationFilter jwtAuthenticationFilter) throws Exception {
31         http.setSharedObject(PathPatternParser.class, new PathPatternParser());
32
33         http
34             .csrf(csrf -> csrf.disable())
35             .authorizeHttpRequests(auth -> auth
36                 .requestMatchers("/swagger-ui.html",
37                     "/v3/api-docs",
38                     "/v2/api-docs/**",
39                     "/swagger-resources/**",
40                     "/webjars/**",
41                     "/actuator/**",
42                     "/api/auth/**",
43                     "/api/content/**",
44                     "/api/lists/public",
45                     "/api/users",
46                     "/api/users/{userId:[0-9]+}",
47                     "/api/users/{userId:[0-9]+}/profile",
48                     "/api/users/username/**",
49                     "/api/ratings/{ratingId:[0-9]+}/likes",
50                     "/api/tmdb/tv/**",
51                     "/api/tmdb/movies/**"
52                 )
53                 .permitAll()
54                 .anyRequest().authenticated()
55             )
56             .sessionManagement(session -> session
57                 .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
58             )
59             .authenticationProvider(authenticationProvider())
60             .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);
61         return http.build();
62     }

```

Figura 19. Configuración de seguridad

4.3.1.5 Flujo de una petición

El procesamiento de una petición en el *back-end* sigue una secuencia estructurada alineada con la arquitectura en capas definida anteriormente. En primer lugar, el cliente envía una petición HTTP a la API REST. Esta petición es interceptada por los **filtros de seguridad** de Spring Security, en particular por `JwtAuthFilter`, que se encarga de extraer y validar el token JWT incluido en la cabecera `Authorization`.

Una vez validada la autenticación, la petición es dirigida al **controlador REST** correspondiente, que actúa como punto de entrada. En esta fase se validan los datos de entrada (parámetros, cuerpo de la petición) y se gestionan posibles errores de formato o validación. Posteriormente, el controlador delega la ejecución en la capa de **servicios**, donde se implementa la lógica de

negocio. El servicio aplica las reglas del dominio, coordina operaciones entre distintas entidades, e interactúa con los repositorios para acceder o modificar los datos. Los **repositorios** realizan las operaciones necesarias sobre la base de datos PostgreSQL mediante Spring Data JPA.

Una vez obtenidos los resultados, estos se devuelven al controlador, que los transforma a su correspondiente DTO. Finalmente, se construye la respuesta HTTP en formato JSON, incluyendo el código de estado adecuado. Este flujo garantiza una separación clara de responsabilidades entre las distintas capas del sistema, facilitando la mantenibilidad, la escalabilidad y la testabilidad de la aplicación.

4.3.2 Implementación del *front-end*

El *front-end* de MovieMate se ha implementado como una **Single Page Application (SPA)** desarrollada con React 19 y TypeScript. Esta arquitectura permite construir una experiencia de usuario fluida, evitando recargas completas de página y delegando la renderización en el cliente.

La aplicación sigue un enfoque **modular orientado a características (*feature-based*)**, en el que el código se organiza en módulos independientes que agrupan páginas, componentes, *hooks* y lógica de acceso a datos asociados a una funcionalidad concreta. Este enfoque contrasta con estructuras tradicionales por capas (por tipo de archivo) y favorece una mayor cohesión interna y escalabilidad a medida que crece la aplicación. Además, esta estrategia mejora la escalabilidad del *front-end*, permitiendo que nuevas funcionalidades puedan incorporarse de forma independiente sin afectar al resto del sistema.

4.3.2.1 Estructura del proyecto

La raíz del código fuente (`src/`) se organiza en los siguientes directorios principales (ver Figura 20):

- **pages/** — contiene las vistas completas de la aplicación, cada una asociada a una ruta navegable (por ejemplo, `HomePage`, `ProfilePage`, `DiscoverPage`).
- **components/** — incluye componentes reutilizables y desacoplados (tarjetas de contenido, modales, botones), utilizados transversalmente en distintas partes de la aplicación.
- **hooks/** — agrupa *hooks* personalizados que encapsulan lógica reutilizable de la interfaz, como gestión de formularios o composición de estados derivados.
- **api/** — centraliza la comunicación con el back-end mediante módulos organizados por recurso (`tmdbApi`, `usersApi`, `listsApi`, `ratingsApi`, etc.).
- **stores/** — gestiona el estado global sincrónico mediante Zustand, destacando el *store* de autenticación (`useAuthStore`).
- **lib/** — contiene utilidades compartidas, como la instancia global de Axios, la definición de claves de caché (`queryKeys`) y funciones auxiliares.
- **types/** — define los tipos TypeScript compartidos, asegurando consistencia en toda la aplicación.

Esta organización mejora la mantenibilidad del código al reducir el acoplamiento entre módulos y facilitar la localización de funcionalidades específicas durante el desarrollo.

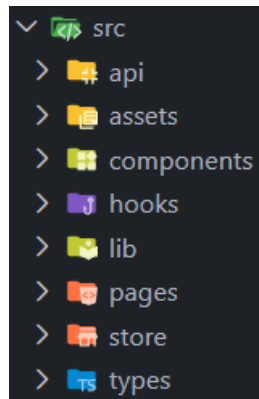


Figura 20. Árbol de directorios del proyecto *front-end* de MovieMate

El *front-end* se compone de varias páginas principales: inicio, descubrimiento, ficha de detalle de contenido, perfil de usuario, listas personalizadas, actividad y panel de administración. Cada una de ellas se corresponde con un componente de página conectado a React Router y construido a partir de componentes funcionales reutilizables.

La página de inicio es uno de los componentes más representativos del *front-end* de MovieMate, ya que integra el uso de prácticamente todas las capas de la arquitectura del cliente: el módulo *api*, la biblioteca de gestión del estado global *store*, las utilidades de *lib* y los tipos de *types*. A continuación, se describe cómo se construye esta página a partir de estas capas, como ejemplo del patrón general aplicado en el *front-end*.

4.3.2.2 Página de inicio

La página de inicio (HomePage, ruta /), representada en la Figura 21, es el punto de entrada principal de la aplicación. Para usuarios no autenticados muestra un bloque de bienvenida con una llamada a la acción que invita al registro. Para usuarios autenticados ofrece tres secciones de contenido personalizado:

- **Contenido en tendencia:** carrusel horizontal alimentado por el *endpoint* `GET /api/tmdb/trending`, que muestra los títulos cinematográficos y televisivos más populares del momento.
- **Recomendaciones personalizadas («Para ti 🌟»):** carrusel generado a partir del historial de valoraciones del usuario mediante `GET /api/users/me/recommendations`. El algoritmo selecciona los géneros más frecuentes en las valoraciones altas del usuario y recupera títulos relacionados que aún no han sido valorados.
- **Usuarios sugeridos:** lista de perfiles cinéfilos propuestos para seguir, fomentando la dimensión social de la plataforma.

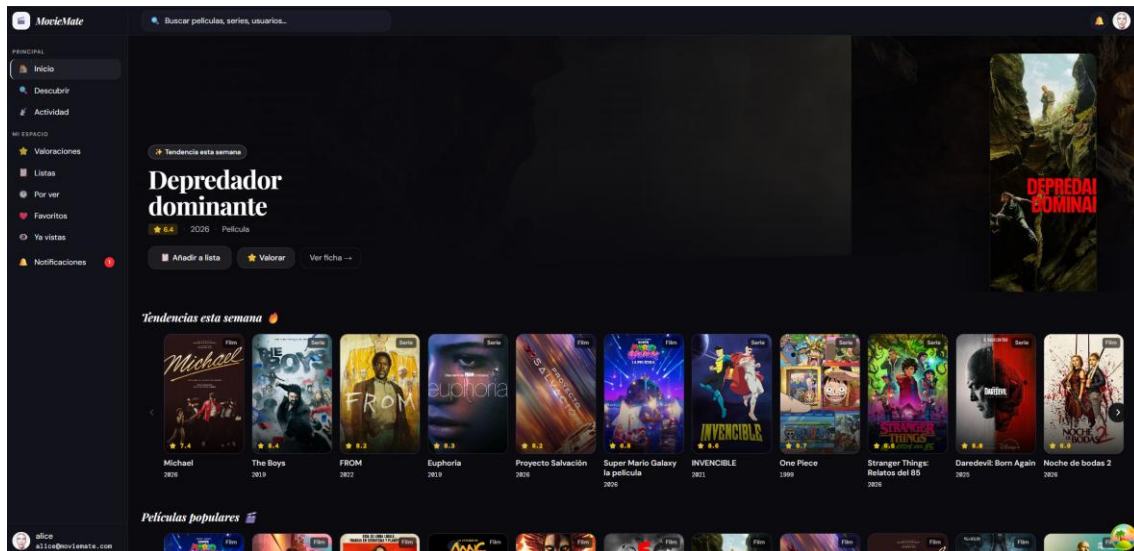


Figura 21. Página de inicio de MovieMate para un usuario autenticado



Figura 22. Fragmento de definición de tipos ContentResponse y UserResponse

Los datos que maneja la página de inicio están tipados en `src/types/index.ts`, que centraliza todas las interfaces del *front-end*. En este caso, las respuestas de contenido y usuario se representan mediante `ContentResponse` y `UserResponse`, como se muestra en la Figura 22. Estas interfaces actúan como contrato entre la capa de API y los componentes de la interfaz, garantizando que los datos consumidos por el *front-end* son coherentes con los DTOs expuestos por el *back-end*.

La comunicación con el *back-end* se centraliza en `src/lib/apiClient.ts`, que configura una instancia de `Axios` con la URL base del entorno, un *timeout* de petición y dos interceptores: uno que añade automáticamente el token JWT en cada petición, y otro que gestiona de forma global los errores de autenticación, como se muestra en la Figura 23.

```

1 // Lee la URL base del fichero .env correspondiente al entorno
2 const apiClient = axios.create({
3   baseURL: import.meta.env.VITE_API_BASE_URL,
4   headers: { 'Content-Type': 'application/json' },
5   timeout: 15000, // 15 segundos máximo por petición
6 })
7
8 // — Interceptor de REQUEST —
9 // Añade el token JWT automáticamente en cada petición
10 apiClient.interceptors.request.use(
11   (config) => {
12     const token = localStorage.getItem('mm_token')
13     if (token) {
14       config.headers.Authorization = `Bearer ${token}`
15     }
16     return config
17   },
18   (error) => Promise.reject(error)
19 )
20
21 // — Interceptor de RESPONSE —
22 // Maneja errores globales sin romper el router de React
23 apiClient.interceptors.response.use(
24   (response) => response,
25   (error: AxiosError) => {
26     if (error.response?.status === 401) {
27       // Token expirado o inválido: limpia la sesión
28       localStorage.removeItem('mm_token')
29       // Emite un evento personalizado que el componente raíz escucha
30       window.dispatchEvent(new CustomEvent('mm:unauthorized'))
31     }
32     // Rechaza con el error original para que React Query lo propague
33     return Promise.reject(error)
34   }
35 )

```

Figura 23. Configuración del cliente HTTP `apiClient`

```

1 // TmdbController devuelve la entidad Content directamente
2 // que tiene la misma forma que ContentResponse
3 export const tmdbApi = {
4   // Películas populares — para la HomePage y DiscoverPage
5   getPopularMovies: (page = 1) =>
6     apiClient.get<ContentResponse[]>('/tmdb/movies/popular', { params: { page } }),
7
8   // Series populares
9   getPopularTvShows: (page = 1) =>
10     apiClient.get<ContentResponse[]>('/tmdb/tv/popular', { params: { page } }),
11
12   // Búsqueda de películas
13   searchMovies: (query: string, page = 1) =>
14     apiClient.get<ContentResponse[]>('/tmdb/movies', { params: { query, page } }),
15
16   // Búsqueda de series
17   searchTvShows: (query: string, page = 1) =>
18     apiClient.get<ContentResponse[]>('/tmdb/tv', { params: { query, page } }),
19
20   // Trending (mix películas + series) — GET /api/tmdb/trending
21   getTrending: (page = 1) =>
22     apiClient.get<ContentResponse[]>('/tmdb/trending', { params: { page } }),
23
24   // Géneros
25   getMovieGenres: () =>
26     apiClient.get<GenreDto[]>('/tmdb/genres/movies'),

```

Figura 24. Módulo `tmdbApi` con funciones tipadas para obtener contenido popular y tendencias

Sobre esta instancia se construyen los módulos de API por dominio. Para la página de inicio, el módulo `src/api/tmdb.ts` expone funciones tipadas que mapean los *endpoints del backend* relacionados con TMDb, como se muestra en la Figura 24.

```

1  export const queryKeys = {
2    // TMDb — búsqueda y populares (datos directos de TMDb, sin BD)
3    tmdb: {
4      popularMovies: () => ['tmdb', 'popular', 'movies'] as const,
5      popularTv:    () => ['tmdb', 'popular', 'tv'] as const,
6      trending:    () => ['tmdb', 'trending'] as const,
7      searchMovies: (q: string) => ['tmdb', 'search', 'movies', q] as const,
8      searchTv:    (q: string) => ['tmdb', 'search', 'tv', q] as const,
9      sync: (tmdbId: number, type: string) => ['tmdb', 'sync', type, tmdbId] as const,
10     genresMovies: () => ['tmdb', 'genres', 'movies'] as const,
11     genresTv:    () => ['tmdb', 'genres', 'tv'] as const,
12     discoverMovies: (params: object) => ['tmdb', 'discover', 'movies', params] as const,
13     discoverTv:    (params: object) => ['tmdb', 'discover', 'tv', params] as const,
14     watchProviders: (tmdbId: number, type: string) => ['tmdb', 'providers', type, tmdbId] as const,
15     person: (id: number) => ['tmdb', 'person', id] as const,
16     personCredits: (id: number) => ['tmdb', 'person', id, 'credits'] as const,
17     contentCredits: (tmdbId: number, type: string) => ['tmdb', type, tmdbId, 'credits'] as const,
18     tvSeasons: (tmdbId: number) => ['tmdb', 'tv', tmdbId, 'seasons'] as const,
19     seasonDetail: (tmdbId: number, season: number) => ['tmdb', 'tv', tmdbId, 'seasons', season] as const,
20   },
21   // Episodios vistos (por usuario autenticado)
22   episodes: {
23     watched: (tmdbSeriesId: number) => ['episodes', 'watched', tmdbSeriesId] as const,
24     progress: () => ['episodes', 'watched', 'summary'] as const,
25   },

```

Figura 25. Definición de claves de caché de TanStack Query en `queryKeys` (extracto)

```

1  export const useAuthStore = create<AuthStore>((
2    persist(
3      (set) => ({
4        // Estado inicial
5        sessionUser: null,
6        token: null,
7        isAuthenticated: false,
8        isInitialized: false,
9      })
10     // Acciones
11     setAuth: (username, email, token) => {
12       localStorage.setItem('mm_token', token)
13       set({ sessionUser: { username, email }, token, isAuthenticated: true })
14     },
15     logout: () => {
16       localStorage.removeItem('mm_token')
17       set({ sessionUser: null, token: null, isAuthenticated: false })
18     },
19     setInitialized: () => set({ isInitialized: true }),
20   })
21   {
22     name: 'mm-auth',
23     storage: createJSONStorage(() => localStorage),
24     // Solo persiste estos campos (no isInitialized)
25     partialize: (state) => ({
26       sessionUser: state.sessionUser,
27       token: state.token,
28       isAuthenticated: state.isAuthenticated,
29     }),
30   }
31 )
32 )
33 )
34 )

```

Figura 26. Store de autenticación `authStore`

Las claves de caché de TanStack Query se centralizan en `src/lib/queryKeys.ts`, evitando cadenas literales dispersas por el código y facilitando la invalidación selectiva del estado, como se muestra en la Figura 25.

El estado de sesión del usuario se gestiona con Zustand en `src/store/authStore.ts`. Este *store* persiste el token JWT y el estado de autenticación en `localStorage`, y expone las acciones `setAuth` y `logout` que consumen las páginas de autenticación y el componente raíz de la aplicación, como se muestra en la Figura 26.

La página de inicio (`src/pages/home/HomePage.tsx`) orquesta todas estas capas. En su cuerpo principal lanza múltiples consultas en paralelo usando `useQuery` de TanStack Query, cada una con su clave de caché de `queryKeys` y su función de obtención de datos de `tmdbApi` o `usersApi`. Además, consume el store de autenticación para mostrar contenido personalizado únicamente a los usuarios autenticados, como se muestra en la Figura 27.



```
1 export default function HomePage() {
2   const navigate = useNavigate()
3   const { isAuthenticated } = useAuthStore()
4
5   const { data: trending, isLoading: loadingTrending } = useQuery({
6     queryKey: queryKeys.tmdb.trending(),
7     queryFn: () => tmdbApi.getTrending(),
8     select: (res) => res.data,
9     staleTime: 1000 * 60 * 10,
10  })
11
12  const { data: popularMovies, isLoading: loadingMovies } = useQuery({
13    queryKey: queryKeys.tmdb.popularMovies(),
14    queryFn: () => tmdbApi.getPopularMovies(),
15    select: (res) => res.data,
16    staleTime: 1000 * 60 * 10,
17  })
18
19  const { data: popularTv, isLoading: loadingTv } = useQuery({
20    queryKey: queryKeys.tmdb.popularTv(),
21    queryFn: () => tmdbApi.getPopularTvShows(),
22    select: (res) => res.data,
23    staleTime: 1000 * 60 * 10,
24  })
25
26  const { data: topRatedMovies, isLoading: loadingTopMovies } = useQuery({
27    queryKey: queryKeys.tmdb.discoverMovies({ sortBy: 'vote_average.desc', minRating: 7.5 }),
28    queryFn: () => tmdbApi.discoverMovies({ sortBy: 'vote_average.desc', minRating: 7.5 }),
29    select: (res) => res.data,
30    staleTime: 1000 * 60 * 30,
31  })
32 }
```

Figura 27. Fragmento de `HomePage`

Cada `useQuery` define de forma declarativa qué datos obtener, cuánto tiempo mantenerlos en caché (*staleTime*) y cuándo ejecutarse (*enabled*). El resultado se pasa directamente al componente `ContentCarousel`, definido en el mismo archivo, que gestiona el desplazamiento horizontal y los estados de carga mediante esqueletos de UI, como se muestra en la Figura 28.

En este fragmento se aprecia que `ContentCarousel` recibe como *prop* el array de `ContentResponse[]` (tipo definido en *types*) y delega la presentación de cada elemento en el componente reutilizable `PosterCard`, ubicado en `src/components/shared/`. Este patrón de separación entre página, componente reutilizable y capa de datos se repite de forma consistente en el resto de páginas de las MovieMate.

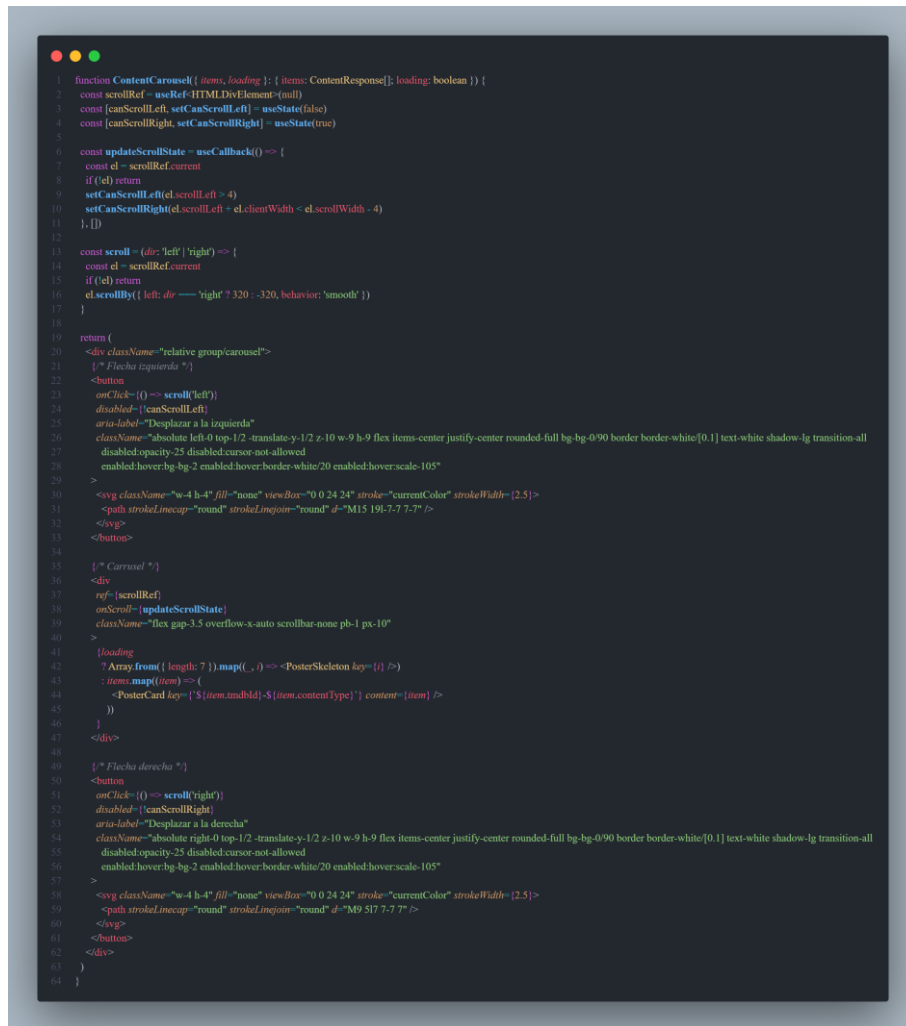


Figura 28. Componente ContentCarousel

Aunque en la página de inicio se trabaja directamente con `useQuery` de TanStack Query, en otras partes de la aplicación se encapsula esta lógica en *hooks* personalizados definidos en `src/hooks`, lo que permite reutilizar patrones comunes de obtención de datos y manejo de estados de carga y error. Un ejemplo representativo es el módulo `useDetail.ts`, utilizado en la ficha de detalle de contenido, que agrupa en un único lugar las operaciones relacionadas con la sincronización del contenido con la base de datos, la gestión de valoraciones y la obtención de créditos y proveedores de visionado como se puede observar en la Figura 29.

En este fragmento puede observarse cómo los *hooks* personalizados combinan TanStack Query con los módulos de API (`tmbdApi`, `ratingsApi`) y las claves de caché definidas en `queryKeys`. De este modo, las páginas y componentes que representan la ficha de detalle pueden reutilizar esta lógica de sincronización y mutación sin duplicar código ni preocuparse por la invalidación manual de la caché.

Además de la página de inicio, la aplicación incluye otras vistas principales como la página de descubrimiento, la ficha de detalle de contenido, el perfil de usuario, la gestión de listas personalizadas, el feed de actividad, la bandeja de notificaciones y los paneles de administración. La descripción funcional detallada de estas pantallas, junto con capturas de interfaz, se recoge en el Anexo C – Manual de usuario.

```

1 // Carga el contenido sincronizando con BD para tener stats actualizadas (appRating, appVoteCount)
2 export function useSyncContent(
3   tmdbId: number,
4   contentType: 'MOVIE' | 'TV',
5   enabled: boolean
6 ) {
7   return useQuery({
8     queryKey: queryKeys.tmdb.sync(tmdbId, contentType),
9     queryFn: () =>
10       contentType === 'MOVIE'
11         ? tmdbApi.syncMovie(tmdbId)
12         : tmdbApi.syncTvShow(tmdbId),
13     select: (res) => res.data,
14     enabled,
15     staleTime: 0, // siempre refresca al montar para tener appRating/appVoteCount frescos
16     gcTime: 1000 * 60 * 5,
17   })
18 }
19
20 // Crear o actualizar valoración
21 export function useCreateRating(content: ContentResponse) {
22   const queryClient = useQueryClient()
23
24   return useMutation({
25     mutationFn: (data: Omit<RatingRequest, 'tmdbId' | 'contentType'>) =>
26       ratingsApi.createOrUpdate({
27         ...data,
28         tmdbId: content.tmdbId,
29         contentType: content.contentType,
30       }),
31     onSuccess: () => {
32       if (content.id) {
33         queryClient.invalidateQueries({ queryKey: queryKeys.ratings.byContent(content.id) })
34       }
35       queryClient.invalidateQueries({ queryKey: queryKeys.users.ratings() })
36       // Invalidar sync para que appRating/appVoteCount se actualicen en el aside
37       queryClient.invalidateQueries({ queryKey: queryKeys.tmdb.sync(content.tmdbId, content.contentType) })
38       toast.success('Valoración guardada')
39     },
40     onError: (err: any) => {
41       toast.error(err?.response?.data?.message || 'Error al guardar la valoración')
42     },
43   })
44 }

```

Figura 29. Hooks useSyncContent y useCreateRating para la ficha de detalle

4.3.2.3 Navegación y autenticación

La navegación entre páginas se gestiona mediante **React Router v6**. Las rutas se declaran de forma centralizada en el componente raíz `App.tsx`, distinguiendo entre **rutas públicas** (accesibles sin sesión) y **rutas protegidas** (que requieren autenticación). El acceso a estas últimas se controla mediante el componente guardián **PrivateRoute**, que comprueba el estado de autenticación almacenado en Zustand y redirige al usuario a la pantalla de inicio de sesión si no dispone de sesión activa.

El flujo de autenticación sigue los pasos siguientes: el usuario introduce sus credenciales en el formulario de *login*; la petición se envía al *endpoint* `POST /api/auth/login`; si las credenciales son válidas, el servidor devuelve un token JWT que se almacena tanto en el store de Zustand (estado en memoria) como en el almacenamiento local del navegador (persistencia entre recargas de página); a partir de ese momento, una instancia global de Axios incorpora automáticamente el token como cabecera *Authorization: Bearer <token>* en cada petición autenticada.

4.3.2.4 Gestión del estado y comunicación con el servidor

La gestión del estado se divide en dos capas claramente separadas para evitar mezclar lógica de servidor y estado local en los componentes.

El **estado global sincrónico**, como los datos del usuario autenticado y el token JWT, se gestiona mediante Zustand. Su *store* reactivo de bajo peso permite que cualquier componente suscriba cambios de forma eficiente sin renderizados innecesarios.

El **estado asíncrono** derivado del servidor se gestiona con **TanStack Query v5**, que automatiza el ciclo de vida de las peticiones HTTP: almacena los resultados en caché mediante un sistema de claves estructuradas (`queryKeys`), ejecuta revalidaciones transparentes al recuperar el foco de la ventana o superar el tiempo de *stale*, y expone *hooks* de mutación (`useMutation`) que invalidan selectivamente las entradas de caché afectadas al confirmar un cambio. Por ejemplo, al añadir contenido a una lista se invalidan simultáneamente las claves `lists.mine()` y `users.lists()`, garantizando que todos los componentes suscritos reflejen el cambio de inmediato.

Todas las comunicaciones HTTP se canalizan a través de módulos de API dedicados que emplean la instancia global de Axios preconfigurada con el interceptor de autenticación y la URL base del servidor.

4.3.2.5 Sistema de diseño y adaptabilidad

La interfaz de MovieMate se construye sobre un sistema de diseño coherente definido mediante tokens de Tailwind CSS 4. Los principales tokens son:

- Paleta oscura estructurada: clases `bg-bg-0` a `bg-bg-3` para los distintos niveles de profundidad de los fondos, con gradaciones del azul marino profundo al gris carbón.
- Acento dorado (`#e8c97a`): empleado como color primario de interacción en botones de acción, estrellas de valoración, iconos activos y elementos de énfasis.
- Tipografía dual: *Playfair Display* para encabezados y títulos de películas, DM Sans para el cuerpo del texto y controles, DM Mono para fechas y datos numéricos.

Los componentes accesibles de Radix UI y Shadcn/ui garantizan la compatibilidad con lectores de pantalla y el cumplimiento de los estándares WCAG. La interfaz es completamente adaptable (*responsive*): en pantallas móviles aparece una barra de navegación inferior fija (`BottomNavBar`, visible solo en resoluciones inferiores a `lg`), mientras que en escritorio se emplea una barra de navegación lateral. Esta adaptabilidad se logra exclusivamente con Tailwind CSS sin necesidad de librerías de *layout* adicionales y se puede observar en la Figura 30.

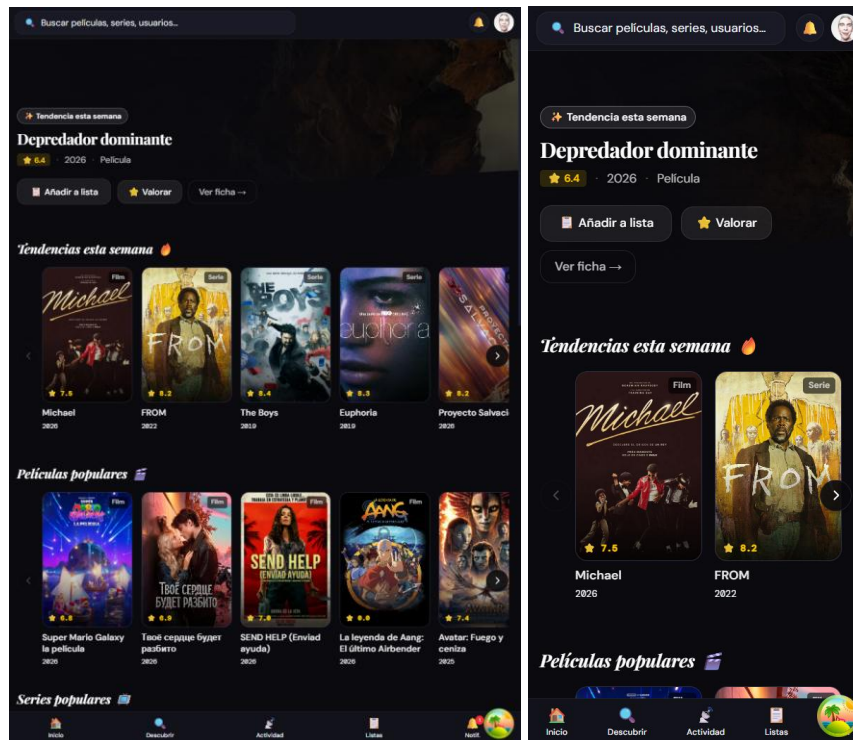


Figura 30. Interfaz de MovieMate en un iPad Air y un iPhone 12 Pro

Este sistema de diseño garantiza la coherencia visual de toda la aplicación y facilita la reutilización de componentes, reduciendo el esfuerzo de desarrollo y mantenimiento.

En conjunto, la arquitectura del *front-end* de MovieMate permite construir una interfaz modular, escalable y altamente interactiva, alineada con las necesidades de una aplicación social centrada en el consumo y la organización de contenido audiovisual.

4.4 Pruebas

La calidad del software desarrollado en MovieMate se ha reforzado mediante la incorporación de un conjunto de pruebas automatizadas orientadas a la validación de la lógica de presentación, negocio, persistencia y seguridad del sistema. Para la implementación de las pruebas se ha utilizado **JUnit 5** como *framework* de *testing* y **Mockito** como herramienta de simulación (*mocking*) de dependencias. Este enfoque permite sustituir los repositorios reales por objetos simulados, garantizando que las pruebas evalúan exclusivamente el comportamiento del servicio/controlador bajo condiciones controladas.

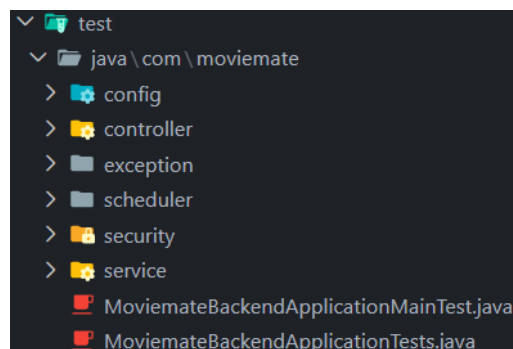


Figura 31. Estructura de la suite de pruebas del *back-end* de MovieMate

La suite de pruebas para el *back-end* se distribuye en 6 carpetas enfocadas a cada capa, cada una con un gran número de clases de prueba, esta estructura se muestra en la Figura 31. En total, estas clases agrupan 340 métodos de prueba individuales.

Un ejemplo representativo es la clase de pruebas asociada al servicio de gestión de listas. En ella se validan distintos escenarios relacionados con la creación de listas personalizadas, la adición y eliminación de contenido, así como el control de permisos de acceso. Entre los casos cubiertos se incluyen:

- Creación correcta de una lista cuando el nombre es único.
- Detección de nombres duplicados.
- Restricción de listas de tipo único por usuario.
- Control de acceso cuando un usuario intenta modificar una lista que no le pertenece.
- Gestión de errores cuando el contenido no existe.
- Prevención de inserciones duplicadas en una lista.

Como ejemplo representativo, en la Figura 32 se muestra un fragmento de código correspondiente a una prueba unitaria del servicio de listas, donde se valida la creación de una lista y la detección de duplicados mediante el uso de *mocks* sobre el repositorio. La prueba sigue el patrón Arrange-Act-Assert, separando claramente la preparación del escenario, la ejecución de la operación y la validación del resultado esperado.



```
1 // ----- createList -----
2
3 @Test
4 void createList_shouldCreateCustomList_whenNameIsUnique() {
5     User user = new User();
6     user.setId(1L);
7
8     ListRequest request = new ListRequest();
9     request.setName("Favoritas");
10    request.setDescription("Mis pelis favoritas");
11    request.setIsPublic(true);
12    request.setListType(List.ListType.CUSTOM);
13
14    when(listRepository.existsByUserAndName(user, "Favoritas")).thenReturn(false);
15    when(listRepository.findByUserAndListType(user, List.ListType.CUSTOM))
16        .thenReturn(Optional.empty());
17
18    ArgumentCaptor<List> listCaptor = ArgumentCaptor.forClass(List.class);
19    when(listRepository.save(listCaptor.capture())).thenAnswer(invocation -> {
20        List l = listCaptor.getValue();
21        l.setId(10L);
22        l.setUser(user);
23        l.setContents(Collections.emptyList());
24        return l;
25    });
26
27    ListResponse response = listService.createList(user, request);
28
29    assertThat(response.getId()).isEqualTo(10L);
30    assertThat(response.getName()).isEqualTo("Favoritas");
31    assertThat(response.getIsPublic()).isTrue();
32    assertThat(response.getListType()).isEqualTo(List.ListType.CUSTOM);
33
34    verify(listRepository).existsByUserAndName(user, "Favoritas");
35    verify(listRepository).save(any(List.class));
36 }
```

Figura 32. Fragmento de *ListServiceTest* mostrando un test de creación de lista

En este fragmento se puede observar cómo se simula el comportamiento del repositorio mediante Mockito, definiendo el resultado esperado de la consulta previa (`existsByNameAndUserId`). A continuación, se ejecuta el método del servicio y se verifica tanto el resultado como las interacciones realizadas con el repositorio, asegurando que la lógica de negocio se ejecuta correctamente. Asimismo, se comprueba el correcto mapeo de entidades a objetos de transferencia (DTOs), garantizando que la información devuelta al cliente cumple con el contrato definido por la API.

Este enfoque aporta múltiples beneficios, ya que permite detectar errores en etapas tempranas del desarrollo, reduce el riesgo de regresiones al introducir nuevas funcionalidades y facilita la refactorización del código al disponer de una red de seguridad automatizada. Asimismo, contribuye a mejorar la mantenibilidad del sistema al garantizar que los cambios realizados no rompen comportamientos previamente validados.

Las pruebas se ejecutan automáticamente durante el proceso de integración continua, asegurando que cualquier modificación en el código sea validada antes de su incorporación definitiva al repositorio principal. En conjunto, la adopción de pruebas unitarias automatizadas contribuye significativamente a la robustez y fiabilidad del sistema, alineándose con buenas prácticas de desarrollo profesional.

El pipeline de integración continua genera, además, de forma opcional, un informe de cobertura de código mediante JaCoCo, que mide el porcentaje de líneas, ramas y métodos de la capa de servicios ejercitados por los *test*. Este informe (ver Figura 33) constituye un indicador objetivo de la exhaustividad de la batería de pruebas.

moviemate-backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.moviemate.service		66 %		42 %	243 473	422 1.458	60 228	0 18
com.moviemate.config		65 %		52 %	17 51	83 276	2 33	0 7
com.moviemate.controller		97 %		68 %	10 106	4 227	0 90	0 14
com.moviemate.security		89 %		78 %	12 38	6 72	6 22	0 4
com.moviemate.entity		100 %	n/a	n/a	0 12	0 42	0 12	0 12
com.moviemate.exception		100 %	n/a	n/a	0 16	0 64	0 16	0 5
com.moviemate.dto		100 %	n/a	n/a	0 5	0 20	0 5	0 4
com.moviemate.scheduler		100 %	n/a	n/a	0 1	0 5	0 1	0 1
com.moviemate		100 %	n/a	n/a	0 2	0 3	0 2	0 1
Total	2.601 of 9.806	73 %	299 of 564	46 %	282 704	515 2.167	68 409	0 66

Figura 33. Informe de cobertura de test de JaCoCo

En conjunto, la estrategia de pruebas adoptada garantiza un alto nivel de fiabilidad del sistema, proporcionando una base sólida para la evolución y el mantenimiento de la aplicación.

4.5 Despliegue

Con el objetivo de garantizar la portabilidad, reproducibilidad y automatización del entorno de ejecución, el despliegue del *back-end* de MovieMate se ha basado en tecnologías de contenerización y en la integración continua mediante *pipelines* automatizados. Esta estrategia permite asegurar la coherencia entre los entornos de desarrollo y producción, simplificar la configuración de dependencias y automatizar tanto la validación como la construcción del proyecto. Además, facilita la escalabilidad futura del sistema al proporcionar una base sólida y reproducible para su despliegue.

4.5.1 Contenerización con Docker

Para encapsular la aplicación *back-end* se ha utilizado **Docker**. La aplicación se construye mediante un proceso *multi-stage* definido en el *Dockerfile*, lo que permite separar la fase de compilación de la fase de ejecución. En la primera etapa (*build stage*), se utiliza una imagen basada en Maven con Java 21, donde se compila el proyecto mediante el comando ``mvn clean package``, generando el archivo JAR ejecutable. En la segunda etapa (*runtime stage*), se emplea una imagen ligera basada en Eclipse Temurin JRE 21, en la que se crea un usuario no privilegiado para mejorar la seguridad y se copia únicamente el JAR generado previamente. Asimismo, se expone el puerto 8080 y se configura la ejecución mediante variables de entorno (``JAVA_OPTS``).

El uso de esta estrategia reduce significativamente el tamaño final de la imagen al eliminar dependencias innecesarias del entorno de compilación, lo que se traduce en mayor seguridad, imágenes más ligeras, entornos reproducibles y un adecuado aislamiento respecto al sistema anfitrión.

4.5.2 Orquestación con Docker Compose

Para facilitar el despliegue conjunto del *back-end* y la base de datos se ha utilizado **Docker Compose**, definiendo dos servicios principales.

Por un lado, el servicio de base de datos PostgreSQL se basa en la imagen oficial en su versión Alpine, incorporando persistencia mediante volúmenes Docker y configurando las credenciales a través de variables de entorno. Además, se incluye un mecanismo de *healthcheck* que permite verificar la disponibilidad del servicio antes de iniciar el *back-end*.

Por otro lado, el servicio del *back-end* se construye a partir del *Dockerfile* definido y establece una dependencia explícita del servicio de base de datos, garantizando el correcto orden de arranque. La configuración del servicio se realiza mediante variables de entorno, lo que permite desacoplar los parámetros sensibles y específicos del entorno del código fuente, cuyos detalles se describen en la sección de implementación del *back-end*.

La utilización de Docker Compose permite levantar todo el entorno con un único comando, simplificando la puesta en marcha del sistema, reduciendo errores de configuración manual y garantizando una comunicación segura entre contenedores mediante una red interna aislada.

4.5.3 Integración continua y automatización (CI/CD)

Con el objetivo de asegurar la calidad del código y automatizar el ciclo de construcción y despliegue del proyecto, se han definido dos *pipelines* diferenciados mediante **GitHub Actions**: uno de **integración continua** (CI) orientado a la validación del código y otro de **entrega continua** (CD) encargado de la construcción, publicación y despliegue de la aplicación.

El pipeline de integración continua se ejecuta automáticamente en cada *push* sobre ramas distintas de *main* y en las *pull requests* dirigidas a esta rama. Este flujo, que se puede observar en la Figura 34, tiene como objetivo validar que los cambios introducidos no rompen la funcionalidad existente y que el sistema es correctamente construible.

En primer lugar, se ejecuta un *job* de pruebas del *back-end*, en el que se configura el entorno con Java 21 y se lanzan los tests unitarios mediante Maven utilizando un perfil de pruebas

específico. Durante este proceso se genera además un informe de cobertura de código con JaCoCo, que se almacena como artefacto del *workflow* para su posterior consulta.

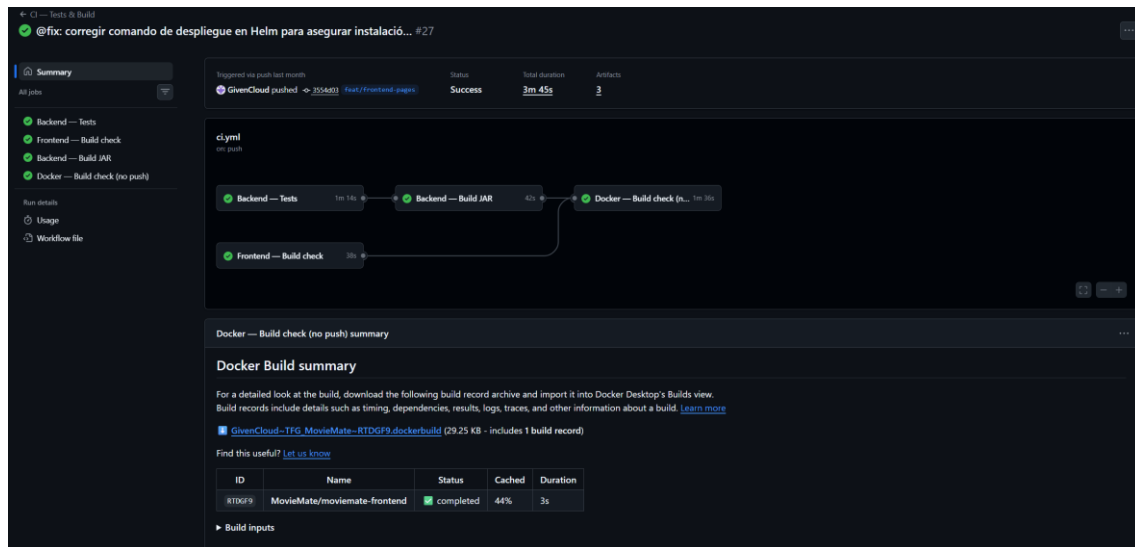


Figura 34. Pipeline CI de GitHub Actions

A continuación, se realiza la construcción del artefacto del *back-end*, generando el archivo JAR ejecutable. Este paso se ejecuta únicamente si las pruebas han finalizado correctamente, garantizando así la integridad del código compilado.

De forma paralela, se valida el *front-end* mediante un proceso de construcción que instala las dependencias con npm y ejecuta el *build* de la aplicación, verificando que el código TypeScript y la configuración de **Vite** [46] no presentan errores de compilación.

Finalmente, el *pipeline* incluye una fase de verificación de contenedores, en la que se construyen las imágenes Docker del *back-end* y del *front-end* sin publicarlas en ningún registro. Este paso permite validar que los Dockerfile son correctos y que la aplicación puede ser contenerizada antes de su integración en la rama principal.

Por otro lado, el *pipeline* de entrega continua, que se muestra en la Figura 35, se activa en cada *push* sobre la rama *main* y también puede ejecutarse manualmente. Este flujo comienza con una ejecución de pruebas del *back-end* como paso previo al despliegue, asegurando que la versión candidata mantiene la calidad del sistema. A continuación, se construyen las imágenes Docker del *back-end* y del *front-end* utilizando *Docker Buildx*, aplicando técnicas de cacheo para optimizar el tiempo de construcción. Estas imágenes se etiquetan tanto con el identificador único del *commit* como con la etiqueta *latest*, y se publican en el registro de contenedores **GitHub Container Registry** (GHCR).

Finalmente, el despliegue se realiza de forma manual mediante un *job* específico que se ejecuta en un runner *self-hosted* conectado a un clúster local de Kubernetes gestionado con Minikube; esto se explica más detalladamente en Anexo D - Despliegue en Kubernetes con Minikube. Este proceso utiliza **Helm** para actualizar o instalar los recursos del sistema, configurando dinámicamente las imágenes a desplegar en función del *commit* generado en el pipeline. Durante el despliegue se garantiza la autenticación contra el registro de

contenedores mediante un *secret* de Kubernetes, y se verifica el correcto arranque de los servicios mediante comprobaciones de estado (*rollout status*).

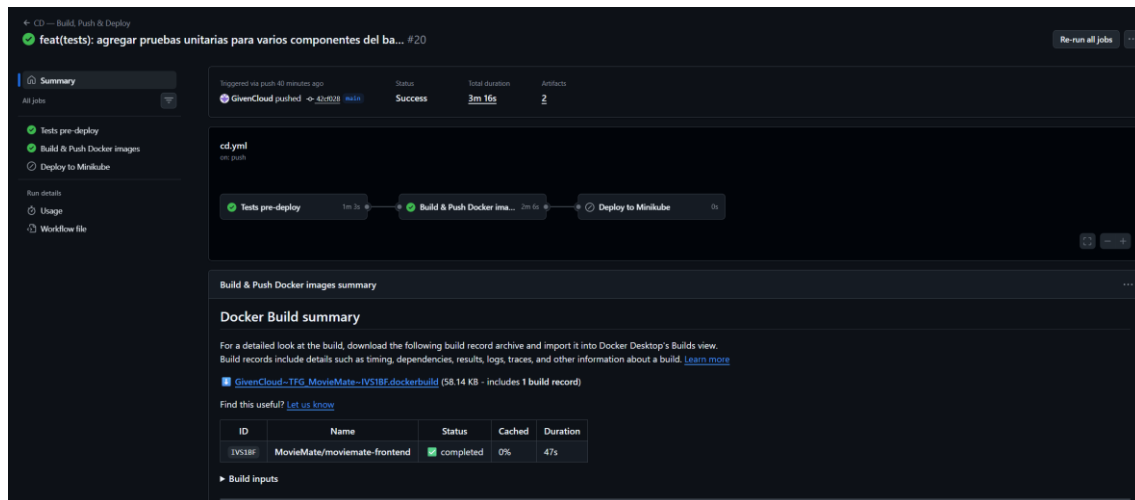


Figura 35. Pipeline CD de GitHub Actions

Esta separación entre integración continua y entrega continua permite desacoplar la validación del código del proceso de despliegue, proporcionando un flujo robusto.

4.5.4 Estrategia de ramificación

El proyecto sigue una estrategia de ramificación (*branching strategy*) basada en dos ramas permanentes. La rama *develop* concentra todo el desarrollo activo: cada nueva funcionalidad, corrección de errores o tarea de refactorización se implementa en esta rama antes de ser integrada. Sobre esta rama se ejecuta automáticamente el *pipeline* de integración continua, que valida el código mediante pruebas automatizadas y procesos de *build*.

La rama *main* representa la versión estable y desplegable del sistema. La integración de cambios desde *develop* a *main* se realiza mediante *pull requests*, asegurando que el código ha sido previamente validado. Cada actualización en *main* desencadena el *pipeline* de entrega continua, que construye y publica las imágenes Docker correspondientes, dejándolas listas para su despliegue.

Esta estrategia permite mantener un flujo de desarrollo controlado, garantizando que únicamente versiones verificadas del sistema alcanzan el entorno de producción, y facilitando la trazabilidad entre versiones desplegadas y *commits* específicos.

5 Conclusiones y vías futuras

5.1 Conclusiones

El desarrollo de MovieMate ha permitido construir una plataforma web completa para la gestión y valoración de contenido audiovisual, cumpliendo con los objetivos planteados al inicio del proyecto. Se ha implementado una aplicación *full-stack* funcional que integra de forma coherente un *back-end* basado en Spring Boot y una interfaz de usuario desarrollada con React y TypeScript, desplegable en un entorno contenerizado mediante Docker y con un *pipeline* de integración y entrega continua automatizado.

Desde el punto de vista técnico, el proyecto ha supuesto la aplicación práctica de un conjunto amplio de tecnologías y patrones propios del desarrollo web moderno. La arquitectura en capas adoptada en el *back-end* ha demostrado ser una elección acertada para un proyecto de esta escala, favoreciendo la separación de responsabilidades y la mantenibilidad del código. La estrategia de sincronización bajo demanda con la API de TMDb ha permitido reducir la dependencia de servicios externos y optimizar el rendimiento del sistema, almacenando el contenido únicamente cuando existe interacción real por parte de los usuarios.

En el ámbito del cliente, el uso de TanStack Query y Zustand ha simplificado considerablemente la gestión del estado asíncrono, reduciendo la complejidad asociada al desarrollo de aplicaciones de página única con múltiples fuentes de datos. La adopción de un sistema de diseño consistente ha resultado en una interfaz cohesiva, responsive y con una experiencia de usuario fluida, alineada con las expectativas de los usuarios de plataformas de referencia como Letterboxd o IMDb.

La incorporación de pruebas unitarias automatizadas y de un *pipeline* de integración continua ha reforzado la calidad del sistema, garantizando que las modificaciones al código son validadas de forma automática antes de su incorporación al repositorio principal.

A lo largo del desarrollo se ha hecho uso de herramientas de inteligencia artificial generativa, principalmente Claude como apoyo en la escritura de código, depuración de errores y redacción de fragmentos de documentación. Su uso ha requerido un juicio crítico constante: el código generado se ha verificado sistemáticamente, adaptado al contexto específico del proyecto y validado mediante la *suite* de pruebas automatizadas. Las herramientas de IA han actuado como acelerador del proceso de desarrollo, sin sustituir la toma de decisiones de diseño arquitectónico ni la comprensión profunda de las tecnologías empleadas.

En conjunto, MovieMate constituye una solución sólida dentro del alcance de un proyecto académico, demostrando la viabilidad de integrar múltiples tecnologías y enfoques arquitectónicos en el desarrollo de una aplicación web moderna orientada a la gestión y descubrimiento de contenido audiovisual.

5.2 Trabajo futuro

A pesar de que el sistema desarrollado cumple los objetivos planteados, presenta una serie de limitaciones que abren la puerta a futuras mejoras y extensiones.

En primer lugar, el sistema se limita actualmente a un entorno web, por lo que una línea de evolución natural sería el desarrollo de aplicaciones móviles nativas para iOS y Android. En este contexto, el uso de tecnologías como React Native permitiría reutilizar parte de la lógica del cliente, facilitando la expansión multiplataforma.

En segundo lugar, el sistema de recomendaciones implementado se basa en reglas simples derivadas del historial de valoraciones del usuario. Como mejora, podría incorporarse un motor de recomendaciones más avanzado basado en técnicas de filtrado colaborativo o aprendizaje automático, lo que permitiría ofrecer sugerencias más precisas y personalizadas.

Asimismo, la integración con plataformas de *streaming* constituye otra limitación actual del sistema. Incorporar información sobre la disponibilidad de contenido en servicios externos añadiría un valor significativo para el usuario final. En esta misma línea, podrían incorporarse funcionalidades adicionales como la planificación de visualizaciones en grupo, la ampliación del sistema de gamificación o la exportación del historial de actividad.

Finalmente, desde el punto de vista arquitectónico, el sistema se ha diseñado como un monolito modular, adecuado para el alcance actual del proyecto. No obstante, en un escenario con un volumen elevado de usuarios, podría evaluarse la transición hacia una arquitectura basada en microservicios, permitiendo desacoplar componentes críticos y escalar de forma independiente según la carga del sistema.

6 Bibliografía

- [1] Letterboxd. (2024). *Letterboxd - Social film discovery*. [Online]. Available: <https://letterboxd.com/> (último acceso: 8 de noviembre de 2025)
- [2] IMDb. (2024). *IMDb - Ratings, Reviews, and Where to Watch the Best Movies & TV Shows*. [Online]. Available: <https://www.imdb.com/> (último acceso: 8 de noviembre de 2025)
- [3] Serializd. (2024). *Serializd - The social platform for TV*. [Online]. Available: <https://serializd.com/> (último acceso: 8 de noviembre de 2025)
- [4] Nielsen. (2024). *El tiempo dedicado al streaming supera el 40 % en junio, la mayor cuota de uso de la televisión en la historia de The Gauge™ de Nielsen*. [Online]. Available: <https://www.nielsen.com/es/news-center/2024/time-spent-streaming-surges-to-over-40-percent-in-june-2024/> (último acceso: 8 de noviembre de 2025)
- [5] ADSLZone. (2025). *Mejores apps para cinéfilos: listas de películas por ver*. [Online]. Available: <https://www.adslzone.net/noticias/moviles/apps-control-peliculas-vistas-por-ver/> (último acceso: 8 de noviembre de 2025)
- [6] Robert Steiner. (2025). *Why Letterboxd became the go-to platform for online film buffs*. Variety. [Online]. Available: <https://variety.com/vip/letterboxd-year-end-report-growth-1236277320/> (último acceso: 8 de noviembre de 2025)
- [7] Spring. (2024). *Spring Boot documentation*. [Online]. Available: <https://spring.io/projects/spring-boot> (último acceso: 9 de noviembre de 2025)
- [8] Oracle. (2024). *Java Platform, Standard Edition*. [Online]. Available: <https://www.java.com/> (último acceso: 9 de noviembre de 2025)
- [9] PostgreSQL Global Development Group. (2024). *PostgreSQL Database*. [Online]. Available: <https://www.postgresql.org/> (último acceso: 9 de noviembre de 2025)
- [10] jsonwebtoken/jjwt Authors. (2024). *JWT – Java JWT: JSON Web Token for Java and Android*. [Online]. Available: <https://github.com/jwtkt/jjwt> (último acceso: 9 de noviembre de 2025)
- [11] Amazon Web Services. (2024). *¿Qué es una API de RESTful?*. [Online]. Available: <https://aws.amazon.com/es/what-is/restful-api/> (último acceso: 9 de noviembre de 2025)
- [12] OpenAPI Initiative. (2025). *OpenAPI Specification – Define standard, language-agnostic RESTful APIs*. [Online]. Available: <https://www.openapis.org/> (último acceso: 9 de noviembre de 2025)
- [13] Swagger. (2025). *API Development Tools*. [Online]. Available: <https://swagger.io/> (último acceso: 9 de noviembre de 2025)
- [14] H2 Database Engine. (2025). *H2 – Java SQL Database*. [Online]. Available: <https://www.h2database.com/> (último acceso: 9 de noviembre de 2025)
- [15] JaCoCo. (2025). *JaCoCo – Java Code Coverage Library*. [Online]. Available: <https://www.jacoco.org/jacoco/> (último acceso: 9 de noviembre de 2025)
- [16] Meta Open Source. (2024). *React – A JavaScript library for building user interfaces*. [Online]. Available: <https://react.dev/> (último acceso: 10 de noviembre de 2025)
- [17] Microsoft. (2024). *TypeScript – Typed JavaScript at Any Scale*. [Online]. Available: <https://www.typescriptlang.org/> (último acceso: 10 de noviembre de 2025)

- [18] Tailwind Labs. (2024). *Tailwind CSS Documentation*. [Online]. Available: <https://tailwindcss.com/> (último acceso: 8 de noviembre de 2025)
- [19] Zustand Contributors. (2024). Zustand – A small, fast and scalable bearbones state-management solution. [Online]. Available: <https://zustand-demo.pmnd.rs/> (último acceso: 8 de noviembre de 2025)
- [20] TanStack Query. (2024). *TanStack Query React Components Library*. [Online]. Available: <https://tanstack.com/query/latest> (último acceso: 8 de noviembre de 2025)
- [21] Axios Contributors. (2024). Axios – Promise based HTTP client for the browser and Node.js. [Online]. Available: <https://axios-http.com/> (último acceso: 8 de noviembre de 2025)
- [22] Remix Software. (2024). React Router – Declarative routing for React. [Online]. Available: <https://reactrouter.com/> (último acceso: 8 de noviembre de 2025)
- [23] Radix UI Contributors. (2024). Radix UI – Unstyled, accessible components for building high-quality design systems. [Online]. Available: <https://www.radix-ui.com/> (último acceso: 8 de noviembre de 2025)
- [24] shadcn. (2024). shadcn/ui – Beautifully designed components built with Radix UI and Tailwind CSS. [Online]. Available: <https://ui.shadcn.com/> (último acceso: 8 de noviembre de 2025)
- [25] SockJS Contributors. (2024). SockJS-client – WebSocket emulation library. [Online]. Available: <https://github.com/sockjs/sockjs-client> (último acceso: 8 de noviembre de 2025)
- [26] STOMP.js Contributors. (2024). @stomp/stompjs – STOMP client over WebSocket for JavaScript. [Online]. Available: <https://stomp-js.github.io/stomp-websocket/> (último acceso: 8 de noviembre de 2025)
- [27] Spring. (2024). Spring Framework WebSocket Support. [Online]. Available: <https://docs.spring.io/spring-framework/reference/web/websocket.html> (último acceso: 8 de noviembre de 2025)
- [28] The Movie Database (TMDB). (2024). *TMDB API and documentation*. [Online]. Available: <https://www.themoviedb.org/> (último acceso: 10 de noviembre de 2025)
- [29] Microsoft. (2024). *Visual Studio Code*. [Online]. Available: <https://code.visualstudio.com/> (último acceso: 10 de noviembre de 2025)
- [30] Git SCM. (2024). *Git – Distributed version control system*. [Online]. Available: <https://git-scm.com/> (último acceso: 10 de noviembre de 2025)
- [31] GitHub. (2024). *GitHub – Where the world builds software*. [Online]. Available: <https://github.com/> (último acceso: 10 de noviembre de 2025)
- [32] Postman. (2024). *Postman API Platform*. [Online]. Available: <https://www.postman.com/> (último acceso: 10 de noviembre de 2025)
- [33] pgAdmin Development Team. (2025). *pgAdmin 4 – PostgreSQL Tools and Management*. [Online]. Available: <https://www.pgadmin.org/> (último acceso: 10 de noviembre de 2025)
- [34] DBeaver Corp. (2025). *DBeaver – Universal Database Tool*. [Online]. Available: <https://dbeaver.com/> (último acceso: 10 de noviembre de 2025)
- [35] OpenAI. (2024). *ChatGPT*. [Online]. Available: <https://chatgpt.com/> (último acceso: 10 de noviembre de 2025)

- [36] JUnit Team. (2025). *JUnit 5 User Guide*. [Online]. Available: <https://junit.org/junit5/> (último acceso: 10 de noviembre de 2025)
- [37] Mockito Contributors. (2025). *Mockito – Tasty mocking framework for unit tests in Java*. [Online]. Available: <https://site.mockito.org/> (último acceso: 10 de noviembre de 2025)
- [38] Docker Inc. (2025). *Docker – Accelerated Container Application Development*. [Online]. Available: <https://www.docker.com/> (último acceso: 8 de noviembre de 2025)
- [39] Docker Inc. (2025). *Docker Compose – Define and run multi-container applications*. [Online]. Available: <https://docs.docker.com/compose/> (último acceso: 10 de noviembre de 2025)
- [40] GitHub. (2025). *GitHub Actions – Automate your workflow*. [Online]. Available: <https://github.com/features/actions> (último acceso: 10 de noviembre de 2025)
- [41] Anthropic. (2025). *Claude – AI Assistant by Anthropic*. [Online]. Available: <https://www.anthropic.com/claude> (último acceso: 10 de noviembre de 2025)
- [42] The Kubernetes Authors. (2024). *Kubernetes – Production-Grade Container Orchestration*. [Online]. Available: <https://kubernetes.io/> (último acceso: 14 de marzo de 2026)
- [43] Kubernetes SIGs. (2024). *Minikube – Run Kubernetes locally*. [Online]. Available: <https://minikube.sigs.k8s.io/> (último acceso: 14 de marzo de 2026)
- [44] Roy T. Fielding. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. [Online]. Available: https://www.ics.uci.edu/~fielding/pubs/dissertation/fielding_dissertation.pdf (último acceso: 22 de diciembre de 2025)
- [45] ClickIT. (2021). *Microservicios vs. Arquitectura Monolítica: Debate sobre ...* [Online]. Available: <https://www.clickittech.com/devops/microservices-vs-monolith/> (último acceso: 17 de diciembre de 2025)
- [46] Evan You & Vite Contributors. (2024). *Vite – Next Generation Frontend Tooling*. [Online]. Available: <https://vitejs.dev/> (último acceso: 27 de abril de 2026)

Anexo A – Especificación de casos de uso

Este anexo recoge la especificación formal de los casos de uso principales del sistema MovieMate. Cada caso de uso se documenta utilizando una plantilla estructurada que describe el actor implicado, la funcionalidad del sistema, el objetivo perseguido y el resultado esperado.

Esta documentación complementa los diagramas presentados en el capítulo 4.1, proporcionando mayor detalle sobre el comportamiento funcional del sistema.

A.1 Usuarios no registrados

Caso de uso: Consultar catálogo de contenido

Actor: Usuario no registrado

Descripción: Permite visualizar el catálogo general de películas y series disponibles en la plataforma.

Objetivo: Explorar el contenido disponible sin necesidad de autenticación.

Resultado: El sistema muestra el catálogo actualizado de películas y series.

Caso de uso: Consultar ficha de contenido

Actor: Usuario no registrado

Descripción: Permite visualizar la información detallada de una película o serie concreta.

Objetivo: Obtener detalles específicos de un contenido seleccionado.

Resultado: El sistema muestra la ficha completa del contenido.

Caso de uso: Explorar contenido destacado

Actor: Usuario no registrado

Descripción: Permite visualizar listados de contenido popular o recomendado.

Objetivo: Facilitar el descubrimiento de contenido relevante.

Resultado: El sistema muestra contenidos destacados.

Caso de uso: Visualizar perfil público

Actor: Usuario no registrado

Descripción: Permite consultar la información pública de otro usuario.

Objetivo: Conocer la actividad pública y listas visibles de otros usuarios.

Resultado: El sistema muestra el perfil público solicitado.

Caso de uso: Consultar listas públicas

Actor: Usuario no registrado

Descripción: Permite visualizar listas públicas creadas por otros usuarios.

Objetivo: Explorar contenido organizado por la comunidad.

Resultado: El sistema muestra la lista pública seleccionada.

Caso de uso: Registrarse

Actor: Usuario no registrado

Descripción: Permite crear una cuenta en la plataforma.

Objetivo: Obtener acceso a funcionalidades avanzadas.

Resultado: El sistema registra un nuevo usuario.

Caso de uso: Iniciar sesión

Actor: Usuario registrado

Descripción: Permite autenticarse en el sistema.

Objetivo: Acceder a funcionalidades personalizadas.

Resultado: El sistema valida las credenciales y concede acceso.

A.2 Interacción social

Caso de uso: Gestionar perfil de usuario

Actor: Usuario registrado

Descripción: Permite modificar la información personal del perfil.

Objetivo: Mantener actualizados los datos del usuario.

Resultado: El sistema guarda los cambios realizados.

Caso de uso: Configurar privacidad del perfil

Actor: Usuario registrado

Descripción: Permite definir si el perfil del usuario es público o privado.

Objetivo: Controlar quién puede visualizar la información del perfil.

Resultado: El sistema actualiza la configuración de privacidad.

Caso de uso: Seguir usuario

Actor: Usuario registrado

Descripción: Permite seguir a otro usuario de la plataforma con perfil público.

Objetivo: Recibir actualizaciones de actividad de otros usuarios.

Resultado: El sistema registra la relación de seguimiento.

Caso de uso: Solicitar seguimiento

Actor: Usuario registrado

Descripción: Permite solicitar seguir a un usuario con perfil privado.

Objetivo: Obtener acceso al contenido público y la actividad del usuario.

Resultado: El sistema registra la solicitud y genera una notificación.

Caso de uso: Aceptar solicitud de seguimiento

Actor: Usuario registrado

Descripción: Permite aprobar una solicitud de seguimiento recibida.

Objetivo: Autorizar el acceso al contenido permitido.

Resultado: El sistema establece la relación de seguimiento.

Caso de uso: Rechazar solicitud de seguimiento

Actor: Usuario registrado

Descripción: Permite denegar una solicitud de seguimiento.

Objetivo: Mantener la privacidad del perfil.

Resultado: El sistema elimina la solicitud pendiente.

Caso de uso: Dejar de seguir usuario

Actor: Usuario registrado

Descripción: Permite eliminar una relación de seguimiento.

Objetivo: Dejar de recibir actividad de otro usuario.

Resultado: El sistema elimina la relación de seguimiento.

Caso de uso: Consultar seguidores y seguidos

Actor: Usuario registrado

Descripción: Permite visualizar las relaciones sociales del usuario.

Objetivo: Conocer la red de conexiones dentro de la plataforma.

Resultado: El sistema muestra seguidores y usuarios seguidos.

Caso de uso: Consultar feed de actividad

Actor: Usuario registrado

Descripción: Permite visualizar la actividad reciente de usuarios seguidos.

Objetivo: Mantenerse informado sobre novedades sociales.

Resultado: El sistema muestra el feed personalizado.

Caso de uso: Consultar notificaciones

Actor: Usuario registrado

Descripción: Permite visualizar notificaciones relacionadas con actividad social.

Objetivo: Mantenerse informado de interacciones.

Resultado: El sistema muestra el listado de notificaciones.

Caso de uso: Marcar notificaciones como leídas

Actor: Usuario registrado

Descripción: Permite indicar que una notificación ha sido revisada.

Objetivo: Gestionar el estado de las notificaciones.

Resultado: El sistema actualiza el estado de lectura.

A.3 Listas personalizadas

Caso de uso: Crear lista personalizada

Actor: Usuario registrado

Descripción: Permite crear una lista de contenido.

Objetivo: Organizar películas y series según criterios personales.

Resultado: El sistema registra la nueva lista.

Caso de uso: Editar lista personalizada

Actor: Usuario registrado

Descripción: Permite modificar una lista existente.

Objetivo: Actualizar información o visibilidad de la lista.

Resultado: El sistema guarda los cambios realizados.

Caso de uso: Eliminar lista personalizada

Actor: Usuario registrado

Descripción: Permite borrar una lista propia.

Objetivo: Eliminar contenido organizado que ya no es necesario.

Resultado: El sistema elimina la lista.

Caso de uso: Añadir contenido a lista

Actor: Usuario registrado

Descripción: Permite incluir películas o series en una lista.

Objetivo: Organizar contenido visualizado o pendiente.

Resultado: El sistema añade el contenido a la lista.

Caso de uso: Eliminar contenido de lista

Actor: Usuario registrado

Descripción: Permite retirar contenido de una lista.

Objetivo: Mantener la lista actualizada.

Resultado: El sistema elimina el contenido seleccionado.

Caso de uso: Consultar listas del propio usuario

Actor: Usuario registrado

Descripción: Permite visualizar listas creadas por el usuario.

Objetivo: Visualizar las listas del usuario.

Resultado: El sistema muestra las listas del usuario.

Caso de uso: Consultar listas públicas

Actor: Usuario registrado

Descripción: Permite visualizar listas públicas de otros usuarios a los que sigue el usuario propio.

Objetivo: Visualizar las listas públicas de otros usuarios.

Resultado: El sistema muestra las listas públicas de usuarios a los que el usuario sigue.

Caso de uso: Comentar valoración

Actor: Usuario registrado

Descripción: Permite publicar un comentario sobre la reseña de otro usuario.

Objetivo: Fomentar la interacción social en torno a listas públicas.

Resultado: El sistema registra el comentario y una notificación al propietario de la lista.

A.4 Valoraciones y reseñas

Caso de uso: Crear valoración

Actor: Usuario registrado

Descripción: Permite valorar una película o serie.

Objetivo: Expresar opinión sobre contenido.

Resultado: El sistema registra la valoración.

Caso de uso: Editar valoración

Actor: Usuario registrado

Descripción: Permite modificar una valoración previa.

Objetivo: Actualizar opinión registrada.

Resultado: El sistema guarda la modificación.

Caso de uso: Eliminar valoración

Actor: Usuario registrado

Descripción: Permite borrar una valoración.

Objetivo: Retirar una opinión registrada.

Resultado: El sistema elimina la valoración.

Caso de uso: Consultar valoraciones propias

Actor: Usuario registrado

Descripción: Permite visualizar valoraciones creadas por el usuario.

Objetivo: Visualizar las opiniones publicadas.

Resultado: El sistema muestra valoraciones del usuario.

Caso de uso: Consultar valoraciones de contenido

Actor: Usuario registrado

Descripción: Permite visualizar valoraciones de otros usuarios.

Objetivo: Conocer opiniones de la comunidad.

Resultado: El sistema muestra valoraciones disponibles.

Caso de uso: Dar me gusta a valoración

Actor: Usuario registrado

Descripción: Permite indicar interés por una reseña.

Objetivo: Destacar contenido útil.

Resultado: El sistema registra la interacción y genera notificación.

Resultado: El sistema registra la interacción y genera notificación.

Caso de uso: Comentar valoración

Actor: Usuario registrado

Descripción: Permite publicar un comentario sobre la reseña de otro usuario.

Objetivo: Enriquecer el debate en torno a una valoración.

Resultado: El sistema registra el comentario y genera una notificación al autor de la reseña.

A.5 Administración y moderación

Caso de uso: Consultar usuarios del sistema

Actor: Administrador

Descripción: Permite visualizar el listado global de usuarios registrados en la plataforma.

Objetivo: Gestionar las cuentas existentes en el sistema.

Resultado: El sistema muestra el listado de usuarios registrados.

Caso de uso: Cambiar rol de usuario

Actor: Administrador

Descripción: Permite modificar el rol asociado a una cuenta de usuario.

Objetivo: Gestionar privilegios administrativos dentro de la plataforma.

Resultado: El sistema actualiza el rol del usuario seleccionado.

Caso de uso: Banear o desbanear usuario

Actor: Administrador

Descripción: Permite restringir o restaurar el acceso de un usuario a la plataforma.

Objetivo: Moderar comportamientos inapropiados o restaurar cuentas previamente bloqueadas.

Resultado: El sistema actualiza el estado de acceso del usuario.

Caso de uso: Consultar reportes de contenido

Actor: Administrador

Descripción: Permite visualizar reportes pendientes relacionados con valoraciones o comentarios denunciados por usuarios.

Objetivo: Revisar incidencias relacionadas con contenido inapropiado.

Resultado: El sistema muestra el listado de reportes pendientes.

Caso de uso: Eliminar valoración reportada

Actor: Administrador

Descripción: Permite eliminar una valoración considerada inapropiada tras revisar un reporte.

Objetivo: Mantener la calidad y adecuación del contenido publicado.

Resultado: El sistema elimina la valoración seleccionada.

Caso de uso: Eliminar comentario reportado

Actor: Administrador

Descripción: Permite eliminar un comentario que incumple las normas de la plataforma.

Objetivo: Moderar el contenido generado por los usuarios.

Resultado: El sistema elimina el comentario seleccionado.

Caso de uso: Resolver reporte

Actor: Administrador

Descripción: Permite marcar un reporte como resuelto tras aplicar una acción de moderación.

Objetivo: Gestionar el ciclo de vida de las incidencias reportadas.

Resultado: El sistema marca el reporte como resuelto.

Caso de uso: Desestimar reporte

Actor: Administrador

Descripción: Permite cerrar un reporte sin realizar acciones sobre el contenido denunciado.

Objetivo: Registrar incidencias revisadas que no requieren moderación.

Resultado: El sistema marca el reporte como desestimado.

Anexo B – Tabla con los *endpoints* de la aplicación

Tabla 4. *Endpoints* de la aplicación (verde: públicos; rojo: autenticados; azul: administrador)

Endpoint	GET	POST	PUT	DELETE	Descripción
/api/auth/register		✓			Registrar nuevo usuario
/api/auth/login		✓			Iniciar sesión
/api/users/me/profile			✓		Actualizar perfil de usuario
/api/users/me/public-status			✓		Actualizar visibilidad del perfil
/api/users	✓				Buscar usuarios por <i>query</i>
/api/users/{userId}	✓				Obtener un usuario por ID
/api/users/{userId}/stats	✓				Obtener estadísticas de un usuario
/api/users/username/{username}	✓				Obtener un usuario por nombre de usuario
/api/users/suggestions	✓				Obtener sugerencias de usuario
/api/users/me/password			✓		Cambiar contraseña del usuario
/api/users/me	✓				Obtener usuario actual
/api/users/me/ratings	✓				Obtener valoraciones hechas por el usuario
/api/users/me/lists	✓				Obtener las listas del usuario
/api/users/me/stats/full	✓				Obtener estadísticas completas del usuario
/api/users/me/notifications	✓				Obtener las notificaciones del usuario
/api/users/me/follow-requests	✓				Obtener las solicitudes de seguimiento

					enviadas al usuario
/api/users/{userId}/follow-requests		✓			Mandar solicitud de seguimiento a un usuario
/api/users/follow-requests/{requestId}		✓		✓	Aceptar/rechazar solicitud de seguimiento
/api/users/{userId}/following	✓				Obtener usuarios seguidos por el usuario
/api/users/{userId}/following-status	✓				Comprobar si sigo a un usuario
/api/users/{userId}/followers	✓			✓	Obtener seguidores del usuario
/api/lists		✓			Crear una lista
/api/lists/{listId}/content		✓			Añadir contenido a una lista
/api/lists/public	✓				Obtener todas las listas públicas
/api/lists/{listId}/content/{tmdbId}				✓	Eliminar contenido de una lista
/api/ratings		✓			Crear una valoración
/api/ratings/{ratingId}/likes	✓	✓			Obtener la cantidad de me gustas/Dar me gusta
/api/ratings/{ratingId}/like-status	✓				Comprobar si he dado me gusta a una valoración
/api/ratings/{contentId}	✓				Obtener las valoraciones de un contenido
/api/ratings/{ratingId}				✓	Eliminar una valoración
/api/feed/personal	✓				Obtener el <i>feed</i> personal del usuario
/api/feed/global	✓				Obtener el <i>feed</i> global
/api/notifications/{notificationId}/read			✓		Marcar una notificación como leída

/api/notifications/notifications/read-all			✓		Marcar todas las notificaciones como leídas
/api/notifications/notifications/unread-count	✓				Obtener notificaciones no leídas
/api/content	✓				Obtener todo el contenido
/api/content/{contentId}	✓				Obtener contenido por ID
/api/tmdb/tv/{tmdbId}/sync		✓			Sincronizar serie desde TMDB
/api/tmdb/movies/{tmdbId}/sync		✓			Sincronizar película desde TMDB
/api/tmdb/{contentType}/{tmdbId}/providers	✓				Obtener los proveedores de streaming
/api/tmdb/{contentType}/{tmdbId}/credits	✓				Obtener el cast de un contenido
/api/tmdb/tv	✓				Buscar series por nombre
/api/tmdb/tv/{tmdbId}/seasons	✓				Obtener resumen de temporadas de una serie
/api/tmdb/tv/{tmdbId}/seasons/{seasonNumber}	✓				Obtener episodios de una temporada
/api/tmdb/tv/popular	✓				Buscar series populares
/api/tmdb/movies	✓				Buscar películas por nombre
/api/tmdb/movies/popular	✓				Buscar películas populares
/api/tmdb/genres/tv	✓				Consultar los géneros de una serie
/api/tmdb/genres/movies	✓				Consultar los géneros de una película
/api/tmdb/discover/tv	✓				Descubrir series con filtro
/api/tmdb/discover/movies	✓				Descubrir películas con filtro
/api/users/me/recommendations	✓				Obtener recomendaci

					ones personalizada s
/api/users/me/badges	✓				Obtener insignias del usuario actual
/api/users/{userId}/badges	✓				Obtener insignias de un usuario
/api/users/me/avatar		✓			Subir foto de perfil (multipart/form-data)
/api/lists/{listId}			✓	✓	Editar o eliminar una lista propia
/api/tmdb/trending	✓				Obtener contenido en tendencia (películas + series)
/api/tmdb/people/{personId}	✓				Obtener el perfil de un actor
/api/tmdb/people/{personId}/credits	✓				Obtener la filmografía de un actor
/api/ratings/{ratingId}/comments	✓	✓			Obtener o crear comentarios de una valoración
/api/ratings/{ratingId}/comments/{commentId}				✓	Eliminar un comentario de valoración
/api/lists/{listId}/comments	✓	✓			Obtener o crear comentarios de una lista
/api/lists/{listId}/comments/{commentId}				✓	Eliminar un comentario de lista
/api/episodes/watched/summary	✓				Progreso de episodios vistos agrupado por serie
/api/episodes/watched/{tmdbSeriesId}	✓				Obtener episodios vistos de una serie
/api/episodes/watched/{tmdbSeriesId}/{seasonNumber}/{episodeNumber}		✓			Marcar o desmarcar un episodio como visto

				(toggle)
/api/episodes/watched/{tmdbSeriesId}/{seasonNumber}/all	✓		✓	Marcar o desmarcar toda una temporada como vista
/api/reports	✓			Crear un reporte de contenido inapropiado
/api/admin/users	✓			Listar todos los usuarios (solo admin)
/api/admin/users/{id}/role			✓	Cambiar el rol de un usuario
/api/admin/users/{id}/ban			✓	Banear o desbanear un usuario
/api/admin/ratings/{id}	✓		✓	Ver o eliminar una valoración (moderación)
/api/admin/comments/{id}	✓		✓	Ver o eliminar un comentario (moderación)
/api/admin/reports	✓			Obtener reportes pendientes de revisión
/api/admin/reports/{id}/resolve			✓	Resolver un reporte (elimina el contenido denunciado)
/api/admin/reports/{id}/dismiss			✓	Desestimar un reporte sin acción

Anexo C – Manual de usuario

C.1 Página de descubrimiento

La página de descubrimiento (*DiscoverPage*, ruta */discover*) centraliza la función de búsqueda de la aplicación. Dispone de una barra de búsqueda con un mecanismo de retardo en la ejecución de consultas (*debounce*), que evita realizar peticiones excesivas al servidor mientras el usuario escribe. Cuando la barra está vacía se muestran los contenidos en tendencia como estado por defecto. Además, dispone de algunos filtros para refinar la búsqueda de contenido según los intereses del usuario, como se puede ver en la Figura 36.

Los resultados se presentan en una cuadrícula de tarjetas (*ContentCard*) que muestran el póster, el título y el año de estreno. Cada tarjeta es un enlace a la ficha de detalle del contenido. La cuadrícula utiliza *flex flex-wrap* para adaptarse a cualquier resolución de pantalla sin dejar celdas vacías.

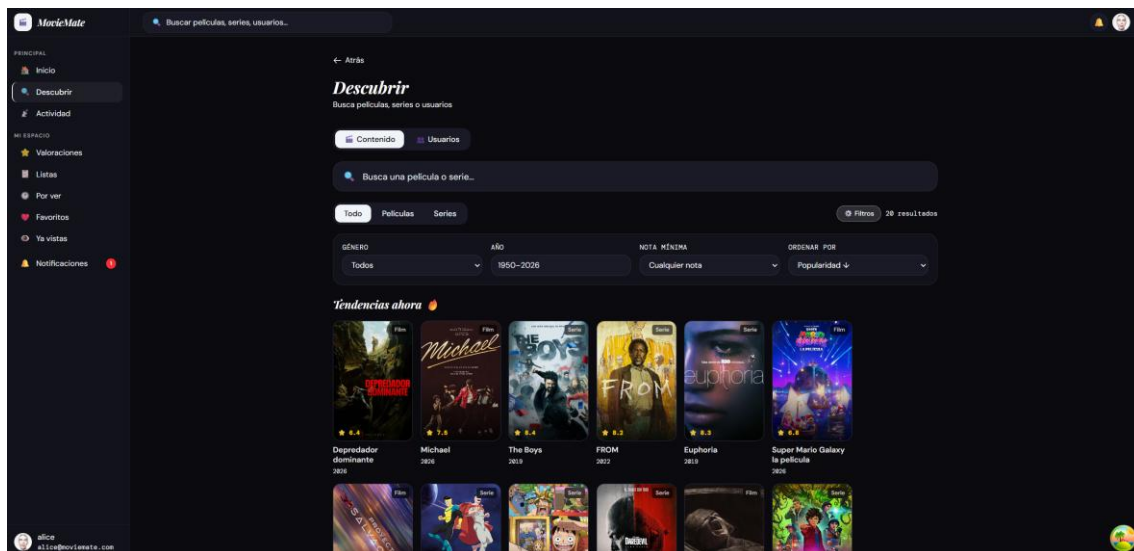


Figura 36. Página de descubrimiento de MovieMate

C.2 Ficha de detalle de contenido

La ficha de detalle de un contenido se muestra en la Figura 37 (*ContentDetailPage*, ruta */content/:contentType/:tmdbId/:slug*), es la vista más compleja de la aplicación y el núcleo de la experiencia de usuario. Recibe el tipo de contenido (MOVIE o TV) y el identificador de TMDB como parámetros de ruta. La página se estructura en los siguientes bloques:

- Cabecera (*ContentHeader*): muestra el *backdrop* a pantalla completa con degradado superpuesto, el póster a la izquierda y los metadatos principales a la derecha (título, año, géneros, puntuación TMDB, número de votos).
- Widget de valoración (*RatingWidget*): permite al usuario puntuar el contenido con 1 a 5 estrellas y adjuntar una reseña textual. Si el usuario ya realizó una valoración previa, la carga automáticamente para permitir su edición.

- Gestión de listas: botón que abre el diálogo `AddContentToListDialog`, desde el que el usuario puede añadir el contenido a cualquiera de sus listas (*Watchlist*, *Favoritos*, *Visto* o listas personalizadas).
- Reparto y equipo: galería de tarjetas `PersonCard` con los principales integrantes, cada una enlazada a la ficha de la persona (`/person/:personId/:slug`).
- Seguimiento de episodios (solo series TV): acordeón `SeasonAccordion` que despliega las temporadas y permite marcar episodios individuales como vistos, registrando la acción en la entidad `EpisodeWatch`.

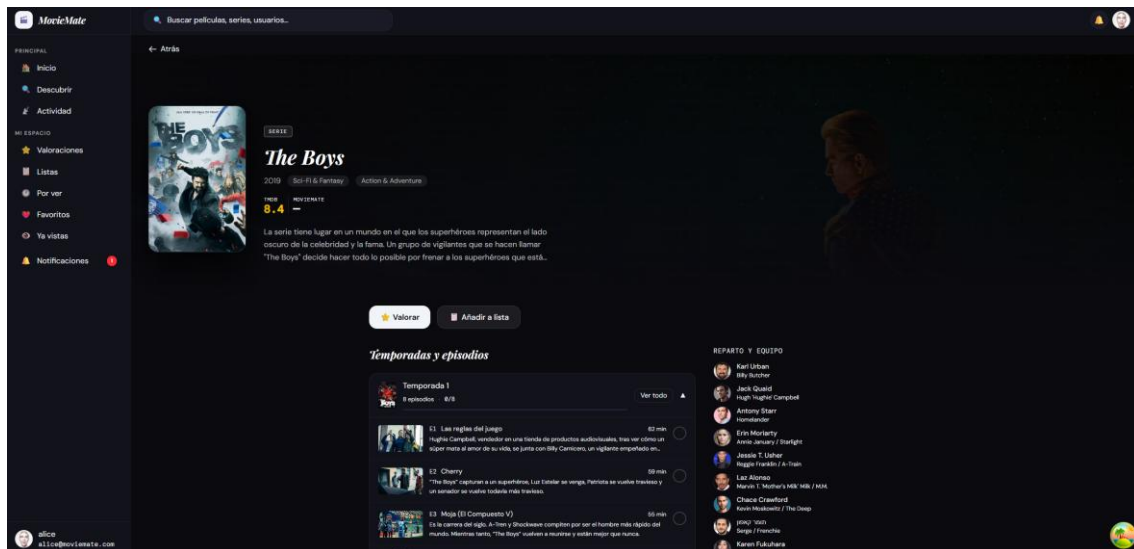


Figura 37. Ficha de detalle de contenido

Esta página concentra la mayor parte de la interacción del usuario con el contenido, actuando como núcleo funcional de la aplicación.

C.3 Perfil de usuario

La página de perfil (`ProfilePage`, ruta `/profile/:username`) muestra la información pública y la actividad de un usuario como se observa en la Figura 38. Soporta *deep-linking* mediante parámetros de consulta en la URL que permiten acceder directamente a una pestaña específica. El perfil se organiza en cuatro pestañas:

- Actividad: línea de tiempo cronológica con las valoraciones, reseñas y movimientos en listas del usuario, con paginación cliente en bloques de cinco elementos.
- Listas: cuadrícula de las listas del usuario con acceso directo a cada `ListDetailPage`.
- Estadísticas (`StatsTab`): visualizaciones avanzadas generadas a partir del *endpoint* `GET /api/users/:id/stats`, que devuelve un `FullStatsDto` con la distribución de valoraciones (1-5), los géneros más valorados y la actividad mensual.
- Insignias (`BadgesSection`): colección de insignias obtenidas por el usuario a través del uso de la plataforma, gestionadas por `BadgeService` con diez tipos definidos (primera valoración, primera lista, diez valoraciones, etc.).

Para perfiles privados o usuarios no autenticados la página gestiona el código HTTP 403 (`ProfilePrivateException`) mostrando un mensaje de perfil privado sin reintentar la petición. Los diálogos `FollowListDialog` muestran la lista de seguidores y de cuentas seguidas. El componente `QuickEditRatingDialog` permite editar una valoración directamente desde la cuadrícula de pósteres sin abandonar la página.

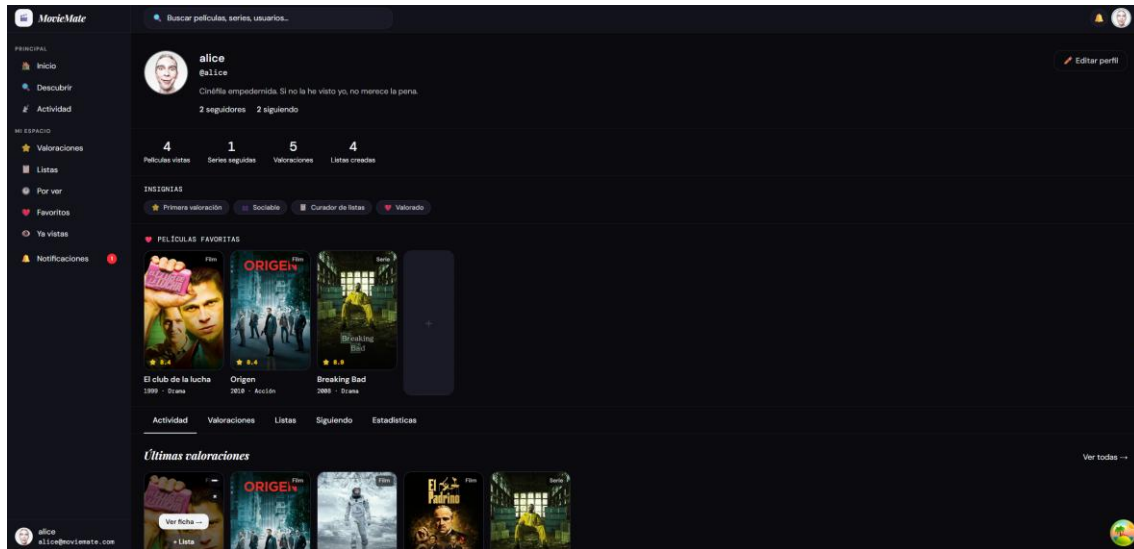


Figura 38. Página de perfil de usuario

C.4 Listas personalizadas

La funcionalidad de listas personalizadas se articula en tres páginas complementarias que comparten la capa de acceso a datos, pero difieren en su presentación y propósito.

`ListsPage (/lists)` muestra la cuadrícula de listas del usuario autenticado como se puede ver en la Figura 39. Cada lista se representa mediante el componente `MyListCard`, que ofrece acceso directo a las acciones de edición (`EditListDialog`, para modificar nombre, descripción y visibilidad) y eliminación. Desde esta misma página se puede crear una nueva lista personalizada con nombre único por usuario.

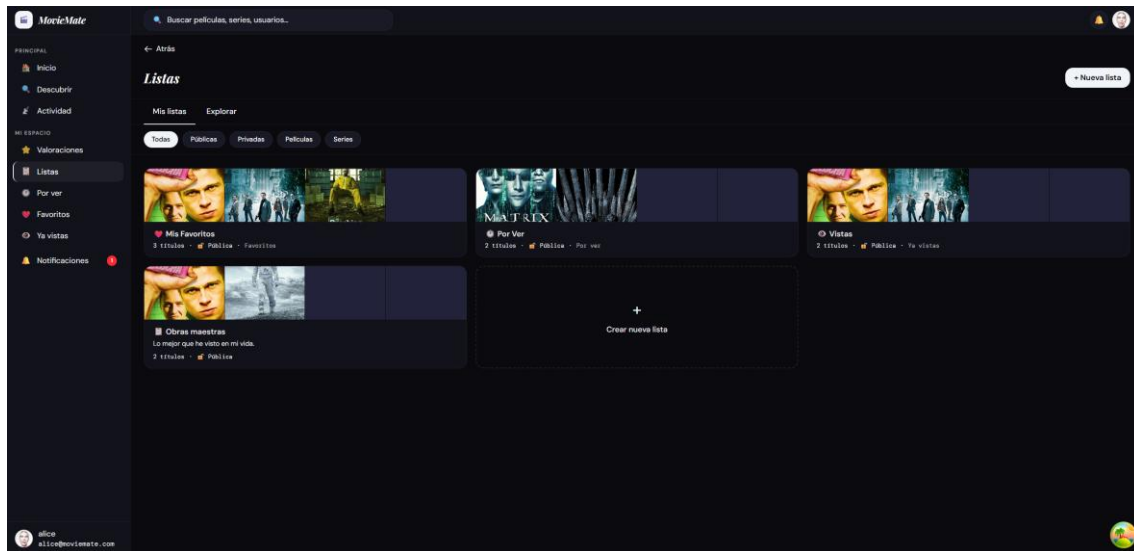


Figura 39. Página de listas personalizadas del usuario

ListDetailPage (/lists/:listId) muestra el contenido completo de una lista concreta como se puede ver en la Figura 40, cuenta con carga instantánea gracias al patrón placeholderData, que recupera los datos preliminares del estado de navegación (location.state) mientras se completa la petición al servidor. En la parte inferior aparece la sección de comentarios, donde cualquier usuario puede dejar valoraciones públicas sobre la lista (entidad ListComment, gestionada por ListCommentService).

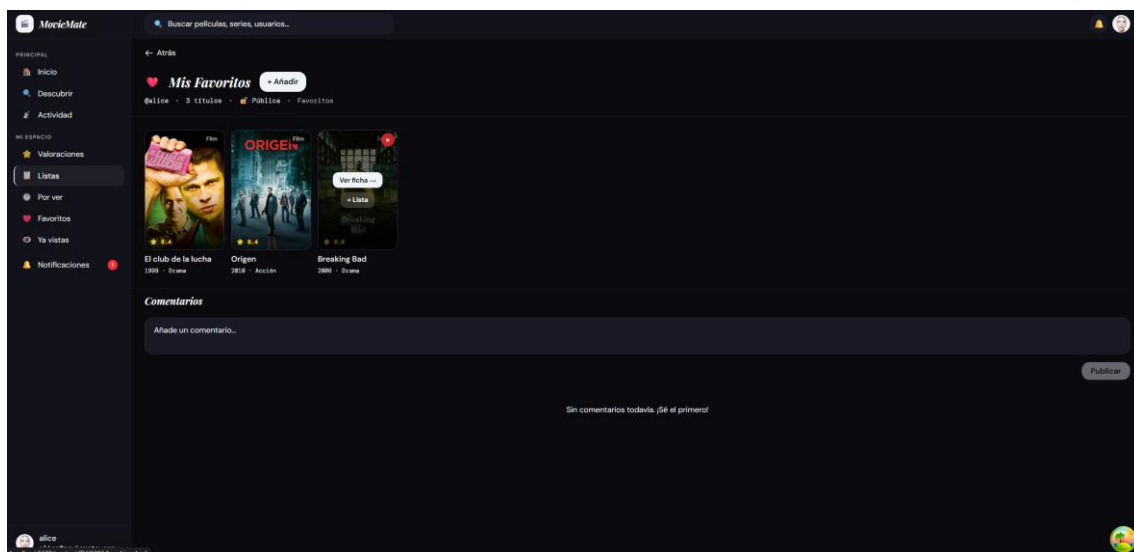


Figura 40. Página de detalle de una lista personalizada

SpecialListPage es una vista compartida para las rutas /watchlist, /favorites y /watched, diferenciadas únicamente por el prop listType (WATCHLIST, FAVORITES, WATCHED). El diálogo AddContentToListDialog permite buscar películas y series directamente desde la interfaz de la lista, deduplicando los resultados para evitar sugerir contenido ya presente en la misma.

C.5 Actividad, notificaciones y comunicación en tiempo real

La página de actividad (`ActivityPage`, ruta `/activity`) muestra un *feed* cronológico de las acciones recientes de los usuarios seguidos (nuevas valoraciones, reseñas publicadas o modificadas, y añadidos a listas) como se observa en la Figura 41. El *feed* se obtiene del *endpoint* `GET /api/activity` y se presenta con paginación.

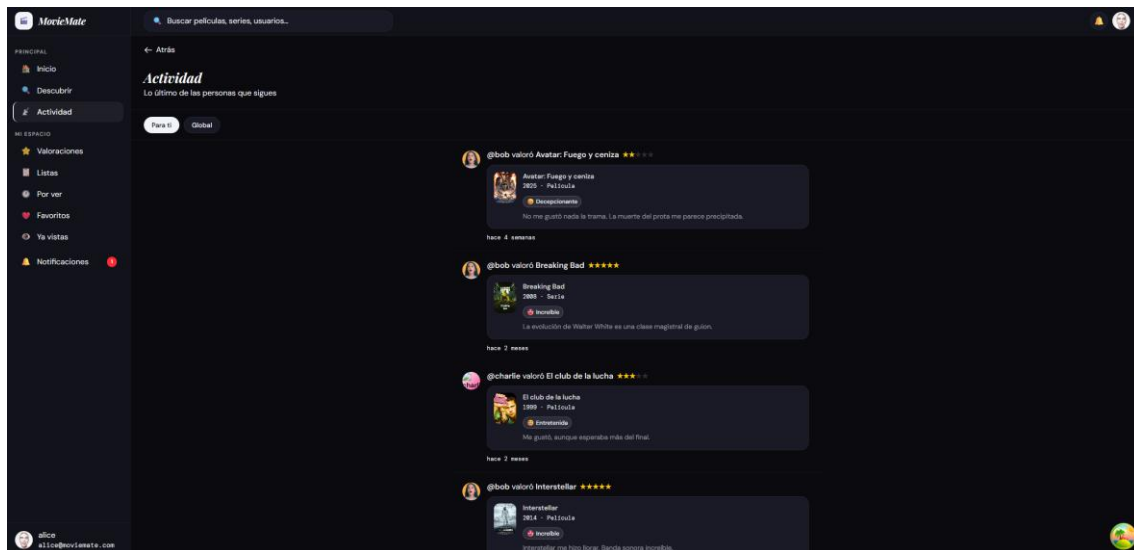


Figura 41. Página de actividad

La página de notificaciones (`NotificationsPage`, ruta `/notifications`) lista todos los eventos dirigidos al usuario autenticado como se ve en la Figura 42, obtenidos mediante `GET /api/users/me/notifications`. La página ofrece cuatro filtros: Todas, No leídas (con contador dinámico), Seguimientos y Me gustas. Cada entrada muestra el avatar del remitente, el texto descriptivo del evento y la marca temporal relativa. Las notificaciones de comentario en reseña o en lista incluyen el título del contenido o el nombre de la lista como sufijo destacado, permitiendo al usuario identificar el recurso afectado de un vistazo. Las notificaciones de nuevo seguidor incorporan un botón de seguimiento directo; las de solicitud de seguimiento permiten aceptar o rechazar sin abandonar la página. La marcación individual como leída se realiza de forma optimista. Un botón global permite marcar todas como leídas. El recuento de notificaciones no leídas se refleja en un *badge* sobre el icono de la barra de navegación, actualizado en tiempo real a través del canal `WebSocket`; al recibir una nueva notificación se antepone a la lista mediante `setQueryData` y se invalida el contador de no leídas.

Para la recepción de notificaciones en tiempo real, el componente raíz `Layout.tsx` inicializa al cargar la aplicación una conexión `WebSocket` mediante `SockJS` y el protocolo `STOMP`. El token `JWT` se incluye en las cabeceras del `frame STOMP CONNECT` y es validado por `WebSocketAuthInterceptor` en el servidor. Cuando se emite un evento, el cliente actualiza la caché de `TanStack Query` con `setQueryData` para añadir la nueva notificación de forma reactiva e inmediata, sin necesidad de una nueva petición `HTTP`, e invalida la consulta para mantener la coherencia a largo plazo.

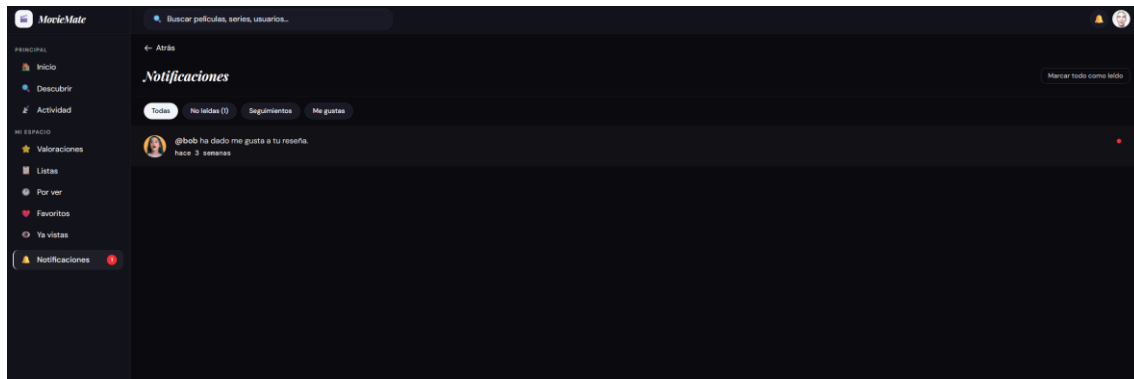


Figura 42. Bandeja de notificaciones de MovieMate

Este enfoque permite ofrecer una experiencia reactiva y en tiempo real, alineada con el comportamiento esperado en aplicaciones sociales modernas.

C.6 Panel de administración y moderación de contenido

En el *front-end*, la funcionalidad de administración se implementa como una página específica accesible únicamente para usuarios con rol de administrador mediante el componente de ruta protegida `AdminRoute`, que verifica que el usuario autenticado tiene el rol adecuado antes de permitir el acceso. El módulo `adminApi` centraliza las llamadas HTTP a los endpoints `/api/admin`, proporcionando operaciones tipadas para listar usuarios, cambiar su rol, banear o desbanear cuentas, consultar valoraciones y comentarios concretos y gestionar el ciclo de vida de los reportes de contenido. Sobre esta capa se construyen varios *hooks* de React Query (`useAdminUsers`, `useChangeRole`, `useBanUser`, `useAdminReports`, entre otros) que encapsulan la lógica de obtención y actualización de datos y se encargan de invalidar las cachés correspondientes tras cada operación.

La página `AdminPage` organiza la interfaz en dos pestañas principales: gestión de usuarios (ver Figura 43) y gestión de reportes (ver Figura 44). En la pestaña de usuarios se presenta una tabla con buscador que permite localizar cuentas por nombre de usuario o correo electrónico y ofrece acciones directas para alternar entre los roles de usuario estándar y administrador, así como para banear o desbanear cuentas, mostrando indicadores visuales del estado y rol de cada usuario. En la pestaña de reportes se muestra un listado filtrable por estado (`PENDING`, `RESOLVED`, `DISMISSED`), en el que el administrador puede desplegar una vista previa de la valoración o comentario denunciado y resolver o desestimar cada reporte mediante los *hooks* `useResolveReport` y `useDismissReport`, actualizando automáticamente la interfaz tras cada acción.

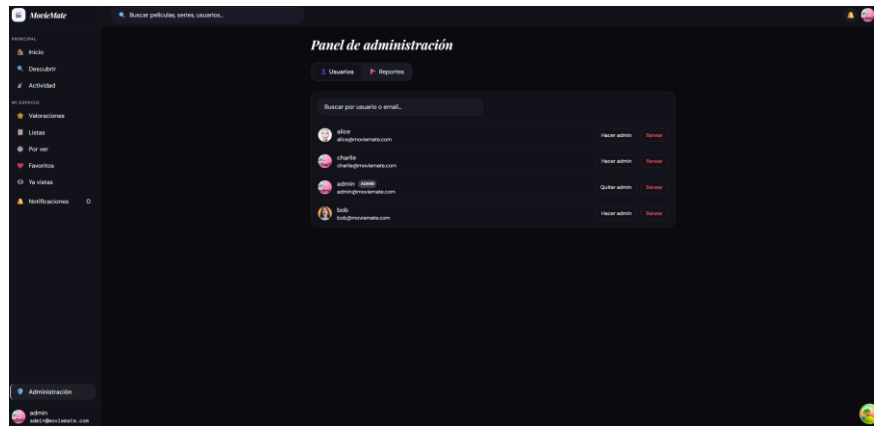


Figura 43. Panel de administración de usuarios

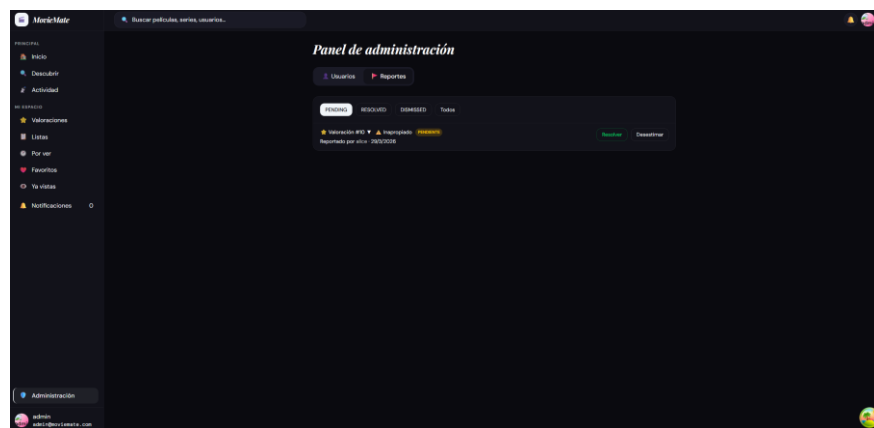


Figura 44. Panel de administración de reportes

Desde la perspectiva del usuario estándar, la creación de reportes se realiza mediante el componente `ReportDialog` (ver Figura 45), que se puede invocar desde distintos puntos de la interfaz asociados a valoraciones y comentarios. Este componente presenta un cuadro de diálogo donde el usuario selecciona el motivo del reporte a partir de una lista predefinida y envía la incidencia al servidor utilizando el `hook useCreateReport`, que delega en la API `reportsApi` para llamar al `endpoint /api/reports`. De este modo, la moderación se integra de forma natural en el flujo de uso de la aplicación, permitiendo que la comunidad señale contenido problemático y que el administrador lo gestione desde un panel centralizado.



Figura 45. Formulario para reportar contenido

Anexo D - Despliegue en Kubernetes con Minikube

Para facilitar la validación local del despliegue orquestado sin necesidad de infraestructura en la nube, se ha utilizado **Minikube**.

El despliegue de MovieMate en **Kubernetes** se describe mediante los siguientes recursos:

- Namespace `moviemate`: agrupa y aísla todos los recursos del proyecto de otros *workloads* del clúster, facilitando la gestión de permisos y la limpieza del entorno.
- ConfigMap: almacena la configuración no sensible de la aplicación, como el nombre de la base de datos, el perfil activo de Spring Boot, el puerto de escucha y la URL del servidor.
- Secret: almacena las credenciales sensibles codificadas en base64, contraseña de PostgreSQL, clave secreta JWT y clave de la API de TMDB. Kubernetes inyecta estos valores como variables de entorno en los contenedores en tiempo de arranque.
- PersistentVolumeClaim (PVC): solicita almacenamiento persistente para los datos de PostgreSQL, garantizando que la información se conserva entre reinicios de pod o actualizaciones del Deployment.
- Deployment de PostgreSQL: define el pod de base de datos, monta el PVC como volumen y carga las credenciales desde el Secret.
- Service ClusterIP para PostgreSQL: expone la base de datos internamente dentro del clúster mediante un nombre de servicio estable, desacoplando la dirección IP del pod de la configuración del back-end.
- Deployment del back-end: define el pod de la aplicación Spring Boot con la imagen Docker generada en el pipeline de CI/CD. Incluye sondas de *liveness* y *readiness* para garantizar que el servicio únicamente recibe tráfico cuando está completamente iniciado y operativo.
- Service NodePort para el back-end: expone el back-end externamente para permitir el acceso desde el navegador durante el desarrollo local con Minikube.

El proceso de despliegue local con Minikube sigue los pasos a continuación:

- Inicialización del clúster: `minikube start` arranca el nodo local y configura el contexto de `kubectl` para apuntar al clúster Minikube.
- Carga de la imagen: `minikube image load moviemate-backend:latest` inyecta la imagen Docker directamente en el registro interno del clúster, evitando la necesidad de un registro externo en el entorno de desarrollo.
- Aplicación de manifiestos: `kubectl apply -f k8s/` aplica todos los recursos definidos en el directorio de manifiestos. Kubernetes gestiona automáticamente el orden de creación respetando las dependencias entre recursos.
- Verificación: `kubectl get pods -n moviemate` permite monitorizar el estado de los pods y confirmar que todos se encuentran en estado *Running*.
- Acceso al servicio: `minikube service backend-service -n moviemate` abre el servicio en el navegador, resolviendo automáticamente la IP del nodo y el puerto `NodePort` asignado.

En el entorno de integración continua, una vez que el *pipeline* construye y publica las imágenes Docker tras un *push* a la rama *main*, estas quedan disponibles en el registro para su despliegue. Sin embargo, el despliegue en el clúster no se realiza de forma automática, sino que se lanza manualmente mediante la ejecución del *workflow* correspondiente.

Este enfoque permite mantener un mayor control sobre el proceso de despliegue, evitando que cambios no deseados se propaguen automáticamente al entorno de ejecución y facilitando la validación previa de las versiones generadas.